Machine Learning

Machine Learning

From linear regression to convolutional neural networks, built from scratch in C++

Erik Pihl

QRTECH ACADEMY
17 October 2026

Machine Learning
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/`.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the demo programs and their reference implementations, the test suites, the finished network, and the solutions to the exercises:

`https://github.com/qrtech-academy/machine-learning`

The C++ in this book is C++17 throughout, compiled with GCC, and uses nothing outside the standard library: every model, layer and network in it is written from scratch. Arm Cortex-M0 and Cortex-M0+ appear in the text as examples of processors without a hardware floating-point unit; they are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with LuaLaTeX.

Typeset from the ten lectures of the course, their exercises, and the two self-assessment papers. This edition: 17 October 2026.

Contents

Preface **vii**

PART I LINEAR REGRESSION AND NEURAL NETWORKS

1 Linear Regression **3**
1.1 Vectors in C++ . . . 3
1.2 Introduction to Machine Learning . . . 5
1.3 Linear Regression . . . 7
1.4 Linear Regression in C++ . . . 12
1.5 Review . . . 16
1.6 Exercises . . . 16

2 Training Order, Precision and Learning Rate **27**
2.1 Randomizing the Training Order . . . 27
2.2 Precision Calculation . . . 32
2.3 Review . . . 37
2.4 Exercises . . . 38

3 Neural Networks **47**
3.1 Neural Networks . . . 47
3.2 Dense Layer Architecture . . . 51
3.3 Review . . . 53
3.4 Exercises . . . 53

4 A Network Built on Interfaces **63**
4.1 Neural Network Architecture . . . 63
4.2 Training Loop . . . 64
4.3 Review . . . 67
4.4 Exercises . . . 68

5 The Dense Layer **83**
5.1 Dense Layer Architecture . . . 83
5.2 From Math to Code . . . 84
5.3 Review . . . 87
5.4 Exercises . . . 87

PART II Convolutional Neural Networks

6 Convolutional Neural Networks **103**
6.1 Why Regular Neural Networks Struggle with Images . . . 103
6.2 Convolutional Neural Networks (CNN) . . . 104
6.3 Review . . . 107
6.4 Exercises . . . 107

7 Convolution, Pooling and Flatten Layers **113**
7.1 Review . . . 114
7.2 Exercises . . . 114

8 The Convolutional Layer **121**
8.1 The CNN Pipeline . . . 121
8.2 Review . . . 125
8.3 Exercises . . . 125

9 The Max Pooling Layer **133**
9.1 Review . . . 133
9.2 Exercises . . . 134

10 The Flatten Layer and the Complete Network **139**
10.1 Course Review . . . 140
10.2 Constraints on Resource-Constrained Targets . . . 141
10.3 Review . . . 144
10.4 Exercises . . . 145

A Worked Solutions **155**
A.1 Chapter 1: Linear Regression . . . 155
A.2 Chapter 3: Neural Networks . . . 159
A.3 Chapter 6: Convolutional Neural Networks . . . 163
A.4 Chapter 10: The Flatten Layer and the Complete Network . . . 167

B The Self-Assessment Papers **173**

C Self-Assessment Paper A **177**

D Self-Assessment Paper B **185**

E Model Answers to Paper A **195**
Question 1: Learning the rules instead of writing them 195
Question 2: The training loop that trains nothing. 197
Question 3: A network trained by hand 199
Question 4: The activation function, and the argument it is given 201
Question 5: The dense layer, written out 202
Question 6: Why an image is not just a long vector. 205
Question 7: A convolutional layer, traced by hand 206
Question 8: Max pooling, and the value it forgets 209
Question 9: The finished network, and the target it has to run on 211

F Model Answers to Paper B **213**
Question 1: The model, and the rate that destroys it 213
Question 2: Order, precision, and knowing when to stop 215
Question 3: Two output nodes 216
Question 4: The network class, and what a stub can prove 218
Question 5: The dense layer, and the mistake only Tanh can catch. 220
Question 6: Padding, sharing, and the kernel's gradient 222
Question 7: A pooling layer that runs and returns true 224
Question 8: Every layer, side by side 227
Question 9: Off the laptop 229

Preface

This book covers the fundamentals of machine learning, and it covers them by building them. Every model in it is implemented from scratch in C++, without an external machine learning library, so that nothing happens inside it that you have not written yourself:

- Linear regression and gradient-based optimization.
- Neural networks: feedforward, backpropagation, and activation functions.
- Convolutional neural networks (CNNs) for image classification: kernels, pooling, and flatten layers.
- Implementing machine learning algorithms from scratch in C++.

Along the way you will train models by hand as well as in software. Every algorithm the book implements is first worked through on paper, with numbers small enough to follow, and then written as code that reproduces those numbers. The code you write adds up to:

- A linear regression model trained with a fixed learning rate, and a second one that picks its own.
- A simple neural network, built on a dense layer written from scratch.
- Convolutional, pooling, and flatten layers, first as simple structs and then as classes behind an interface, wired together into a network that tells four hand-drawn digits apart.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Implement basic machine learning algorithms from scratch in C++.
- Train and evaluate neural networks, both by hand and in software.
- Reason about design choices and limitations.

What you are assumed to know

The subject of this book is **machine learning**, not C++. It introduces `std::vector`, the one container every model in it is built on, in §1.1, and uses the rest of C++17 without

introducing it: classes and structs, public inheritance and pure virtual interfaces, references and `const`, the keywords `explicit`, `noexcept`, `final`, `override` and `[[nodiscard]]`, deleted copy and move operations, and, from Chapter 8 on, `std::unique_ptr` and a factory. You should also be at home in a terminal, compiling a program with `g++` and building one with `make`.

The mathematics asks for less than it may look. Everything the book computes is sums, products and differences, plus the derivatives of the three activation functions it implements, and every one of those derivatives is given. A calculator is all the hand calculations need.

How the book is organised

The book is typeset from a ten-lecture course, and each chapter is one lecture. The course is taught in two parts with a natural break between them, and the book keeps the break: Part I builds up from linear regression to a neural network with a dense layer written from scratch, and Part II builds a convolutional neural network one layer type at a time, closing with a review that ties linear regression, neural networks, and CNNs together.

Chapter	Lecture	Topic
Part I: Linear Regression and Neural Networks		
1	`L01`	Linear regression (part I)
2	`L02`	Linear regression (part II)
3	`L03`	Neural networks (part I)
4	`L04`	Neural networks (part II)
5	`L05`	Dense layer
Part II: Convolutional Neural Networks		
6	`L06`	Convolutional neural networks (part I)
7	`L07`	Convolutional neural networks (part II)
8	`L08`	Convolutional neural networks (part III)
9	`L09`	Convolutional neural networks (part IV)
10	`L10`	Flatten layer and course review

A chapter's sections are its lecture's appendices, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side. Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold the worked solutions to the exercises answered on paper, and the two written papers described below.

What the exercises look like

The exercises come in **sets**. A set builds one program, or one part of a larger one, or works through one calculation, and each exercise in a set adds a piece of it. Every exercise is labelled with its kind:

Kind	What it asks of you
CODE	Write C++, compile it, and check it against the lecture's test suite, which specifies the behaviour the exercise asks for, case by case.
CALCULATION	Work it out on paper: train a model by hand, trace an image through a layer, or count what a network has to store.
REFLECTION	Answer in prose: recognise a layer from its algorithm, or decide whether a claim about one is true, and say why.

The solutions to the Calculation and Reflection exercises are in this book, in Appendix A, worked through step by step, and every such exercise ends with the page its solution is on. Work each one before you turn to it; a hand calculation you have only read teaches you much less than one you got wrong first.

The solutions to the Code exercises are not in this book. They are published in the companion repository, lecture by lecture, and each lecture's README says where. Clone the repository (see *Setting up* below) and you can run a reference program beside your own.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercises after every chapter, the test suite most of them ship with, and the self-assessment exercises in §10.4, each of which checks a component you built or a number you computed beside it.

Appendices B to F hold two four-hour papers with worked solutions, and they check something else: **your theoretical knowledge, on paper, with nothing in front of you.** Nine questions each, mixing theory with code, and every derivation the chapters make asked for without the chapter open beside you.

They exist purely so that you can test your own knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter. Both papers draw on all ten chapters, so sitting one partway through examines material you have not read yet.

Setting up

Everything in this book runs in a Linux terminal: on Linux itself, or on Windows through WSL. You need three things:

- **GCC and G++**, with C++17 support, to compile the programs.

- **Linux or WSL**, to compile and run them in a terminal environment.
- **An editor**; Visual Studio Code is the one the course uses, available at `https://code.visualstudio.com/download`.

Chapter 7 also points to reference implementations of its layers in Python, which run with `python3`. For a small program you want to try without installing anything, an online compiler such as `https://www.onlinegdb.com/online_c_compiler` works too, with the language switched to C++17.

Then clone the companion repository, together with the test framework it keeps as a submodule:

```
git clone --recursive https://github.com/qrtech-academy/machine-learning.git
```

(In a clone made without `--recursive`, run `git submodule update --init` once.) Three commands at its root cover the repository as a whole:

```
make build               # Build every lecture demo that has a Makefile.
make test                # Build and run every test suite that has a solution to run
    ↪ against.
make format-check        # Check the formatting of every C++ and Python file.
```

You write your exercise solutions in the repository, and each chapter's exercises say where. Each lecture's test suite already knows where that is and runs with a single `make`: its tests use nothing but the public interface the exercises specify, so any implementation that follows the specification passes, however it is written on the inside, and a test that fails names the assertion, the two values involved, and the file and line it came from.

PART I

Linear Regression and Neural Networks

Chapters 1 to 5, taught as lectures L01 to L05. A linear regression model, trained by hand and then in software, and taught to stop training on its own and to pick its own learning rate; then neural networks, with feedforward, backpropagation and optimization worked through on paper before a network is built from interfaces. The part ends with a dense layer written from scratch, which is the first layer in the book that actually learns.

CHAPTER 1

Linear Regression

A first model that finds its own rules in the data it is given.

IN THIS CHAPTER

- A brief introduction to `std::vector`, used throughout the book.
- What machine learning is, and how it differs from traditional programming.
- Linear regression: prediction, parameters, error, epochs, and learning rate.
- Training a linear regression model by hand.
- Implementing the linear regression model in software, and training it.

This chapter introduces machine learning through its simplest model. It starts with `std::vector`, the container every model in the book stores its training data and parameters in, and then shows why a traditional program is not well suited to mimicking human intelligence, and how machine learning differs from traditional programming in terms of input, output, and rules. The rest of the chapter is linear regression: the theory behind it, a model trained by hand for one epoch, and a first implementation in C++.

Read §1.1 to §1.4 in order. The worked example in §1.3.3 is the one to follow with a pencil, because the first exercise in §1.6 asks you to do the same by hand for a different line; the rest of the exercises build the model as an interface and a class, with a test suite to check it against.

1.1 Vectors in C++

`std::vector`, used throughout the book to store training data and parameters, is defined in the `<vector>` header:

```
#include <vector>
```

1.1.1 Creating and initializing a vector

Vectors can be created as shown below:

```cpp
// Creates an empty vector that can store integers.
std::vector<int> v1{};

// Creates a floating-point vector initialized with three elements.
std::vector<double> v2{1.0, 2.0, 3.0};

// Creates a byte vector containing 100 fives.
std::vector<std::uint8_t> v3(100U, 5U);
```

1.1.2 Common methods

Method	Description
`push_back(val)`	Adds `val` to the end of the vector
`pop_back()`	Removes the last element
`size()`	Returns the number of elements
`resize(size, val)`	Resizes the vector to `size` elements; new elements are set to `val`
`clear()`	Empties the vector
`empty()`	Returns `true` if the vector is empty
`begin()` / `end()`	Iterators to the beginning/end
`erase(it)`	Removes the element at iterator `it`

1.1.3 Iteration

- Iteration via range-based for loop (recommended):

```cpp
for (const auto& val : values)
{
    std::cout << val << "\n";
}
```

- Iteration via index (C-style):

```cpp
for (std::size_t i{}; i < values.size(); ++i)
{
    std::cout << values[i] << "\n";
}
```

- Iteration via vector iterators:

```cpp
for (auto it = values.begin(); it != values.end(); ++it)
{
    std::cout << *it << "\n";
```

```
}
```

1.1.4 Passing vectors by reference

Vectors should generally be passed by reference to avoid unnecessary copying:

```
void print(const std::vector<double>& values);      // Read-only
void fill(std::vector<double>& values, double val); // Read and write
```

More information on `std::vector` is available at https://en.cppreference.com/w/cpp/container/vector.

1.2 Introduction to Machine Learning

1.2.1 Traditional programming

As an example of a traditional program within an embedded system, temperature can be read as input via a temperature sensor, with rules implemented via a conditional statement to control a ventilation system based on the current room temperature. Below is an example of what a method implementing this could look like in C++, for a method `runVentilation()` in a fictional class named `HardwareController`:

```
/**
 * @brief Run the PWM controlled ventilation system.
 */
void HardwareController::runVentilation() noexcept
{
    constexpr std::int8_t lowerTempLimit{10};
    constexpr std::int8_t upperTempLimit{25};

    constexpr double ventilatorOff{0.0};
    constexpr double ventilatorPwm{0.5};
    constexpr double ventilatorOn{1.0};

    // Read the room temperature with the tempsensor.
    const auto temperature = myTempSensor.read();

    // Disable the ventilator if the temperature is below 10 degrees Celsius.
    if (lowerTempLimit > temperature)
    {
        myVentilator.run(ventilatorOff);
    }
```

```
    // Run the ventilator at 50% if the temperature is between 10 and 25 degrees ↩
    ↪ Celsius.
    else if ((lowerTempLimit <= temperature) && (upperTempLimit >= temperature))
    {
        myVentilator.run(ventilatorPwm);
    }
    // Run the ventilator at 100% if the temperature exceeds 25 degrees Celsius.
    else
    {
        myVentilator.run(ventilatorOn);
    }
}
```

1.2.2 Machine learning

For the ventilation control example above, we could instead create training sets consisting of various temperatures as input and the values `0.0`, `0.5`, and `1.0` as output:

Input	Output
−10	0.0
−5	0.0
0	0.0
5	0.0
10	0.5
15	0.5
20	0.5
25	0.5
30	1.0
35	1.0
40	1.0

1.2.3 Why machine learning?

Mimicking human intelligence with a traditional program would require an enormous number of conditional statements to cover every conceivable decision and combination of conditions, which isn't practically feasible. This approach is called *symbolic AI* and was the dominant paradigm in artificial intelligence before machine learning became widespread. It worked well for complex but well-defined tasks (e.g. chess), but was inadequate for less well-defined tasks such as computer vision, speech recognition, and language translation, where the input can vary widely. Machine learning solves this by letting the machine find the rules itself through training on large amounts of data.

1.3 Linear Regression

One of the most common algorithms in machine learning is linear regression, which in practice means training the machine to detect a linear pattern between input x and output y. From this pattern, a straight line can be derived using the following first-degree equation:

$$y = kx + m,$$

where k is the slope of the line and m is the line's resting value (the value of output y when input x equals zero).

1.3.1 Weight and bias

In machine learning, the terms **weight** and **bias** are often used instead of k- and m-value:

1. Input x is said to be multiplied by a weight k.
2. The model's bias m is its resting value, i.e. the output y when input x equals zero.

This is especially true for neural networks, where a so-called node holds a bias (m-value) and one or more weights (k-values). In practice it's the same thing; the node's inputs x are multiplied by weights k, and the sum $k \cdot x$ of these together with the bias m makes up the node's output y.

Note! Nodes in neural networks are somewhat more complex than that: the sum is passed through an *activation function*, which for example clamps negative sums to zero (ReLU). The bias plays the role a threshold would, shifting where that function's behaviour changes, but the function is applied to every sum unconditionally rather than gating the node on or off. Neural networks are covered from Chapter 3 on.

1.3.2 Training data

Training the model requires training data in the form of input x and corresponding output y. A pair consisting of one input x and one output y makes up a **training set**. The table below shows an example of five training sets, where the relationship between input x and output y can be described by the formula $y = 2x + 1$:

x	y
0	1
1	3
2	5
3	7
4	9

The training data should contain several different training sets, i.e. several pairs of input x and output y. During training, all training sets can be used to train the model over multiple

rounds, preferably in random order so the model doesn't become too familiar with the training data. The number of rounds the training data is used to train the model is called **epochs**.

During training, the predicted output y_p should then be compared against the corresponding reference value from the training data, y_{ref}. The difference between these makes up the deviation δ:

$$\delta = y_{\text{ref}} - y_p,$$

where δ is the deviation between reference value y_{ref} and predicted output y_p.

The goal is for the deviation δ to end up as close to zero as possible, since that's when the model performs best:

$$\delta = 0 \Rightarrow \text{the model performs excellently}$$

If the deviation δ is positive, the predicted value y_p is too low, meaning the model's k- and m-value should be increased:

$$\delta > 0 \Rightarrow k \text{ and } m \text{ should be increased}$$

Likewise, if the deviation δ is negative, the predicted value y_p is too high, meaning the model's k- and m-value should be decreased:

$$\delta < 0 \Rightarrow k \text{ and } m \text{ should be decreased}$$

When there's a deviation, the model's k- and m-value should be adjusted by a certain adjustment amount Δe, which is a factor of the deviation δ as follows:

$$\Delta e = \delta \cdot LR,$$

where LR is the learning rate, which should be adjusted based on how well the model performs. The higher the learning rate, the more strongly the k- and m-value are adjusted on deviation. Too high a learning rate can, however, cause too large an adjustment per k- and m-value. As a starting point, LR can be set to around 1%, which corresponds to 0.01 in the calculations. This value should then be tuned together with the number of epochs (the number of training rounds over the current set of training sets).

The model's m-value should be increased by the adjustment amount Δe:

$$m = m + \Delta e$$

The model's k-value should be increased by the adjustment amount Δe multiplied by the current input x. This means that when x equals zero (when only the m-value determines output y), the k-value isn't adjusted on deviation, only the m-value. At the same time, the higher the x-value, the more the k-value is adjusted for the given input x:

$$k = k + \Delta e \cdot x$$

1.3.3 Training a regression model by hand

A regression model is to be trained using the five training sets defined by the formula $y = 2x + 1$:

x	y
0	1
1	3
2	5
3	7
4	9

Assume the model's bias (m-value) and weight (k-value) are zero at the start:

$$\begin{cases} k = 0 \\ m = 0 \end{cases}$$

Run training for one epoch with a learning rate LR of 10%:

$$LR = 0.1$$

Then perform prediction for input consisting of all integers in the range $[-5, 5]$.

1.3.4 Solution

All calculations are shown below. We run training for each training set one at a time.

Training set 1

From the first training set we get input $x = 0$ and reference value $y_{\text{ref}} = 1$:

$$\begin{cases} x = 0 \\ y_{\text{ref}} = 1 \end{cases}$$

Since the model's parameters are zero at the start, the predicted output y_p is zero:

$$y_p = k \cdot x + m = 0 \cdot 0 + 0 = 0$$

The deviation δ is therefore one:

$$\delta = y_{\text{ref}} - y_p = 1 - 0 = 1$$

For a learning rate LR of 10%, the adjustment amount Δe is 0.1:

$$\Delta e = \delta \cdot LR = 1 \cdot 0.1 = 0.1$$

The model's m-value is increased directly by the adjustment amount Δe:

$$m = m + \Delta e = 0 + 0.1 = 0.1$$

The model's k-value is increased by Δe multiplied by x, which when $x = 0$ results in no change:

$$k = k + \Delta e \cdot x = 0 + 0.1 \cdot 0 = 0$$

After the first round of training:

$$\begin{cases} k = 0 \\ m = 0.1 \end{cases}$$

Training set 2

$$\begin{cases} x = 1 \\ y_{\text{ref}} = 3 \end{cases}$$
$$y_p = k \cdot x + m = 0 \cdot 1 + 0.1 = 0.1$$
$$\delta = y_{\text{ref}} - y_p = 3 - 0.1 = 2.9$$
$$\Delta e = \delta \cdot LR = 2.9 \cdot 0.1 = 0.29$$
$$m = m + \Delta e = 0.1 + 0.29 = 0.39$$
$$k = k + \Delta e \cdot x = 0 + 0.29 \cdot 1 = 0.29$$

After the second round of training:

$$\begin{cases} k = 0.29 \\ m = 0.39 \end{cases}$$

Training set 3

$$\begin{cases} x = 2 \\ y_{\text{ref}} = 5 \end{cases}$$
$$y_p = k \cdot x + m = 0.29 \cdot 2 + 0.39 = 0.97$$
$$\delta = y_{\text{ref}} - y_p = 5 - 0.97 = 4.03$$
$$\Delta e = \delta \cdot LR = 4.03 \cdot 0.1 = 0.403$$
$$m = m + \Delta e = 0.39 + 0.403 = 0.793$$
$$k = k + \Delta e \cdot x = 0.29 + 0.403 \cdot 2 = 1.096$$

After the third round of training:

$$\begin{cases} k = 1.096 \\ m = 0.793 \end{cases}$$

Notice the parameters are starting to approach the desired values $(k = 2, m = 1)$.

Training set 4

$$\begin{cases} x = 3 \\ y_{\text{ref}} = 7 \end{cases}$$
$$y_p = k \cdot x + m = 1.096 \cdot 3 + 0.793 = 4.081$$
$$\delta = y_{\text{ref}} - y_p = 7 - 4.081 = 2.919$$

Notice the deviation δ has now started decreasing for the first time.

$$\Delta e = \delta \cdot LR = 2.919 \cdot 0.1 = 0.2919$$
$$m = m + \Delta e = 0.793 + 0.2919 = 1.0849$$
$$k = k + \Delta e \cdot x = 1.096 + 0.2919 \cdot 3 = 1.9717$$

After the fourth round of training:

$$\begin{cases} k = 1.9717 \\ m = 1.0849 \end{cases}$$

Notice the parameters are very close to the desired values ($k = 2, m = 1$).

Training set 5

$$\begin{cases} x = 4 \\ y_{\text{ref}} = 9 \end{cases}$$
$$y_p = k \cdot x + m = 1.9717 \cdot 4 + 1.0849 = 8.9717$$
$$\delta = y_{\text{ref}} - y_p = 9 - 8.9717 = 0.0283$$
$$\Delta e = \delta \cdot LR = 0.0283 \cdot 0.1 = 0.00283$$
$$m = m + \Delta e = 1.0849 + 0.00283 = 1.08773$$
$$k = k + \Delta e \cdot x = 1.9717 + 0.00283 \cdot 4 = 1.98302$$

After the fifth round of training:

$$\begin{cases} k = 1.98302 \\ m = 1.08773 \end{cases}$$

Notice that after just one epoch, the regression model's parameters have already landed very close to the desired values ($k = 2, m = 1$). Normally, many more epochs are run, e.g. 1,000 to 10,000. At the same time, the learning rate tends to be lower, resulting in smaller parameter adjustments per epoch.

After training, the regression model predicts according to the following formula:

$$y_p = 1.98302 \cdot x + 1.08773$$

1.3.5 Verification

The table below shows predicted output y_p and reference values y_{ref} for input x in the range $[-5, 5]$. Predicted output has been rounded to two decimal places.

x	y_p	y_{ref}
−5	−8.83	−9
−4	−6.84	−7
−3	−4.86	−5
−2	−2.88	−3
−1	−0.90	−1
0	1.09	1
1	3.07	3
2	5.05	5
3	7.04	7
4	9.02	9
5	11.00	11

Notice that the predicted output y_p ends up close to the desired output y_{ref} in every case, after training for just a single epoch!

1.4 Linear Regression in C++

Below is a minimal implementation of linear regression in C++ using a struct `LinReg` that:

- Holds the model's parameters `bias` and `weight`.
- Predicts with the method `predict()`.
- Trains with the method `train()`.

Note! An alias `Matrix1d` is used as a substitute for `std::vector<double>`.
The full implementation is shown below:

```
/**
 * @brief Minimalistic linear regression example.
 */
#include <algorithm>
#include <cstdint>
#include <cstdio>
#include <exception>
#include <vector>

namespace
{
/** One-dimensional matrix. */
```

```cpp
using Matrix1d = std::vector<double>;

/**
 * @brief Linear regression structure.
 */
struct LinReg
{
    /** Bias value. */
    double bias;

    /** Weight value. */
    double weight;

    /**
     * @brief Predict with the given linear regression model.
     *
     * @param[in] input Input value.
     *
     * @return The predicted value.
     */
    double predict(const double input) const noexcept { return weight * input + bias; }

    /**
     * @brief Train the given linear regression model with given training sets.
     *
     * @param[in] trainIn Input values. Size must be greater than 0.
     * @param[in] trainOut Output values. Size must be greater than 0.
     * @param[in] epochCount Number of epochs to train. Must be greater than 0.
     * @param[in] learningRate Learning rate (default = 1 %). Must be in range (0.0, 1.0).
     *
     * @return Always true; invalid arguments terminate the program rather than being
     *         reported, since this model has no way to recover from them.
     */
    bool train(const Matrix1d& trainIn, const Matrix1d& trainOut,
               const std::uint32_t epochCount, const double learningRate = 0.01) noexcept
    {
        // Check input parameters and training set count, terminate if invalid.
        const auto setCount = static_cast<std::uint32_t>(std::min(trainIn.size(), trainOut.size()));
```

```
        if (0U == setCount)
        {
            std::fprintf(stderr, "Invalid training data: no training sets provided!\n");
            std::terminate();
        }
        if (0U == epochCount)
        {
            std::fprintf(stderr, "Invalid epoch count: must train for at least one epoch!\n");
            std::terminate();
        }
        if ((0.0 >= learningRate) || (1.0 <= learningRate))
        {
            std::fprintf(stderr, "Invalid learning rate %g: must be in range (0.0, 1.0)!\n",
                learningRate);
            std::terminate();
        }

        // Train the model for the given number of epochs.
        for (std::uint32_t epoch{}; epoch < epochCount; ++epoch)
        {
            // Iterate through all training sets.
            for (std::uint32_t i{}; i < setCount; ++i)
            {
                const auto input  = trainIn[i];
                const auto output = trainOut[i];

                // Perform prediction and calculate the error.
                const auto prediction = predict(input);
                const auto error      = output - prediction;

                // Adjust the parameters in accordance with the error.
                bias   += error * learningRate;
                weight += error * learningRate * input;
            }
        }
        return true;
    }
};
} // namespace

/**
```

```
 * @brief Train and predict with a simple linear regression model.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    constexpr std::uint32_t epochCount{100U};
    constexpr double learningRate{0.1};

    // Create linear regression model to predict y = 3x + 1.
    const Matrix1d trainIn{0.0, 1.0, 2.0, 3.0, 4.0};
    const Matrix1d trainOut{1.0, 4.0, 7.0, 10.0, 13.0};
    LinReg linReg{};

    // Train the model, return -1 on failure.
    if (!linReg.train(trainIn, trainOut, epochCount, learningRate))
    {
        std::printf("Training failed!\n");
        return -1;
    }
    // Predict and print result on success.
    for (const auto& input : trainIn)
    {
        const auto prediction = linReg.predict(input);
        std::printf("x = %g, ypred = %g\n", input, prediction);
    }
    return 0;
}
```

The model above is intentionally minimal. It could be extended with, for example:

- A special case for `x = 0`, where the bias is set directly to the reference value (`bias = output`). Worth knowing that this is a shortcut, not a better learning rule: it overwrites a trained parameter from a single sample, so with noisy data, or with more than one training set at `x = 0`, the final bias is just whichever of them was visited last. The ordinary update rule already handles `x = 0` correctly, since k has no effect on a prediction made with an input of zero.
- Randomizing the training order to avoid the model becoming too familiar with the training data.
- Computing the mean error per epoch to track the model's training progress.

1.5 Review

What you should be able to do now

After this chapter, you should:

- Know how `std::vector` is used in C++.
- Know why a traditional program isn't well suited for mimicking human intelligence.
- Know how machine learning differs from traditional programming in terms of input, output, and rules.
- Be able to train simple linear regression models by hand.
- Be able to implement training of a linear regression model in software.

Questions to test yourself

You should be able to explain:

1. What's the difference between input, rules, and output in a traditional program compared to a machine learning system?
2. What's meant by an epoch when training a machine learning model?
3. How are the parameters in a linear regression model used to compute a prediction for a given input?
4. Why is an interface used for the linear regression model? What does it enable?
5. What happens to the model's weight and bias value when input x equals zero during training?
6. How does the learning rate affect training? What happens if it's set too high or too low?

Looking ahead

Chapter 2 continues the implementation of regression models in software:

- Randomizing the training order.
- Computing the precision of a trained model.

1.6 Exercises

How to work these. Set 1 is a calculation: work it on paper, with the worked example in §1.3.3 as your model. Set 2 is one program, written in the lecture's exercises directory, `lectures/L01/exercises` in the companion repository, and built with a Makefile you write yourself.
How to check your work. The worked solution to Set 1 is in Appendix A. Set 2 comes with a test suite, already in `lectures/L01/exercises/test`, which Exercise 1.15 runs. It tests the directory above it by default; `make ML_DIR=<path>` points it at a tree of your own elsewhere.

> The solution to Set 2 is published in the companion repository, in `lectures/L01/exercises`, with a README that walks through it. Try each exercise yourself before looking at it.

EXERCISE SET 1

Linear Regression by Hand

Exercise 1.1 Training by hand

CALCULATION

A regression model is to be trained using the five training sets defined by the formula $y = 3x + 2$:

x	y
0	2
1	5
2	8
3	11
4	14

Assume the model's bias (m-value) and weight (k-value) are zero at the start:

$$\begin{cases} k = 0 \\ m = 0 \end{cases}$$

Run training for one epoch with a learning rate LR of 10%:

$$LR = 0.1$$

Use the same update rule as the worked example in §1.3.3, i.e. `m += δ * LR` and `k += δ * LR * x` for every training set, including the one at `x = 0`. The `x = 0` shortcut introduced in Exercise 1.13 below is a different rule and gives different answers here.

Then perform prediction for input consisting of all integers in the range $[-5, 5]$.

The worked solution is on page 155.

EXERCISE SET 2

Implementing Linear Regression in C++

You'll implement an interface and an accompanying class for a linear regression model in C++.

Exercise 1.2 Directory structure

CODE

You'll write your code in this lecture's exercises directory, `lectures/L01/exercises`. Build up the following structure inside it:

```
exercises/
├── include/
│   └── ml/
│       ├── lin_reg/
│       │   ├── fixed.hpp
│       │   └── interface.hpp
│       └── types.hpp
├── source/
│   ├── ml/
│   │   └── lin_reg/
│   │       └── fixed.cpp
│   └── main.cpp
├── test/                <- already here, see Exercise 1.15
└── Makefile
```

The `test` directory is already in place; Exercise 1.15 covers running it.

Exercise 1.3 Makefile

CODE

Write a makefile that compiles and builds a program with the file structure above via the `make` command.

Test-run the program by adding an empty main function to `main.cpp`.

Exercise 1.4 main.cpp

CODE

Paste the contents below into the corresponding file:

```cpp
/**
 * @file Application entry point.
 */
#include <cstdio>

#include "ml/lin_reg/fixed.hpp"
#include "ml/types.hpp"
```

```
namespace
{
/**
 * @brief Evaluate the model with the given inputs.
 *
 * @param[in] linReg Model to evaluate.
 * @param[in] inputs Input values for the evaluation.
 */
void evaluateModel(const ml::lin_reg::Interface& linReg, const ml::Matrix1d& inputs)
    ↪ noexcept
{
    for (const auto& input : inputs)
    {
        std::printf("Input: %g, ", input);
        std::printf("prediction: %g\n", linReg.predict(input));
    }
}
} // namespace

/**
 * @brief Train and predict with a linear regression model.
 *
 * @return 0 if the model was trained, -1 otherwise.
 */
int main()
{
    constexpr std::size_t epochCount{1000U};
    constexpr double learningRate{0.1};

    // Create linear regression model holding five training sets (y = 3x + 2).
    const ml::Matrix1d trainIn{0.0, 1.0, 2.0, 3.0, 4.0};
    const ml::Matrix1d trainOut{2.0, 5.0, 8.0, 11.0, 14.0};
    ml::lin_reg::Fixed linReg{trainIn, trainOut};

    // Train the model, print the results on success.
    const auto trained = linReg.train(epochCount, learningRate);

    if (trained) { evaluateModel(linReg, trainIn); }
    else { std::fprintf(stderr, "Training failed!\n"); }
    return trained ? 0 : -1;
}
```

Exercise 1.5 Matrix alias

CODE

In the header file `ml/types.hpp`, add a namespace named `ml`.

In this namespace, implement the following aliases:

- `Matrix1d`: alias for one-dimensional floating-point vectors (`std::vector<double>`).
- `Matrix2d`: alias for two-dimensional floating-point vectors (`std::vector<Matrix1d>`).
- `Matrix3d`: alias for three-dimensional floating-point vectors (`std::vector<Matrix2d>`).

Exercise 1.6 Interface

CODE

In the header file `ml/lin_reg/interface.hpp`, add a namespace named `ml::lin_reg`.

In this namespace, implement an interface named `Interface` with the following public methods:

- **`~Interface()`:**
  - The interface's destructor.
  - Should be marked `default`, `virtual`, and `noexcept`.
- **`predict()`:**
  - Should perform a prediction.
  - Should take a single floating-point argument named `input`.
  - Should return the predicted value as a floating-point number.
  - Should be pure virtual (`= 0`).
  - Should be marked `const` and `noexcept`.

Exercise 1.7 Subclass: Fixed

CODE

In the header file `ml/lin_reg/fixed.hpp`, add the namespace `ml::lin_reg`. Implement a subclass named `Fixed` that inherits `Interface` via public inheritance. The class shouldn't be inheritable further, i.e. it should be marked `final`.

The class is named `Fixed` because it's trained with a fixed learning rate. An adaptive-learning-rate variant, `Adaptive`, follows the same overall structure but adjusts its learning rate dynamically during training.

The class should have the following public methods:

- **`Fixed()`:**
  - The class's only implemented constructor.
  - Should take the following arguments:
    - `trainIn`: reference to a read-only vector of floating-point numbers (training data, input).
    - `trainOut`: reference to a read-only vector of floating-point numbers (training data, output).
  - Should be marked `explicit` and `noexcept`.

- **`~Fixed()`:**
  - Destructor overriding the interface's destructor.
  - Should be marked `default`, `noexcept`, and `override`.
- **`predict()`:**
  - Overrides the corresponding method in the interface.
  - Should be marked `const`, `noexcept`, and `override`.
- **`train()`:**
  - Trains the model with the arguments:
    - `epochCount`: number of epochs to train (unsigned integer).
    - `learningRate`: learning rate as a floating-point number. Default value: `0.01` (1%).
  - Returns `false` if `epochCount` is 0, or if `learningRate` is outside the range `(0.0, 1.0)`. Otherwise returns `true` once training is complete.
  - Should be marked `noexcept`.

The class should have the following private method:

- **`optimize()`:**
  - Arguments:
    - `input`: input value (x) as a floating-point number.
    - `output`: reference value (y_{ref}) as a floating-point number.
    - `learningRate`: the learning rate (LR) as a floating-point number.
  - Returns nothing.
  - Should be marked `noexcept`.

Exercise 1.8 Removed constructors and operators CODE

Delete the class's default constructor, copy and move constructors, and the corresponding operators.

Exercise 1.9 Private member variables CODE

Add the following private member variables to `Fixed`:

- **`myTrainIn`:**
  - Reference to the training data's input points.
  - Initialized via the constructor.
  - Should be marked `const`.
- **`myTrainOut`:**
  - Reference to the training data's output points.
  - Initialized via the constructor.

  - Should be marked `const`.

- **`mySetCount`:**
  - The number of complete training sets, i.e. `std::min(trainIn.size(), trainOut.size())` (`std::min` is available in `<algorithm>`).
  - Training data holding more inputs than outputs, or the other way around, has no complete set beyond the shorter of the two vectors, so the surplus values are unusable.
  - Initialized via the constructor.
  - Should be an unsigned integer (`std::size_t`, from `<cstddef>`) and marked `const`.
- **`myBias`:**
  - The model's bias value (the m-value in $y = kx + m$).
  - Should be a floating-point type.
- **`myWeight`:**
  - The model's weight value (the k-value in $y = kx + m$).
  - Should be a floating-point type.

Exercise 1.10 Constructor

CODE

Implement the constructor in `source/ml/lin_reg/fixed.cpp`:

- Initialize all member variables.
- Set `mySetCount` to the number of complete training sets, i.e. the smaller of the two vectors' sizes.
- `myBias` and `myWeight` must both be initialized to `0.0`, so that the model predicts `0` for every input until it has been trained.
- Print an error message to `stderr` and call `std::terminate()` (from `<exception>`) if `mySetCount` is 0. A model without training data can never be trained, and since the constructor can't return a failure code, there's no way to report the problem to the caller.

Exercise 1.11 Prediction

CODE

Implement the method `predict()` in `source/ml/lin_reg/fixed.cpp`.

Return a prediction according to the formula

$$y = kx + m,$$

where:

- y is the prediction returned.
- k is the model's weight (`myWeight`).
- x is the input (`input`).
- m is the model's bias (`myBias`).

Exercise 1.12 Training

CODE

Implement the method `train()` in `source/ml/lin_reg/fixed.cpp`:

- Return `false` immediately if `epochCount` is 0, or if `learningRate` is outside the range `(0.0, 1.0)`:
  - A learning rate of `0.0` or less can never improve the model.
  - A learning rate of `1.0` or more corrects by at least the full error each step, which makes training oscillate or diverge instead of converging.
- Missing training data is already handled by the constructor, so `train()` doesn't need to check for it.
- Train the model for the given number of epochs.
- During each epoch, iterate through all `mySetCount` training sets. Perform optimization by calling the method `optimize()` with the current training set (obtained from the vectors, e.g. via index).
- Return `true` after training is complete.

Exercise 1.13 Optimization

CODE

Implement the method `optimize()` in `source/ml/lin_reg/fixed.cpp`:

- If `input == 0`: set `myBias = output` directly and return, leaving `myWeight` untouched. Since $y = k \cdot 0 + m = m$, the reference value *is* the bias, and no value of k changes a prediction made with an input of zero.
- Otherwise:
  - Compute a prediction with the given input: `prediction = myWeight * input + myBias`.
  - Compute the error as the difference between the reference value and the prediction: `error = output - prediction`.
  - Adjust the model's parameters based on the error:
    - `myBias = myBias + error * learningRate`
    - `myWeight = myWeight + error * learningRate * input`

Exercise 1.14 Compiling and running CODE

Compile the program and make sure you get the following output:

```
Input: 0, prediction: 2
Input: 1, prediction: 5
Input: 2, prediction: 8
Input: 3, prediction: 11
Input: 4, prediction: 14
```

The model has been trained on the same five training sets you trained by hand in Exercise 1.1, so this output is what $y = 3x + 2$ looks like once the model has found the line on its own.

Exercise 1.15 Running the tests CODE

A ready-made test suite for the model is available in `lectures/L01/exercises/test`. It checks the behaviour specified above: the initial parameter values, the argument validation in `train()`, the parameter updates made by `optimize()`, and a handful of complete training runs.

There's nothing to copy: `test` already sits alongside the code you just wrote. Build and run it from the exercises directory:

```
make -C test
```

The test framework is checked out as a submodule at the root of the repository. If `libs/test` is empty, fetch it once with `git submodule update --init`.

All 15 test cases should pass. When one fails, the output names the assertion that failed, the two values involved, and the file and line it came from. Use it to find the part of the specification your implementation doesn't match yet.

The test suite's README, `lectures/L01/exercises/test/README.md`, has more information, including the one piece of behaviour it deliberately doesn't cover.

Exercise 1.16 Tuning the training CODE

The `main.cpp` you pasted trains for 1000 epochs at a learning rate of 10%, which is far more work than these five training sets need. Tune the two constants at the top of `main()`, i.e. lower `epochCount` and raise `learningRate`, and find the fewest epochs that still reproduce the output in Exercise 1.14 exactly.

Change one constant at a time and rebuild with `make` after each change. A few things worth knowing while you experiment:

- `learningRate` must stay inside the range `(0.0, 1.0)`, otherwise `train()` returns `false` and the program prints `Training failed!` instead of any predictions.

- Don't expect the epoch count to drop smoothly as the learning rate rises. There's a stretch where a higher learning rate needs *more* epochs, not fewer, because each pass overshoots the line and the next has to correct back.
- Raise the learning rate far enough and training stops converging altogether: the predictions grow by orders of magnitude per epoch and print as values like `-2.60711e+06`, or as `inf` and `nan` once the rate is higher still. Find roughly where that starts. It's well below the 1.0 the interface allows, which is why the limit in `train()` is an outer bound rather than a recommendation.
- The predictions are printed with `%g`, which shows six significant digits. Matching the output in Exercise 1.14 therefore means "correct to six digits", not "exact". Print with `%.10f` instead if you want to see the error that's left.
- The test suite in Exercise 1.15 doesn't use `main.cpp`, so it keeps passing no matter which constants you settle on.

It can be done in well under ten epochs. Note down the pair you end up with, along with roughly where training started to diverge.

Keep the pair you settled on in `main.cpp` here, but set `epochCount` back to `1000` and `learningRate` back to `0.1` when you carry the code forward into Chapter 2. That chapter teaches the model to stop training on its own once it's accurate enough, which only shows as a difference when the model is given more epochs than it needs.

CHAPTER 2

Training Order, Precision and Learning Rate

A training loop that knows when to stop, and a model that picks its own learning rate.

In this chapter

- Further implementation of regression models in software, including randomizing the training order and computing precision.
- A second regression model, which adapts its own learning rate while training.
- Concepts and implementation techniques that prepare you for the neural networks of Chapter 3.

This chapter extends the linear regression implementation from Chapter 1 in two ways: first with a randomized training order, so the model no longer sees its training sets in the same sequence every epoch, and then with a precision calculation, which tells the model when it is sufficiently trained and lets it stop long before the epochs run out. The exercises then use that same precision to build a second model, one that raises or lowers its own learning rate depending on how much it is still improving.

Read §2.1 and §2.2 in order; each extends the struct from §1.4 one step further. The exercises in §2.4 carry the class you wrote in Chapter 1 forward through the same two steps, and then add the adaptive model beside it.

2.1 Randomizing the Training Order

If the model is always trained in the same order, it risks becoming too fitted to that particular sequence; it learns the pattern in the ordering rather than in the data. By shuffling the training order every epoch, the model is exposed to the data in varying order, which produces more robust training and stops the parameters from settling into a rhythm dictated by the ordering. For a linear model there are no local minima to get stuck in, since its squared-error surface has exactly one minimum; that benefit arrives with the deeper, non-linear networks from

Chapter 3 onward, where the same shuffling helps the parameters escape poor regions of the loss surface.

Below is an extension of the regression struct from §1.4 with randomized training order. The struct's `train()` method:

- Creates an index vector for the training sets.
- Shuffles the index vector randomly before each epoch.
- Iterates through the training sets in the randomized order.

Note! In addition to `Matrix1d`, the alias `MatrixU32` is used as a substitute for `std::vector<std::uint32_t>`.

The full implementation is shown below:

```
/**
 * @brief Linear regression with randomized training order.
 */
#include <algorithm>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <exception>
#include <vector>

namespace
{
/** One-dimensional matrix. */
using Matrix1d = std::vector<double>;

/** Unsigned 32-bit integer matrix. */
using MatrixU32 = std::vector<std::uint32_t>;

/**
 * @brief Linear regression structure.
 */
struct LinReg
{
    /** Bias value. */
    double bias;

    /** Weight value. */
    double weight;

    /**
```

```
 * @brief Predict with the given linear regression model.
 *
 * @param[in] input Input value.
 *
 * @return The predicted value.
 */
double predict(const double input) const noexcept { return weight * input + bias; }

/**
 * @brief Train the given linear regression model with given training sets.
 *
 * @param[in] trainIn Input values. Size must be greater than 0.
 * @param[in] trainOut Output values. Size must be greater than 0.
 * @param[in] epochCount Number of epochs to train. Must be greater than 0.
 * @param[in] learningRate Learning rate (default = 1 %). Must be in range (0.0, 1.0).
 *
 * @return Always true; invalid arguments terminate the program rather than being
 *         reported, since this model has no way to recover from them.
 */
bool train(const Matrix1d& trainIn, const Matrix1d& trainOut,
           const std::uint32_t epochCount, const double learningRate = 0.01) noexcept
{
    // Check input parameters and training set count, terminate if invalid.
    const auto setCount = static_cast<std::uint32_t>(std::min(trainIn.size(), trainOut.size()));

    if (0U == setCount)
    {
        std::fprintf(stderr, "Invalid training data: no training sets provided!\n");
        std::terminate();
    }
    if (0U == epochCount)
    {
        std::fprintf(stderr, "Invalid epoch count: must train for at least one epoch!\n");
        std::terminate();
    }
    if ((0.0 >= learningRate) || (1.0 <= learningRate))
    {
```

```
            std::fprintf(stderr, "Invalid learning rate %g: must be in range (0.0, ↩
    ↪ 1.0)!\n",
                learningRate);
            std::terminate();
        }

        // Create train order matrix.
        MatrixU32 trainOrder{trainOrderMatrix(setCount)};

        // Train the model for the given number of epochs.
        for (std::uint32_t epoch{}; epoch < epochCount; ++epoch)
        {
            // Randomize training order each epoch.
            shuffle(trainOrder);

            // Iterate through all training sets in randomized order.
            for (const auto& i : trainOrder)
            {
                const auto input  = trainIn[i];
                const auto output = trainOut[i];

                // Perform prediction and calculate the error.
                const auto prediction = predict(input);
                const auto error      = output - prediction;

                // Adjust the parameters in accordance with the error.
                bias   += error * learningRate;
                weight += error * learningRate * input;
            }
        }
        return true;
    }

private:
    // ------------------------------------------------------------------------
    static MatrixU32 trainOrderMatrix(const std::uint32_t setCount) noexcept
    {
        MatrixU32 order(setCount);
        for (std::uint32_t i{}; i < order.size(); ++i) { order[i] = i; }
        return order;
    }

    // ------------------------------------------------------------------------
    static void shuffle(MatrixU32& order) noexcept
```

```
    {
        for (std::uint32_t i{}; i < order.size(); ++i)
        {
            // Swap values at index r and i.
            const auto r    = static_cast<std::uint32_t>(std::rand() % order.size())
    ↪ ;
            const auto temp = order[i];
            order[i]        = order[r];
            order[r]        = temp;
        }
    }
};
} // namespace
```

Note that drawing `r` from the whole range on every pass makes some orderings more likely than others, so this shuffle isn't uniform. It's plenty for varying the training order, which is all it's used for here. Drawing from the remaining range instead, `i + std::rand() % (order.size() - i)`, gives a uniform shuffle for the same effort; that's the Fisher-Yates algorithm, and `std::shuffle()` from `<algorithm>` is what you'd normally reach for outside a course like this.

```
/**
 * @brief Train and predict with a simple linear regression model.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    constexpr std::uint32_t epochCount{100U};
    constexpr double learningRate{0.1};

    // Initialize the random generator.
    std::srand(std::time(nullptr));

    // Create linear regression model to predict y = 3x + 1.
    const Matrix1d trainIn{0.0, 1.0, 2.0, 3.0, 4.0};
    const Matrix1d trainOut{1.0, 4.0, 7.0, 10.0, 13.0};
    LinReg linReg{};

    // Train the model, return -1 on failure.
    if (!linReg.train(trainIn, trainOut, epochCount, learningRate))
    {
        std::printf("Training failed!\n");
```

```
        return -1;
    }
    // Predict and print result on success.
    for (const auto& input : trainIn)
    {
        const auto prediction = linReg.predict(input);
        std::printf("x = %g, ypred = %g\n", input, prediction);
    }
    return 0;
}
```

The model above builds on §1.4 with a randomized training order. The next extension is computing the mean error per epoch to track the model's training progress, covered next.

2.2 Precision Calculation

Without precision calculation, the model trains for a fixed number of epochs regardless of how well it's actually performing. By computing the precision after each epoch, training can stop as soon as the model is sufficiently trained, saving time and avoiding unnecessary training.

Precision is computed as `1.0 - MAE`, where MAE (*mean absolute error*) is the average absolute error across all training sets. A precision close to `1.0` means the model predicts very close to the reference values.

A note on the name. This is not *precision* in the sense the wider machine learning literature uses it, where precision counts how many of the positive predictions were right and only applies to classification. What's computed here is a fit score for a regression model, and it has two properties worth knowing: it's scale-dependent, since an absolute error of `0.1` means something very different for outputs around `1` than for outputs around `1000`, and it can go negative, since nothing stops the mean absolute error from exceeding `1.0`. Reading `0.999999` as "99.9999% correct" is therefore a convenient shorthand rather than a percentage of anything.

Below is an extension of the regression struct from §2.1 with precision calculation. The private method `computePrecision()` computes the precision, and `train()` stops training early once the precision exceeds a given threshold.

The full implementation is shown below:

```
/**
 * @brief Linear regression with precision calculation.
 */
#include <algorithm>
#include <cmath>
```

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <exception>
#include <vector>

namespace
{
/** One-dimensional matrix. */
using Matrix1d = std::vector<double>;

/** Unsigned 32-bit integer matrix. */
using MatrixU32 = std::vector<std::uint32_t>;

/**
 * @brief Linear regression structure.
 */
struct LinReg
{
    /** Bias value. */
    double bias;

    /** Weight value. */
    double weight;

    /**
     * @brief Predict with the given linear regression model.
     *
     * @param[in] input Input value.
     *
     * @return The predicted value.
     */
    double predict(const double input) const noexcept { return weight * input + bias
    ↪ ; }

    /**
     * @brief Train the given linear regression model with given training sets.
     *
     * @param[in] trainIn    Input values. Size must be greater than 0.
     * @param[in] trainOut   Output values. Size must be greater than 0.
     * @param[in] epochCount Number of epochs to train. Must be greater than 0.
     * @param[in] learningRate Learning rate (default = 1 %). Must be in range (0.0,
    ↪  1.0).
```

```
 * @param[in] precision  Desired precision (default = 99.9999 %). Must be in ↩
↪ range (0.0, 1.0).
 *
 * @return Always true; invalid arguments terminate the program rather than ↩
↪ being
 *         reported, since this model has no way to recover from them.
 */
bool train(const Matrix1d& trainIn, const Matrix1d& trainOut,
           const std::uint32_t epochCount, const double learningRate = 0.01,
           const double precision = 0.999999) noexcept
{
    // Check input parameters and training set count, terminate if invalid.
    const auto setCount = static_cast<std::uint32_t>(std::min(trainIn.size(), ↩
↪ trainOut.size()));
    if (0U == setCount)
    {
        std::fprintf(stderr, "Invalid training data: no training sets provided!\↩
↪ n");
        std::terminate();
    }
    if (0U == epochCount)
    {
        std::fprintf(stderr, "Invalid epoch count: must train for at least one ↩
↪ epoch!\n");
        std::terminate();
    }
    if ((0.0 >= learningRate) || (1.0 <= learningRate))
    {
        std::fprintf(stderr, "Invalid learning rate %g: must be in range (0.0, ↩
↪ 1.0)!\n",
            learningRate);
        std::terminate();
    }
    if ((0.0 >= precision) || (1.0 <= precision))
    {
        std::fprintf(stderr, "Invalid precision %g: must be in range (0.0, 1.0)↩
↪ !\n",
            precision);
        std::terminate();
    }

    // Create train order matrix.
    MatrixU32 trainOrder{trainOrderMatrix(setCount)};
```

```
        // Train the model for the given number of epochs.
        for (std::uint32_t epoch{}; epoch < epochCount; ++epoch)
        {
            // Randomize training order each epoch.
            shuffle(trainOrder);

            // Iterate through all training sets in randomized order.
            for (const auto& i : trainOrder)
            {
                const auto input  = trainIn[i];
                const auto output = trainOut[i];

                // Perform prediction and calculate the error.
                const auto prediction = predict(input);
                const auto error      = output - prediction;

                // Adjust the parameters in accordance with the error.
                bias   += error * learningRate;
                weight += error * learningRate * input;
            }

            // Calculate the precision, return true if exceeded.
            const auto currentPrecision = computePrecision(trainIn, trainOut, ↩
    ↪ setCount);
            if (currentPrecision >= precision)
            {
                const auto percent = currentPrecision * 100.0;
                std::printf("Reached %g %% precision after %u epochs!\n", percent, ↩
    ↪ epoch + 1U);
                return true;
            }
        }
        return true;
    }

private:
    static MatrixU32 trainOrderMatrix(const std::uint32_t setCount) noexcept
    {
        MatrixU32 order(setCount);
        for (std::uint32_t i{}; i < order.size(); ++i) { order[i] = i; }
        return order;
    }

    static void shuffle(MatrixU32& order) noexcept
```

```
    {
        for (std::uint32_t i{}; i < order.size(); ++i)
        {
            // Swap values at index r and i.
            const auto r    = static_cast<std::uint32_t>(std::rand() % order.size())
    ↪ ;
            const auto temp = order[i];
            order[i]        = order[r];
            order[r]        = temp;
        }
    }

    double computePrecision(const Matrix1d& trainIn, const Matrix1d& trainOut,
                            const std::uint32_t setCount) const noexcept
    {
        double totalError{};
        for (std::uint32_t i{}; i < setCount; ++i)
        {
            const auto input      = trainIn[i];
            const auto output     = trainOut[i];
            const auto prediction = predict(input);
            const auto error      = output - prediction;

            totalError += std::abs(error);
        }
        const auto avgError = totalError / setCount;
        return 1.0 - avgError;
    }
};
} // namespace

/**
 * @brief Train and predict with a simple linear regression model.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    constexpr std::uint32_t epochCount{100U};
    constexpr double learningRate{0.1};

    // Initialize the random generator.
    std::srand(std::time(nullptr));
```

```
    // Create linear regression model to predict y = 3x + 1.
    const Matrix1d trainIn{0.0, 1.0, 2.0, 3.0, 4.0};
    const Matrix1d trainOut{1.0, 4.0, 7.0, 10.0, 13.0};
    LinReg linReg{};

    // Train the model, return -1 on failure.
    if (!linReg.train(trainIn, trainOut, epochCount, learningRate))
    {
        std::printf("Training failed!\n");
        return -1;
    }
    // Predict and print result on success.
    for (const auto& input : trainIn)
    {
        const auto prediction = linReg.predict(input);
        std::printf("x = %g, ypred = %g\n", input, prediction);
    }
    return 0;
}
```

The model above builds on the randomized-order version from §2.1 with precision calculation and early stopping. Linear regression is a special case of a single-node neural network; the next step is to see how these same ideas (weights, biases, error-driven adjustment) extend to full neural networks with multiple layers, starting in Chapter 3.

2.3 Review

What you should be able to do now

After this chapter, you should:

- Be able to randomize the order of the training sets before each new epoch.
- Be able to compute the precision of a trained regression model.
- Be able to determine when the model is sufficiently trained based on a given threshold value.
- Be able to adjust the learning rate during training, based on how much the model is improving.

Questions to test yourself

You should be able to explain:

1. Why is it beneficial to randomize the training order every epoch?
2. Why is a static local variable used in `initRandGen()` to ensure the random number generator is only initialized once? Why does the function belong in `ml/utils.cpp` rather than in an anonymous namespace in each model's source file?
3. How is the precision of a linear regression model computed?
4. What does it mean for the precision to exceed a given threshold, and how is this used to stop training early?
5. Why is precision calculation important when training machine learning models?
6. How does `ml::lin_reg::Adaptive` decide whether to raise or lower its learning rate, and why is the rate clamped at both ends?

Looking ahead

Chapter 3 is about neural networks:

- An introduction to neural networks: feedforward, backpropagation, gradient descent, and activation functions.
- Training neural networks by hand.
- The implementation of a dense layer interface and a stub.

2.4 Exercises

How to work these. All four sets extend one program: the linear regression model you wrote in Chapter 1's Set 2. Carry it forward into this lecture's exercises directory, `lectures/L02/exercises` in the companion repository, and extend it set by set, rebuilding with `make` after each exercise.
How to check your work. Set 4 runs the lecture's test suite, already in `lectures/L02/exercises/test`. It is cumulative: it covers everything from Chapter 1 as well as what this chapter adds, so it supersedes the earlier suite entirely. It tests the directory above it by default; `make ML_DIR=<path>` points it at a tree of your own elsewhere.
The solutions are published in the companion repository, in `lectures/L02/exercises`, with a README that walks through them, including what the adaptive learning rate buys on training data its initial rate is too cautious for. Try each exercise yourself before looking at them.

You'll extend the `ml::lin_reg::Fixed` class from Chapter 1, first with a randomized training order, then with precision calculation. Finally you'll add a second model, `ml::lin_reg::Adaptive`, which picks its own learning rate while it trains.

EXERCISE SET 1

Randomizing the Training Order

Exercise 2.1 MatrixU32 alias

CODE

Add the alias `MatrixU32` to the header file `ml/types.hpp` as a substitute for `std::vector<std::uint32_t>`, in the namespace `ml`.

Include `<cstdint>` in `ml/types.hpp`.

Exercise 2.2 Random number generator

CODE

Add a function named `initRandGen()` in an anonymous namespace in `source/ml/lin_reg/fixed.cpp`. The function should initialize the random number generator exactly once when called:

- Use a static local variable named `initialized` (of type `bool`, initialized to `false`) to track whether the generator has already been initialized.
- Initialize the generator with `std::srand(std::time(nullptr))`.
- Set `initialized` to `true` after initialization.
- Include `<cstdlib>` and `<ctime>` in `fixed.cpp`.
- Should be marked `noexcept`.

Exercise 2.3 Training order

CODE

Add a private member variable named `myTrainOrder` to `Fixed`:

- Should hold the indices of the training sets as unsigned integers (`ml::MatrixU32`).
- Mustn't be marked `const`, since its contents are rearranged before every epoch.

The member variable `mySetCount` from Chapter 1 can now be removed: `myTrainOrder` holds one index per training set, so `myTrainOrder.size()` is the number of training sets.

Update the constructor in `source/ml/lin_reg/fixed.cpp`:

- Compute the number of complete training sets as in Chapter 1, i.e. `std::min(trainIn.size(), trainOut.size())`, but keep it in a local variable rather than a member.
- Print an error message and call `std::terminate()` if the set count is 0, as in Chapter 1.
- Call `initRandGen()`, so the random number generator is ready before the first shuffle. Calling it here rather than in `train()` means it's initialized exactly once per model, no matter how often training is started.
- Resize `myTrainOrder` to the set count, then fill it with the indices `0, 1, 2 ... N-1`.

Exercise 2.4 Shuffling CODE

Add a private method named `shuffle()` to `Fixed`:

- Should shuffle the contents of `myTrainOrder` in random order.
- For each index `i`, pick a random index `r` and swap `myTrainOrder[i]` and `myTrainOrder[r]`.
- Should be marked `noexcept`.

Exercise 2.5 Updating the training method CODE

Update the method `train()` in `source/ml/lin_reg/fixed.cpp`:

- Call `shuffle()` at the start of every epoch.
- Iterate through the training sets in the order given by `myTrainOrder` instead of sequentially by index, e.g. with a range-based for loop over `myTrainOrder`. Note that the loop variable is now the index into the training data, not the counter itself.

Exercise 2.6 Compiling and testing CODE

Compile and test-run the program. The training order should now be random every epoch, but the predicted result should still match the training data once training is complete.

EXERCISE SET 2

Precision Calculation

Exercise 2.7 Precision calculation CODE

Add a private method named `precision()` to the `Fixed` class. The method should compute the model's precision given the training data:

- Compute the mean absolute error (MAE) across all training sets:
  - For each training set, compute the absolute error: `abs(output - predict(input))`.
  - Sum all absolute errors and divide by the number of training sets.
- Return `1.0 - MAE` as a floating-point number.
- Should be marked `[[nodiscard]]`, `const`, and `noexcept`.

Include `<cmath>` in `fixed.cpp` for `std::abs`.

Exercise 2.8 Updating the training method CODE

Update the method `train()` in `source/ml/lin_reg/fixed.cpp` so training stops early once the precision exceeds a given threshold:

- Add an argument named `precisionThreshold` (a floating-point number specifying the precision threshold) as the **last** parameter, so the full signature reads `train(epochCount, learningRate = 0.01, precisionThreshold = 0.999999)`. Default value: `0.999999` (99.9999%). Note that the argument shouldn't be named `precision`: inside `train()`, a parameter of that name hides the method `precision()`, so calling it no longer compiles (unless `this->precision()` is used).
- Return `false` if `precisionThreshold` falls outside the range `(0.0, 1.0)`:
  - A threshold of `1.0` or more can never be reached, since the precision is `1.0 - MAE` and the mean absolute error can't be negative. Training would always run the full epoch count.
  - A threshold of `0.0` or less accepts a model whose mean absolute error is `1.0` or worse, which defeats the purpose of the check.
- As in Chapter 1, `train()` reports invalid arguments through its return value. Only the constructor calls `std::terminate()`, because it has no way to return a failure code to the caller.
- Call `precision()` every tenth epoch, starting with the first; computing it scans every training set, so checking less often than every epoch keeps that overhead down over thousands of epochs. Use a named constant for the interval rather than a bare `10`.
- Stop training and return `true` as soon as the precision reaches or exceeds `precisionThreshold`, printing the achieved precision and the number of epochs it took.
- Return `true` once the epochs run out, if the threshold was never reached.

Exercise 2.9 Compiling and testing CODE

Compile and test-run the program. The model should now stop training as soon as the precision reaches the threshold, long before the 1000 epochs `main.cpp` asks for:

```
Target precision 1.00 reached after 11 epochs!
Input: 0, prediction: 2
Input: 1, prediction: 5
Input: 2, prediction: 8
Input: 3, prediction: 11
Input: 4, prediction: 14
```

The training order is random, so the epoch count can vary between runs. For this training data it settles at 11: the precision is only checked every tenth epoch, so the check that first

passes is the one after the eleventh epoch has run. Report the number of epochs completed, not the loop index, so the first epoch reads as 1 rather than 0.

EXERCISE SET 3

An Adaptive Learning Rate

The learning rate is the one training parameter with no sensible default. Too low and the model crawls, too high and it never settles, and where the line between the two falls depends on the training data rather than on the algorithm. Instead of asking the caller to guess, you'll now write a second model that starts at a cautious rate and revises it while training, using the precision it already computes every tenth epoch.

The new class is called `Adaptive`, and it's the one the `Fixed` name has been pointing at since Exercise 1.7.

Exercise 2.10 Shared utilities

CODE

`initRandGen()` currently sits in an anonymous namespace in `fixed.cpp`, which makes it private to that file. The new model needs the same one-time seeding, and copying the function across would give each file its own `initialized` flag: the second copy would call `std::srand()` again when the second model is constructed, and a re-seed within the same second restarts the very sequence that was already running.

Move the function into a pair of shared files instead:

- Create `include/ml/utils.hpp`. Declare `initRandGen()` there, in the namespace `ml`, taking no arguments, returning nothing, and marked `noexcept`.
- Create `source/ml/utils.cpp`. Move the definition there unchanged, along with the `<cstdlib>` and `<ctime>` includes it needs.
- Remove the function and the `<ctime>` include from `fixed.cpp`, and include `"ml/utils.hpp"` instead. Keep `<cstdlib>`: `shuffle()` still calls `std::rand()`. The call in the constructor stays exactly as it was: `Fixed` lives in `ml::lin_reg`, so the unqualified name still finds `ml::initRandGen()` in the enclosing namespace.
- Add `source/ml/utils.cpp` to `SRC_FILES` in the makefile.

Rebuild and re-run before going further. Nothing about the program's behaviour should have changed; this step only moves code.

Exercise 2.11 The Adaptive class

CODE

Add the header `include/ml/lin_reg/adaptive.hpp` and the source file `source/ml/lin_reg/adaptive.cpp`, and add the source file to `SRC_FILES` in the makefile.

In the namespace `ml::lin_reg`, implement a class named `Adaptive` that inherits `Interface` via public inheritance and is marked `final`. Everything except `train()` is the same as in `Fixed`:

- The constructor takes `trainIn` and `trainOut` and is marked `explicit` and `noexcept`. It checks the set count, terminates when it's 0, fills `myTrainOrder` with `0, 1, 2 ... N-1`, and calls `initRandGen()`.
- `~Adaptive()` is `default`, `noexcept`, and `override`.
- `predict()` returns $y = kx + m$, and is marked `[[nodiscard]]`, `const`, `noexcept`, and `override`.
- The private methods `optimize()`, `shuffle()`, and `precision()` are the same as in `Fixed`.
- The private member variables are the same: `myTrainOrder`, `myTrainIn`, `myTrainOut`, `myBias`, and `myWeight`.
- The default constructor, copy and move constructors, and the corresponding operators are deleted.

`train()` is where the two models part ways:

```
bool train(std::size_t epochCount, double precisionThreshold = 0.999999) noexcept;
```

There's no `learningRate` argument, because the model no longer takes one. Copying the shared parts across rather than sharing them with `Fixed` is deliberate here: two small independent models make the one difference between them easy to see.

Exercise 2.12 The learning rate rule

CODE

Add a function named `updateLearningRate()` in an anonymous namespace in `adaptive.cpp`:

```
void updateLearningRate(double& learningRate, double& prevPrecision,
                        double currentPrecision) noexcept;
```

It takes the current learning rate and the previous precision by reference, since it updates both, and the precision just measured by value. Give each of the four values it needs a named constant: a maximum learning rate of `0.25`, a minimum of `1e-6`, a smallest expected improvement of `0.1`, and a step of `0.05`.

The rule itself works on the difference between the two precisions, i.e. how much the model improved since the last evaluation:

- **Improved by at least the expected minimum:** leave the learning rate alone. It's working.
- **Improved, but by less than that:** raise the learning rate by the step. Progress has stalled, so take bigger steps.

- **Didn't improve at all:** lower the learning rate by the step. The model is overshooting, so take smaller steps.

Neither branch should enforce the limits itself. Once the rate has been adjusted, keep it inside them with a single call to `std::clamp()` (from `<algorithm>`, already included for `std::min()`): `learningRate = std::clamp(learningRate, minLearningRate, maxLearningRate)`. It returns the nearer bound whenever the value falls outside the range, so one call covers both ends and each branch is left saying only what it's about.

Store the precision just measured in `prevPrecision` before returning, so the next call has something to compare against.

That one clamp is what keeps the rule safe. Its maximum stays below the rate at which this kind of training data starts to oscillate instead of converging, and its minimum keeps the rate positive, so a model that keeps missing can still creep towards the line rather than stopping dead.

Exercise 2.13 Training

CODE

Implement `train()` in `adaptive.cpp`:

- Give the evaluation interval (`10`) and the initial learning rate (`0.1`) named constants.
- Declare the learning rate and the previous precision as local variables, initialized to the initial learning rate and `0.0`. Being local means every call starts from the initial rate again, rather than inheriting whatever the previous call ended on.
- Return `false` if `epochCount` is 0, or if `precisionThreshold` falls outside the range `(0.0, 1.0)`, exactly as `Fixed` does. There's no learning rate left to validate.
- For each epoch: call `shuffle()`, then run `optimize()` over all training sets in the order given by `myTrainOrder`, passing the current learning rate.
- Every tenth epoch, except the first, compute the precision once and keep it in a local variable:
  - Stop training and return `true` if it reaches or exceeds `precisionThreshold`, printing the achieved precision and the number of epochs it took, as in Exercise 2.8.
  - Otherwise, pass it to `updateLearningRate()`.
- Return `true` once the epochs run out.

Note that the precision is computed once per evaluation and then used twice. Calling `precision()` a second time inside `updateLearningRate()` would scan every training set again for a value that's already in hand.

Exercise 2.14 Compiling and running CODE

Update `main.cpp` to train an `ml::lin_reg::Adaptive` model rather than a `Fixed` one, and drop the learning rate from the `train()` call. An epoch budget of 100 is plenty now, since the model stops as soon as it reaches the default precision threshold.

Compile and run. The predictions should match the training data, without a learning rate having been chosen anywhere in the program:

```
Input: 0, prediction: 2
Input: 1, prediction: 5
Input: 2, prediction: 8
Input: 3, prediction: 11
Input: 4, prediction: 14
```

Above them sits the early-stop line your own `train()` prints. The epoch count in it varies with the shuffled order.

EXERCISE SET 4

Running the Tests

Exercise 2.15 Running the tests CODE

An updated test suite is available in `lectures/L02/exercises/test`. It's cumulative: it covers everything from Chapter 1 as well as the randomized training order, the precision threshold, and the adaptive learning rate added here, so it supersedes the earlier suite entirely.

Once you've carried your code forward into this lecture's exercises directory, build and run it as before:

```
make -C test
```

All 38 test cases should pass: 20 for `Fixed` and 18 for `Adaptive`. Note that `train()` prints its progress, so the output is interleaved with early-stop reports. Look for the summary on the last line.

The test suite's README, `lectures/L02/exercises/test/README.md`, has more information, including why the tests that check exact values train on a single training set now that the order is randomized, and how a learning rate the model keeps to itself can be tested at all.

CHAPTER 3

Neural Networks

Feedforward, backpropagation and optimization, worked through by hand before a line of the network is written.

IN THIS CHAPTER

- An introduction to neural networks: feedforward, backpropagation, gradient descent, and activation functions.
- Training neural networks by hand.
- A dense layer interface, and a stub implementation of it.

This chapter moves from a single linear model to neural networks: layers of nodes, each holding a bias and a weight per input, whose outputs are passed through an activation function and on to the next layer. It explains what happens during feedforward and backpropagation, how the parameters are optimized from the errors backpropagation computes, and what the activation functions are for. You should come out of it able to explain concepts such as layers, nodes, and weights, and able to train a small neural network by hand.

Read §3.1 first; the three hand-training exercises in §3.4 then put its equations to work on two small networks, and are the best way to find out whether the theory has stuck. §3.2 introduces the dense layer interface you start building in the second half of the exercises, together with a stub implementation of it. That pair is the foundation the network in Chapter 4 is built on, and the real layer in Chapter 5 replaces the stub without anything else having to change.

3.1 Neural Networks

3.1.1 Introduction

- Through machine learning, a given machine can become capable of learning rules without having been explicitly programmed to do so.
- **Deep learning** is a subfield of machine learning and involves techniques that vaguely resemble how our brains work. The name stems from the fact that the technique was

originally inspired by how our brains function. In practice, though, brains and neural networks work very differently.

- Deep learning uses so-called **neural networks** containing several steps, or layers, between input and output, often hundreds, which allows relevant information to be extracted from the input while irrelevant information is discarded. This makes deep learning well suited to images and similar data containing a large amount of information, most of which is usually insignificant.
- The more layers used in a neural network, the deeper the model. Information can be thought of as passing through and being processed by different filters that distill the information. In each layer, a certain type of irrelevant data is filtered out while relevant data is retained. Step by step, the input changes from its original form and increasingly contains information about the output.

3.1.2 Theory Behind Training a Neural Network

- For a neural network to predict with low deviation, the network must be trained so that the parameters (weights and biases) of its nodes are adjusted to suitable values.
- Existing training sets are therefore used to train the network for a certain number of epochs. At each epoch, the order of the training sets is randomized so the network doesn't get used to any patterns arising from the particular order the training sets happen to be stored in.

The following steps are performed for each training set, for a simple neural network consisting of an input layer, a hidden layer, and an output layer.

1. Feedforward

- The nodes in the input layer form the inputs x to every node in the hidden layer.
- Since these inputs x carry different degrees of importance, also called weight, each is multiplied by its own weight w in the hidden layer. These weights should be randomly chosen at the start, and then adjusted during training.

For each node in the hidden layer, the contribution $w \cdot x$ from each input x from the input layer is summed together with the node's resting point, or bias, b into a sum s:

$$s = b + \sum_{i=0}^{n-1} w_i \cdot x_i = b + w_0 \cdot x_0 + w_1 \cdot x_1 + \cdots + w_{n-1} \cdot x_{n-1},$$

where:

- b is the node's bias.
- n is the number of inputs.
- $w_i \cdot x_i$ is the product of each weight w and its connected input x.

The node's predicted output y_p is obtained via an activation function σ:

$$y_p = \sigma(s) = \sigma\left(b + \sum_{i=0}^{n-1} w_i \cdot x_i\right)$$

The outputs from the hidden layer then form the inputs to the output layer, where the same principle applies.

2. Backpropagation

The deviation for each node's output is computed starting from the output layer and moving backward to the hidden layer. For a given node n in the output layer:

$$\delta = y_{\text{ref}} - y_p$$

The computed error Δe for a given node n in the output layer:

$$\Delta e = \delta \cdot y'_p,$$

where y'_p is the derivative of the node's predicted output.

For a given node n in a given hidden layer, the deviation δ is computed as

$$\delta = \sum_{i=0}^{n-1} [\Delta e_i \cdot w_i]$$

and the computed error Δe for the node:

$$\Delta e = \delta \cdot y'_p$$

3. Optimization

Once the error for the nodes' parameters has been computed, optimization is performed via a gradient algorithm. The rate of change Δc_n for a given node n:

$$\Delta c_n = \Delta e_n \cdot L,$$

where:

- Δe_n is the computed error for the current node.
- L is the learning rate.

The node's bias b_n is updated:

$$b_n = b_n + \Delta c_n$$

Each weight w_j connected to node n is updated:

$$w_j = w_j + \Delta c_n \cdot y_j,$$

where y_j is the output from the connected node j in the previous layer (for the input layer, $y_j = x_j$).

3.1.3 Activation Functions

- Some type of activation function is typically used for each node in the hidden and output layers. This introduces non-linearity into the network, which is required to model anything beyond simple linear relationships.
- Some activation functions (e.g. sigmoid, tanh, and softmax) also keep the output within a certain range, while others (e.g. ReLU) are unbounded above.

Activation function	**Output range**	**Typical use**	**Main drawback**
ReLU	$[0, \infty)$	Hidden layers (default choice)	Can "die" on negative inputs
Leaky ReLU	$(-\infty, \infty)$	Hidden layers, alternative to ReLU	Extra hyperparameter (α) to choose
Sigmoid	$(0, 1)$	Output layer for binary classification	Saturates at large $\|s\|$, gives small gradients
Tanh	$(-1, 1)$	Hidden layers, zero-centered output	Saturates at large $\|s\|$, gives small gradients
Softmax	$(0, 1)$, sums to 1	Output layer for multi-class classification	Requires one output node per class

The node's output y is obtained as:

$$y = \sigma(s) = \sigma\left(b + \sum_{i=0}^{n-1} w_i \cdot x_i\right)$$

1. ReLU

The most common activation function in deep learning is **ReLU** (*Rectified Linear Unit*):

$$\begin{cases} s > 0 \Rightarrow y = s \\ s \leq 0 \Rightarrow y = 0 \end{cases}$$

As a rule of thumb, ReLU tends to be the most suitable activation function to implement when there's no clear-cut answer.

Leaky ReLU is an improved variant where signals below zero are retained as a weak signal:

$$\begin{cases} s > 0 \Rightarrow y = s \\ s \leq 0 \Rightarrow y = s \cdot \alpha \end{cases}$$

- As a rule of thumb, $\alpha = 0.01$ can be used.
- Leaky ReLU counteracts nodes "dying" (getting stuck at zero), which can otherwise cause the model to train poorly.

The derivative of ReLU:

$$\begin{cases} s > 0 \Rightarrow dy = 1 \\ s \leq 0 \Rightarrow dy = 0 \end{cases}$$

Every derivative below is a function of the weighted sum s, which is what the implementation passes in. Sigmoid and tanh simply happen to have compact forms that reuse the output y; that's the same number, not a different argument. See §5.2 for why the layer keeps s around alongside y.

2. Sigmoid

Sigmoid is a common activation function when only two classes are used, where the output is set to a number between 0 and 1:

$$y = \frac{1}{1 + e^{-s}}$$

The derivative:

$$\sigma'(s) = y \cdot (1 - y), \quad \text{where } y = \sigma(s)$$

3. Tanh

Tanh (*hyperbolic tangent*) resembles sigmoid but produces an output between -1 and 1, which often leads to faster learning since the outputs become zero-centered:

$$y = \tanh(s) = \frac{e^{s} - e^{-s}}{e^{s} + e^{-s}}$$

The derivative:

$$\sigma'(s) = 1 - \tanh^2(s) = 1 - y^2$$

4. Softmax

Softmax is a common activation function in the output layer, where the sum of all output probabilities equals one. The output y for a node in a layer of n nodes:

$$y = \frac{e^{s}}{\sum_{i=0}^{n-1} e^{s_i}}$$

Softmax is particularly useful for multi-class classification, e.g. image recognition.

3.2 Dense Layer Architecture

3.2.1 Overview

Starting in this chapter, you'll build up a small neural network in C++ step by step, over this chapter and the next. The first piece is a `dense_layer::Interface` and a placeholder implementation of it, `dense_layer::Stub`:

```
ml::dense_layer::Interface   (Interface for a dense layer)
└── ml::dense_layer::Stub    (Dense layer stub)
```

In Chapter 4, this interface will be used by a `neural_network::Shallow` class to build a small working (if not yet properly trained) network. Chapter 5 then replaces `Stub` with a real, trainable implementation; the interface stays the same throughout, so nothing that depends on it needs to change when that happens.

3.2.2 The Dense Layer's Interface

`ml::dense_layer::Interface` defines the contract every dense layer must fulfill:

Method	Description
`nodeCount()`	Number of nodes in the layer
`weightCount()`	Number of weights per node
`output()`	The layer's outputs after feedforward
`error()`	The layer's computed error after backpropagation
`weights()`	The layer's weights (2D: node × weight)
`feedforward(input)`	Computes outputs from the input
`backpropagate(reference)`	Computes error from reference values (output layer)
`backpropagate(nextLayer)`	Computes error from the next layer (hidden layer)
`optimize(input, learningRate)`	Updates bias and weights
`initParams()`	Resets the layer's trainable parameters, i.e. bias and weights

Every method in the table is pure virtual: the interface states what a dense layer must be able to do and nothing about how. `initParams()` is what lets the network in Chapter 4 start each training run from freshly drawn parameters without knowing which kind of layer it holds; a layer with nothing to reset, like the stub, implements it as an empty method. See §3.4.

3.2.3 The Stub Class

`ml::dense_layer::Stub` implements the interface but doesn't do anything meaningful:

- Feedforward always sets the output to a fixed value (e.g. `0.5`).
- Backpropagation and optimization do nothing.
- The parameter reset is implemented as an empty method: there's no bias, and the weights stay at zero.

The stub exists so that other code, namely the `Shallow` network class you'll build in Chapter 4, can be written, compiled, and test-run against a real `dense_layer::Interface` before a correct implementation exists. A correct implementation follows in Chapter 5.

3.3 Review

What you should be able to do now

After this chapter, you should:

- Know what a neural network is and be able to explain concepts such as layers, nodes, and weights.
- Be able to train small neural networks by hand.
- Have created an interface for a dense layer and a stub implementation of it.

Questions to test yourself

You should be able to explain:

1. Can you, in your own words, explain what happens in a neural network during feedforward and backpropagation?
2. What's the purpose of an activation function, and what happens if you don't use one?
3. What happens to training if the learning rate is set too high or too low?
4. Why is it beneficial to define an interface for the dense layer rather than implementing a class directly?
5. Your stub implements `initParams()` as an empty method. Why does the interface declare a method the stub has nothing to do for, and what would the network in Chapter 4 lose without it?

Looking ahead

Chapter 4 continues the exercise:

- A neural network interface, and a concrete `Shallow` class that uses the stub you just built.

3.4 Exercises

How to work these. Set 1 is three calculations: work them on paper with the equations from §3.1 and a calculator, one training set at a time. Set 2 starts a new `ml` codebase in this lecture's exercises directory, `lectures/L03/exercises` in the companion repository, and is built with a Makefile you write yourself.
How to check your work. The worked solutions to Set 1 are in Appendix A. Set 2 comes with a test suite, already in `lectures/L03/exercises/test`, which Exercise 3.7 runs. It tests the directory above it by default; `make ML_DIR=<path>` points it at a tree of your own elsewhere. The solution to Set 2 is published in the companion repository after the lecture; `lectures/L03/README.md` links to it. Try each exercise yourself before looking at it.

EXERCISE SET 1

Hand-Training a Neural Network

Exercise 3.1 Odd parity, first training set

CALCULATION

The neural network in Figure 3.1 should be trained to predict a high output Y when an odd number of the inputs X_1 and X_2 are high, and output $Y = 0$ otherwise:

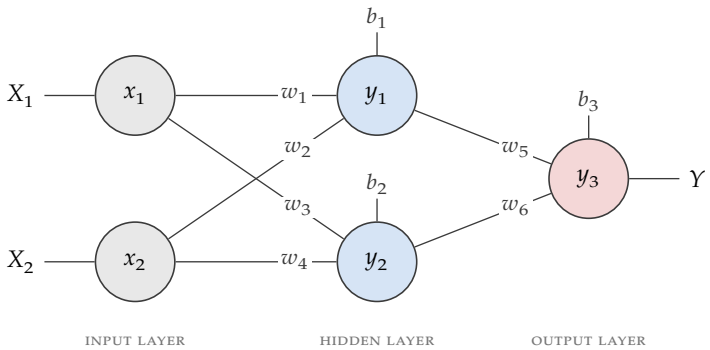


Figure 3.1 The network in Exercises 3.1 and 3.3: two inputs, two hidden nodes and one output node.

The first training set: inputs $X_1X_2 = 01$, $Y = 1$.

Starting parameters

$$
\begin{aligned}
b_1 &= 0.2, & b_2 &= 0.4, & b_3 &= 0.6 \\
w_1 &= 0.5, & w_2 &= 0.6, & w_3 &= 0.3 \\
w_4 &= 0.8, & w_5 &= 0.1, & w_6 &= 0.9
\end{aligned}
$$

Learning rate: $LR = 0.1$. Activation function: ReLU for all nodes.

Train the network on the first training set: perform feedforward, backpropagation, and optimization. Then feed the same input forward again with the new parameters, and compare the deviation against the one you started with.

The worked solution is on page 159.

Exercise 3.2 Inverting two inputs

CALCULATION

Consider the neural network in Figure 3.2:

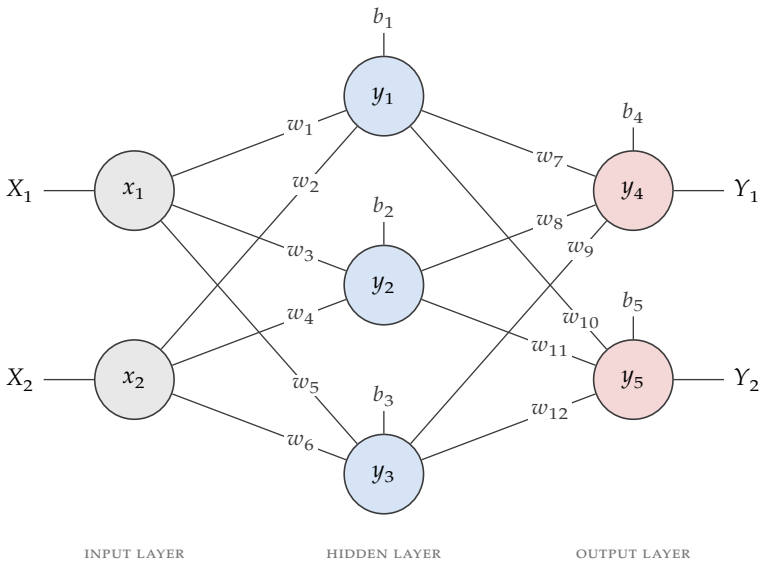


Figure 3.2 The network in Exercise 3.2: two inputs, three hidden nodes and two output nodes.

The network should be trained to predict outputs Y_1Y_2 that form the inverse of inputs X_1X_2:

X_1X_2	Y_1Y_2
00	11
01	10
10	01
11	00

First training set: $X_1X_2 = 10$, $Y_1Y_2 = 01$.

Starting parameters: hidden layer

$$b_1 = 0.1, \quad b_2 = 0.1, \quad b_3 = 0.7$$
$$w_1 = 0.2, \quad w_2 = 0.9, \quad w_3 = 0.3$$
$$w_4 = 0.8, \quad w_5 = 0.5, \quad w_6 = 0.4$$

Starting parameters: output layer

$$\begin{aligned} b_4 &= 0.1, & b_5 &= 0.6 & \\ w_7 &= 0.1, & w_8 &= 0.1, & w_9 &= 1.0 \\ w_{10} &= 0.3, & w_{11} &= 0.0, & w_{12} &= 0.6 \end{aligned}$$

Learning rate: $LR = 0.1$. Activation function: ReLU for all nodes.

Train the network on the first training set: perform feedforward, backpropagation, and optimization. Then feed the same input forward again with the new parameters, and compare the deviations against the ones you started with.

The worked solution is on page 160.

Exercise 3.3 Odd parity, after one epoch

CALCULATION

Consider the neural network in Figure 3.1 again. The network should be trained to predict a high output Y when an odd number of the inputs X_1 and X_2 are high, and output $Y = 0$ otherwise.

Training has already been carried out for one epoch, and the parameters have the following starting values:

Starting parameters

$$\begin{aligned} b_1 &= 0.1, & b_2 &= -0.4, & b_3 &= -0.2 \\ w_1 &= 0.5, & w_2 &= 0.4, & w_3 &= -0.2 \\ w_4 &= 0.1, & w_5 &= 0.7, & w_6 &= 0.8 \end{aligned}$$

Training is now to be carried out with the training set $X_1X_2 = 11$, $Y = 0$.

Learning rate: $LR = 0.1$. Activation function: ReLU for all nodes.

Perform feedforward, backpropagation, and optimization. Then compute the output again. Did the error decrease? If not, what could you have changed to obtain a smaller error?

The worked solution is on page 162.

EXERCISE SET 2

Dense Layer Interface and Stub

You'll start a new `ml` codebase (or extend your existing one, if you'd like to keep the linear regression code alongside it) with an interface for a dense layer and a placeholder implementation of it.

Exercise 3.4 Directory structure

CODE

Set up the following structure in this lecture's exercises directory, `lectures/L03/exercises`:

```
exercises/
├── include/
│   └── ml/
│       ├── dense_layer/
│       │   ├── interface.hpp
│       │   └── stub.hpp
│       └── types.hpp
├── source/
│   └── main.cpp
├── test/                <- already here, see Exercise 3.7
└── Makefile
```

You'll extend this structure further in Chapter 4.

Exercise 3.5 Dense layer interface and stub class

CODE

The hidden layer and output layer of a future neural network will be represented by the interface `ml::dense_layer::Interface`. A concrete implementation isn't created until Chapter 5. Until then, you'll implement a simple stub class `ml::dense_layer::Stub` as a placeholder.

The interface (`ml/dense_layer/interface.hpp`)

In the namespace `ml::dense_layer`, implement an interface named `Interface`. All methods (except the destructor) should be declared pure virtual (`= 0`).

- **`~Interface()`:** should be set to `default` and marked `virtual` and `noexcept`.

Getters, all `const`, `noexcept`, and `[[nodiscard]]`:

Method	Returns
`nodeCount()`	Number of nodes in the layer (`std::size_t`).
`weightCount()`	Number of weights per node (`std::size_t`).
`output()`	Reference to the layer's output (read-only floating-point vector).
`error()`	Reference to the layer's error (read-only floating-point vector).
`weights()`	Reference to the layer's weights (read-only, two-dimensional floating-point vector).

Computation methods, all `noexcept` and returning `bool`. Each returns `false` on invalid input (wrong dimensions or an invalid learning rate) and `true` otherwise, so the caller can decide what to do about the failure. As in Chapters 1 and 2, `std::terminate()` is reserved for the constructor, which has no way to return a failure code to the caller:

- **`feedforward(input)`:** performs feedforward.
  - `input`: read-only floating-point vector of input data.
- **`backpropagate(reference)`** (output layer): computes error from reference values.
  - `reference`: read-only floating-point vector of reference values.
- **`backpropagate(nextLayer)`** (hidden layer): computes error from the next layer.
  - `nextLayer`: reference to the next layer (`const Interface&`).
- **`optimize(input, learningRate)`:** updates bias and weights.
  - `input`: read-only floating-point vector.
  - `learningRate`: floating-point number.

One method more, `noexcept` and returning nothing:

- **`initParams()`:** resets the layer's trainable parameters, i.e. its bias values and weights, to their initial values.
  - Takes no arguments and returns nothing.
  - Pure virtual like the rest, so every layer states what its own reset means, even when the answer is "nothing". `Stub` has no trainable parameters to draw again, so its override is empty; `Dense` overrides it in Chapter 5 by drawing a new random bias and weight for every node, and its constructor calls it too, so the randomization is written once and used from both places.

Who calls it. The network you write in Chapter 4 does, on both of its layers, before its first training epoch. That's what makes a second call to `train()` train a new network rather than continue the one the previous call left behind, which matters when a run ends up badly trained: a network that has settled into a bad set of parameters mostly stays there, while a fresh start draws new ones and can converge on the next attempt. The bias and weights are private to the layer, so without a method on the interface the network couldn't reset them at all.

The stub class (`ml/dense_layer/stub.hpp`)

In the namespace `ml::dense_layer`, implement a subclass named `Stub` that inherits `Interface` via public inheritance. The class should be marked `final`. The stub doesn't perform any real computation; it exists solely so other code can be compiled, test-run, and unit tested against a real `dense_layer::Interface` before a concrete `Dense` implementation exists (see Chapter 5). The network you build in Chapter 4 is tested entirely against this stub, so it's worth getting right.

- **`Stub()`:**
  - The class's only implemented constructor.
  - Should take the following arguments:
    - `nodeCount`: number of nodes in the layer (unsigned integer).
    - `weightCount`: number of weights per node (unsigned integer).
    - `outputValue`: value every output element is set to (floating-point number). Default value: `0.5`.
  - Initializes every element of the output vector to `outputValue`, and the error and weight vectors to zeros.
  - Should print an error message and call `std::terminate()` if either `nodeCount` or `weightCount` is zero:
    - As in Chapters 1 and 2, the constructor is the one place that terminates, since it has no way to return a failure code.
    - It also guarantees at least one node and one weight, which is what makes it safe for `weightCount()` to read the width of the first row of the weight matrix.
  - `outputValue` is an argument rather than a hard-coded constant so that two stubs can be told apart. A network whose layers all report the same output can't show whether a prediction came from the output layer or from the hidden one; giving each layer its own value makes that visible.
  - Should be marked `explicit` and `noexcept`.
- **`~Stub()`:**
  - Destructor overriding the interface's destructor.
  - Should be marked `default`, `noexcept`, and `override`.
- **Getters** (`nodeCount()`, `weightCount()`, `output()`, `error()`, `weights()`):
  - Override the corresponding methods in the interface.
  - Should be marked `override`, retaining the interface's `[[nodiscard]]`, `const`, and `noexcept`.
  - Attributes aren't inherited, so the override needs its own `[[nodiscard]]`. Without it, a discarded call such as `stub.nodeCount();` compiles silently whenever it's made on a `Stub` rather than through an `Interface&`.

- **`feedforward()`**, both overloads of **`backpropagate()`**, and **`optimize()`**:
  - Perform range checks only:
    - Return `false` when the dimensions don't match, or, for `optimize()`, when the learning rate lies outside `(0.0, 1.0)`.
    - Return `true` otherwise.
  - Deliberately compute nothing:
    - The output stays at `outputValue` regardless of what's fed in.
    - The error stays at zero.
  - Should be marked `override` and `noexcept`.
- **`setOutput()`:**
  - Sets every element of the output vector to the given value.
  - Should take a single argument:
    - `outputValue`: value to set every output element to (floating-point number).
  - Returns nothing, and should be marked `noexcept`.
  - **Not** part of `Interface`. It exists on the stub alone.
  - It's what lets a test drive the output of a whole network in Chapter 4:
    - The network stores its layers by reference, so calling `setOutput()` on the layer a network was built with changes what that network predicts.
    - That reveals whether the network reads its output layer live or kept a stale copy of it. Keeping a stale copy is exactly what the note in Chapter 4 about not needing a separate storage variable is there to prevent.
- **`feedforwardCount()`** and **`clearFeedforwardCount()`**:
  - `feedforwardCount()` returns the number of times `feedforward()` has been called on this layer as a `std::size_t`, counting **every** call, not just the ones that passed the range check. Return `std::size_t` rather than `int`: the tests compare the tally against `std::size_t` values, and a signed return trips `-Wsign-compare`, which the suite builds with `-Werror`.
    - Increment the counter (`myFeedforwardCount`) at the very top of `feedforward()`, before the input size is checked, so a rejected call raises it just as an accepted one does.
    - The tally records how often the layer was *asked* to feed forward, not how often it agreed to.
    - Should be marked `[[nodiscard]]`, `const`, and `noexcept`.
  - `clearFeedforwardCount()` resets that tally to zero.
    - Returns nothing, and should be marked `noexcept`.
  - Neither is part of `Interface`.

  - `train()` in Chapter 4 performs one feedforward per training set per epoch, so this is what lets a test pin down the training loop:
    - Nothing else can: a loop that runs a single pass instead of every epoch still lines up dimensionally and still returns `true`, so without a count it's indistinguishable from a correct one.

- **`initParams()`:**
  - Overrides the corresponding method in the interface, and should be marked `override` and `noexcept`.
  - Its body is empty: the stub has no bias, and its weights never move from the zeros it was constructed with, so there's nothing to draw again.
  - An empty body is the implementation here, not a placeholder. It's also the only one the network in Chapter 4 needs: that network calls `initParams()` on both of its layers before every training run, and against a stub the call has to compile, run, and change nothing.

For this class, the default constructor and the copy and move constructors (and corresponding operators) should be deleted.

Add private member variables for the output, the error, the weights, and the feedforward count.

The number of nodes and the number of weights per node don't need member variables of their own. Both are readable from the vectors you already have: `nodeCount()` is the size of the output vector, and `weightCount()` is the width of the weight matrix.

The bias doesn't need one at all. It isn't part of `Interface`, and this stub never optimizes anything, so there'd be nothing to put in it and no way to read it back out.

Note! Leaving it out is a stub-only simplification. The real `Dense` layer you write in Chapter 5 does need a bias vector: it's a trainable parameter, adjusted alongside the weights on every call to `optimize()`.

Exercise 3.6 A quick compile check CODE

There's no network to run yet; that's Chapter 4, once `neural_network::Shallow` exists to make use of this interface. For now, just confirm your `Interface`/`Stub` pair compiles: in `main.cpp`, create a `ml::dense_layer::Stub` with a few nodes and weights, and print its `nodeCount()`, `weightCount()`, and `output()` to the terminal. You should see `nodeCount()`/`weightCount()` match what you constructed it with, and every value in `output()` equal to the default `outputValue` of `0.5`.

Create a second stub with an explicit `outputValue`, and check that its `output()` reports that value instead. Also feed both stubs a vector of the wrong size and check that `feedforward()` returns `false` rather than terminating.

Exercise 3.7 Running the tests CODE

A test suite for the stub is available in `lectures/L03/exercises/test`. It's already in place next to the code you just wrote. Build and run it from the exercises directory:

```
make -C test
```

The test framework is checked out as a submodule at the root of the repository. If `libs/test` is empty, fetch it once with `git submodule update --init`.

All 14 test cases should pass. **This chapter starts a new `ml` codebase**, so this suite replaces nothing and stands on its own.

Testing a class that computes nothing may look pointless, but the range checks are the only real logic the stub has, and the network you write in Chapter 4 is tested entirely against them. A stub whose `feedforward()` accepted an input of any size would let every test in Chapter 4 pass without proving anything.

The test suite's README, `lectures/L03/exercises/test/README.md`, has more information, including which of these tests carry over unchanged to your real `Dense` layer in Chapter 5.

CHAPTER 4

A Network Built on Interfaces

Two layers, one training loop, and nothing that knows what the layers really are.

In this chapter

- A neural network interface, and a concrete `Shallow` class that uses the `dense_layer::Interface`/`Stub` pair from Chapter 3.
- Connecting components via interfaces.
- The network's training method, including the precision it measures itself by and the adaptive learning rate it derives from that measurement.

This chapter builds the piece that puts the dense layer interface to use: a small neural network with one hidden layer and one output layer, written against `dense_layer::Interface` so that it never needs to know which kind of layer it holds. After it, you should have an interface for a neural network and a concrete class implementing it, with a fully working training method, and be able to explain how a model can set its own learning rate, and why a training method that resets its parameters first is worth having. The layers are still the stub from Chapter 3, so the network runs, trains and predicts, but learns nothing until Chapter 5 gives it a real layer.

§4.1 shows how the network is put together, and §4.2 how its training loop maps onto the theory from §3.1. The exercises in §4.4 then declare the network class and implement its training method, with the adaptive learning rate from Chapter 2 carried over to it.

4.1 Neural Network Architecture

4.1.1 Overview

Chapter 3 built a `dense_layer::Interface` and a placeholder implementation of it, `dense_layer::Stub`. This chapter adds the piece that actually uses them: a small neural network class, `neural_network::Shallow`, completing the structure:

```
ml::dense_layer::Interface   (built in Chapter 3)
└── ml::dense_layer::Stub    (built in Chapter 3)

ml::neural_network::Interface   (Interface for neural networks)
└── ml::neural_network::Shallow (Neural network with a single hidden layer)
        ├── ml::dense_layer::Interface& myHiddenLayer
        └── ml::dense_layer::Interface& myOutputLayer
```

`ml::dense_layer::Stub` continues to serve as a placeholder throughout this chapter. Once a concrete implementation is created in Chapter 5, the stub is replaced without needing to change any of the code you write in this one.

4.1.2 The network's structure

`Shallow` holds references to two dense layers and connects them during prediction:

```
input → myHiddenLayer.feedforward(input)
                ↓
        myHiddenLayer.output()
                ↓
        myOutputLayer.feedforward(...)
                ↓
        myOutputLayer.output()  →  prediction
```

`Shallow` doesn't own the layers; it receives them as references via the constructor. This makes it easy to swap out layers without changing the network class, which is exactly what happens in Chapter 5 when the stub is replaced with a real implementation.

4.2 Training Loop

4.2.1 Overview

You'll implement the method `train()` in `ml::neural_network::Shallow`. The method carries out the complete training process, repeated for every training set and epoch:

- Feedforward:
  - Computes outputs for every node in the network.
  - The output from the last layer forms the network's prediction.
  - Must be computed forward (from input to output).
- Backpropagate:
  - Computes gradients (simplified error) for every node in the network.
  - Must be computed backward (from output to input).

- Optimization:
  - Adjusts the trainable parameters (weights and biases) based on the computed error, together with the learning rate the network currently holds.
  - Computation can happen in either direction.

4.2.2 Structure of the training loop

The training loop can be summarized as follows:

```
Reset both layers' trainable parameters

For each epoch:
    For each training set x:
        1. Feedforward: compute predictions
        2. Backpropagation: compute error
        3. Optimization: update weights and biases

    Every hundredth epoch:
        4. Measure the precision, and stop early or set a new learning
           rate from it
```

The order matters:

- Backpropagation depends on the result of feedforward.
- Optimization depends on the result of backpropagation.

The reset at the top is what makes a second call to `train()` train a new network rather than continue the old one, and step 4 is what frees the caller from having to pick a learning rate. Both are covered in the exercises, in §4.4.

4.2.3 Step 1: Feedforward

Call the network's private `feedforward(myTrainInput[x])`. This performs feedforward through both layers and returns `false` if either layer rejected its input. `predict()` performs the same pass and then returns the output layer's output, but training has no use for that output.

```
feedforward(input)
  → myHiddenLayer.feedforward(input)
  → myOutputLayer.feedforward(myHiddenLayer.output())
```

4.2.4 Step 2: Backpropagation

Compute the error backward: always start with the output layer.

The output layer compares its output against the reference value:

```
myOutputLayer.backpropagate(myTrainOutput[x])
```

The hidden layer computes its error from the output layer's error and weights:

```
myHiddenLayer.backpropagate(myOutputLayer)
```

The order is crucial: the hidden layer's calculation depends on the output layer having already computed its error.

4.2.5 Step 3: Optimization

Update the weights and biases in both layers using the errors computed in step 2.

```
myHiddenLayer.optimize(myTrainInput[x], learningRate)
myOutputLayer.optimize(myHiddenLayer.output(), learningRate)
```

The hidden layer's input is the original input. The output layer's input is the hidden layer's output (the one computed during feedforward). `learningRate` is the network's own, a local variable in `train()` rather than an argument, revised in step 4 below.

4.2.6 Step 4: Evaluation

Once every hundredth epoch, measure how close the network is to its training data and act on the number:

```
precision = 1.0 - MAE (over every value of every training set)

precision >= precisionThreshold  ->  stop training, the network is done
otherwise                        ->  updateLearningRate(): raise the rate when
                                     progress has stalled, lower it when the
                                     network is overshooting
```

The learning rate is therefore never supplied by the caller. It starts at `0.01` and is revised from one evaluation to the next, much as in `ml::lin_reg::Adaptive` from Chapter 2, which evaluates every tenth epoch instead: a linear regression model converges in tens of epochs, a network in thousands.

4.2.7 Connection to the theory

These three steps map directly onto the theory from Chapter 3 (see §3.1):

Step	Theory (§3.1)	Code
Feedforward	$y = \sigma(b + \sum w_i x_i)$	`feedforward(input)`
Backpropagation	$\Delta e = \delta \cdot y'$	`backpropagate(...)`
Optimization	$w = w + \Delta e \cdot L \cdot x$	`optimize(input, learningRate)`

4.2.8 Input validation

Check that training is possible before the training loop starts. Print an error message and return `false` immediately if any of the following conditions hold:

Condition	Explanation
`epochCount == 0`	Training must run for at least one epoch.
`precisionThreshold <= 0.0 || precisionThreshold >= 1.0`	Invalid threshold: must be in the range `(0.0, 1.0)`. A threshold of `1.0` or more can never be reached; `0.0` or less is reached by any network at all.

There's no learning rate among them: the network sets its own, and the layers check the rate they're handed on every call to `optimize()`. Missing training data isn't among them either: it never reaches `train()`, because the constructor already terminates when there's no complete training set.

4.3 Review

What you should be able to do now

After this chapter, you should:

- Have created an interface for a neural network.
- Be able to connect components via interfaces.
- Have implemented a concrete class for a simple neural network, including a fully working training method.
- Be able to explain how a model can set its own learning rate, and why a training method that resets its parameters first is worth having.

Questions to test yourself

You should be able to explain:

1. Why is it beneficial to define an interface for the neural network rather than implementing a class directly?
2. Without looking at the code, can you describe how the components in the network are connected via interfaces?
3. Can you explain the flow from input to prediction in your implementation, step by step?

4. Why doesn't `feedforward()` return the layer's output, and how does the rest of the network access it instead?
5. Why is the precision evaluated every hundredth epoch here, rather than every tenth as in Chapter 2, and what does the network do with the number?
6. Why does `train()` reset both layers before its first epoch, and what would calling it twice do without that reset?

Looking ahead

Chapter 5 implements a real dense layer in software, replacing the stub used since Chapter 3.

4.4 Exercises

How to work these. Both sets extend one program, the `ml` codebase you started in §3.4, already carried forward into the lecture's exercises directory, `lectures/L04/exercises` in the companion repository, together with a ready Makefile and `main.cpp`. Set 1 declares the neural network class; Set 2 completes it with a full training method. The network trains with an adaptive learning rate, following the same rule `ml::lin_reg::Adaptive` uses in Chapter 2, so no learning rate has to be chosen anywhere in the program.
How to check your work. The test suite is already in `lectures/L04/exercises/test`, and Exercise 4.14 runs it. It's cumulative: it carries the stub tests from Chapter 3 over unchanged and adds component tests for `Shallow`, so it supersedes the suite from §3.4 entirely. It tests the directory above it by default; `make ML_DIR=<path>` points it at a tree of your own elsewhere. The solution is published in the companion repository after the lecture, and `lectures/L04/README.md` links to it. Try each exercise yourself before looking at it.

EXERCISE SET 1

The Neural Network Class

You'll extend the codebase from Chapter 3, already carried over into this lecture's exercises directory, with an interface and a class for a simple neural network consisting of a hidden layer and an output layer, built on top of the `dense_layer::Interface/Stub` pair you already have.

Exercise 4.1 Directory structure

CODE

This lecture's exercises directory, `lectures/L04/exercises`, is already set up, so there's nothing to create or copy:

```
exercises/
├── include/
│   └── ml/
│       ├── dense_layer/
│       │   ├── interface.hpp    <- from Chapter 3, ready to use
│       │   └── stub.hpp         <- from Chapter 3, ready to use
│       ├── neural_network/
│       │   ├── interface.hpp    <- to implement
│       │   └── shallow.hpp      <- to implement
│       ├── types.hpp            <- from Chapter 3, ready to use
│       └── utils.hpp            <- to implement
├── source/
│   ├── ml/
│   │   ├── neural_network/
│   │   │   └── shallow.cpp      <- to implement
│   │   └── utils.cpp            <- to implement
│   └── main.cpp                 <- ready to use, see Exercise 4.9
├── test/                        <- ready to use, see Exercise 4.14
└── Makefile                     <- ready to use
```

The dense layer interface, the stub, and `types.hpp` are carried over from Chapter 3 unchanged. Each file you're to implement holds a `@todo` comment saying what goes in it; replace it with your code. Note that the sources now sit under `source/ml/`, mirroring `include/ml/`. The makefile already compiles `source/ml/neural_network/shallow.cpp` and `source/ml/utils.cpp` along with `main.cpp`.

Exercise 4.2 Utility functions CODE

Some helpers don't belong to any one class. The first of them is needed in this chapter, and Chapter 5 adds three more to the same pair of files.

The header (`ml/utils.hpp`)

In the namespace `ml`, declare a single function:

- **`initRandGen()`:**
  - Initializes the random number generator, once per program run.
  - Takes no arguments and returns nothing.
  - Should be marked `noexcept`.

The implementation (`source/ml/utils.cpp`)

Define `initRandGen()` in the same namespace:

- Add a static local variable named `initialized` with an initial value of `false`. Being static, it keeps its value between calls, which is what makes the seeding happen only once.
- If `initialized` is already `true`, return early.
- Seed the generator with the current time by calling `std::srand(std::time(nullptr))`. Include `<cstdlib>` and `<ctime>` for these.
- Set `initialized` to `true` so the next call does nothing.

Why a shared file rather than an anonymous namespace? In Chapter 2 the same function started out in an anonymous namespace inside `fixed.cpp`, which was fine while one class needed it, and moved to a shared file the moment `Adaptive` needed it too. An anonymous namespace doesn't scale: it gives every `.cpp` its own private copy, each with its own `initialized` flag, so the generator gets reseeded once per file that asks for it. Seeding must happen once per *program*. A single definition in `utils.cpp`, shared through `utils.hpp`, gives exactly that.

Exercise 4.3 Interface for neural networks CODE

In the header file `ml/neural_network/interface.hpp`, add a namespace named `ml::neural_network`. In this namespace, implement an interface named `Interface`:

- **`~Interface()`:** should be set to `default` and marked `virtual` and `noexcept`.
- **`predict(input)`:** pure virtual. `input`: read-only floating-point vector of the input to base the prediction on. Returns a reference to a floating-point vector holding the predicted value. Should be marked `[[nodiscard]]` and `noexcept` (**not** `const`, since the layers' output is updated on every prediction).

Exercise 4.4 The Shallow class: declaration

CODE

In the header file `ml/neural_network/shallow.hpp`, add the namespace `ml::neural_network`. Implement a subclass named `Shallow` that inherits `Interface` via public inheritance. The class should be marked `final`.

- **`Shallow()`:** takes `hiddenLayer` and `outputLayer` (the network's hidden layer and output layer, `ml::dense_layer::Interface&`), plus `trainInput` and `trainOutput` (read-only, two-dimensional floating-point vectors holding the training data's inputs and outputs). Should be marked `explicit` and `noexcept`.
- **`~Shallow()`:** should be marked `default`, `noexcept`, and `override`.
- **`predict()`:** overrides the corresponding method in the interface. Should be marked `[[nodiscard]]`, `noexcept`, and `override`. Attributes aren't inherited, so the override needs its own `[[nodiscard]]` (see Chapter 3).
- **`train(epochCount, precisionThreshold = 0.999999)`:** trains the network (implemented in full in Set 2). `epochCount`: number of epochs to train (unsigned integer). `precisionThreshold`: precision at which training stops early (floating-point number). Default value: `0.999999` (99.9999%). Returns `true` if training was carried out, `false` otherwise. Should be marked `noexcept`.

There's no `learningRate` argument. The network picks its own rate while training and revises it as it goes, exactly as `ml::lin_reg::Adaptive` does in Chapter 2, so there's nothing for the caller to guess at. The rule itself is written in Exercise 4.11.

The class should also have the following private methods, used to train in a randomized order:

- **`initTrainOrder(setCount)`:** fills `myTrainOrder` (see below) with the indices `0, 1, 2 ... N-1`, where `N` is the number of complete training sets. `setCount`: that set count (unsigned integer). Returns nothing, and should be marked `noexcept`.
- **`randomizeTrainOrder()`:** shuffles the contents of `myTrainOrder` into a random order. For each index `i`, pick a random index `r` and swap `myTrainOrder[i]` and `myTrainOrder[r]`. Takes no arguments, returns nothing, and should be marked `noexcept`.

Each training set passes through the network in three steps, feedforward, backpropagation and optimization (see §4.2), and each step drives both layers. Give every step a private method of its own, so that `train()` reads as the three steps rather than the six layer calls behind them:

- **`feedforward(input)`:** feeds `input` through the hidden layer and then the output layer. `input`: read-only floating-point vector. Returns `true` if both layers accepted their input, `false` otherwise. Should be marked `noexcept`.
- **`backpropagate(reference)`:** computes the error in the output layer and then in the hidden layer. `reference`: read-only floating-point vector holding the reference values for the input last fed forward. Returns `true` if both layers succeeded, `false` otherwise. Should be marked `noexcept`.

- **`optimize(input, learningRate)`:** updates the parameters of the hidden layer and then the output layer. `input`: read-only floating-point vector holding the input last fed forward. `learningRate`: the learning rate to use (floating-point number). Returns `true` if both layers succeeded, `false` otherwise. Should be marked `noexcept`.

Why not let `train()` call `predict()`? A prediction is worth nothing but its return value, which is why `predict()` is `[[nodiscard]]`. `train()` doesn't want that value, only the feed-forward pass behind it, so calling `predict()` there would discard it and fail to compile under `-Werror`. `predict()` also can't say whether that pass succeeded, since it returns the output rather than a `bool`. `feedforward()` does both jobs: `train()` calls it and checks the result, and `predict()` calls it and returns the output layer's output. That's also why `feedforward()` isn't marked `[[nodiscard]]`: `predict()` has no way to pass the result on, so it ignores it.

One more private method is needed by the training method, implemented in Set 2:

- **`precision()`:** computes the network's precision over the whole training data. Takes no arguments and returns a floating-point number. Should be marked `[[nodiscard]]` and `noexcept`, but **not** `const`, unlike its namesake in Chapter 2: it calls `predict()`, which feeds both layers forward and therefore changes their output.

The learning rate rule that `train()` also needs doesn't belong in the class at all. It uses no member of `Shallow`, so it goes in an anonymous namespace in `shallow.cpp`, out of the header (see Exercise 4.11).

Feel free to add more (private) methods as needed.

Exercise 4.5 Removed constructors and operators CODE

Delete the class's default constructor, copy and move constructors, and the corresponding operators.

Exercise 4.6 Private member variables CODE

Add the following private member variables to `Shallow`:

- **`myTrainOrder`:** the indices of the training sets as unsigned integers (`ml::MatrixU32`), in the order they'll be trained in. Initialized by the constructor and reshuffled once per epoch.
- **`myHiddenLayer`:** reference to the network's hidden layer, obtained via the constructor.
- **`myOutputLayer`:** reference to the network's output layer, obtained via the constructor.
- **`myTrainInput`:** reference to the training data's input, obtained via the constructor.
- **`myTrainOutput`:** reference to the training data's output, obtained via the constructor.

No member is needed for the learning rate either, nor for the previous precision. Both are local variables inside `train()`, handed to `updateLearningRate()` by reference, so every call starts from the same initial rate rather than inheriting whatever the previous call ended on.

No separate member is needed for the training set count. `myTrainOrder` holds one index per training set, so `myTrainOrder.size()` is that count, exactly as `myTrainOrder` replaced `mySetCount` in Chapter 2.

Exercise 4.7 Constructor and prediction

CODE

Implement the following in `source/ml/neural_network/shallow.cpp`:

The constructor

- Initialize all member variables as described above.
- Print an error message and call `std::terminate()` (from `<exception>`) if the output layer's weight count doesn't match the hidden layer's node count. The two layers can never be connected in that case, and the constructor has no way to return a failure code to the caller.
- Compute the number of complete training sets, i.e. the smaller of `trainInput.size()` and `trainOutput.size()`. Print an error message and call `std::terminate()` if it's 0, as in Chapters 1 and 2: a network without training data can never be trained.
- Fill `myTrainOrder` by calling `initTrainOrder()` with that set count.
- Call `initRandGen()` (from `ml/utils.hpp`, see Exercise 4.2), so the generator is seeded before the first shuffle. Doing it here rather than inside `randomizeTrainOrder()` keeps the one-time setup out of a method called once per epoch.

The method `feedforward()`

- Perform feedforward through the entire network:
  1. Call `myHiddenLayer.feedforward(input)` with the given input.
  2. Call `myOutputLayer.feedforward(myHiddenLayer.output())` with the hidden layer's output as input.
- Return `false` as soon as either call fails, without feeding the output layer if the hidden layer rejected the input. Return `true` otherwise.

The method `predict()`

- Call `feedforward(input)`.
- Return `myOutputLayer.output()` (a reference; no separate storage variable is needed in `Shallow`).

Exercise 4.8 Training method (placeholder) CODE

Implement a temporary version of `train()` in `source/ml/neural_network/shallow.cpp` that simply returns `true`. The full implementation (feedforward, backpropagation, and optimization for each training set and epoch, plus the precision check that sets the learning rate) follows in Set 2.

Give the private methods that only training uses temporary bodies too, so the header and the source file stay in step until Set 2 fills them in:

- `backpropagate()` and `optimize()` return `false`, so a method you forget to finish reports failure rather than quietly claiming success.
- `precision()` returns `0.0`.

Exercise 4.9 Compiling and testing CODE

`main.cpp` is provided in full. Read it before you run it:

- It creates one `ml::dense_layer::Stub` instance for the hidden layer (3 nodes with 2 weights per node) and one for the output layer (1 node with 3 weights per node). The output layer's weight count matches the hidden layer's node count, which is what connects the two.
- It creates an `ml::neural_network::Shallow` instance from these two layers and the 2-bit XOR pattern as training data.
- The helper function `trainAndTest()` prints a prediction for each training input, trains the network for 100 epochs with the default precision threshold, and prints the predictions again. `main()` returns `0` if that succeeds, and `-1` otherwise.

A few more helper functions in the anonymous namespace keep `main()` short:

- **`printSeparator()`:** prints a line of 80 dashes.
- **`printMatrix(matrix)`:** prints the values of a vector separated by spaces, without a trailing newline.
- **`printPredictions(header, network, trainInput)`:** prints `header` followed by a colon, then one line per training input holding the input and the network's prediction. It takes the network as an `ml::neural_network::Interface&` rather than a `Shallow&`: a prediction is all it needs, so it works for any network.

`trainAndTest()`, on the other hand, takes a `Shallow&`, since `train()` isn't part of `ml::neural_network::Interface`. Note that `train()` is called without a learning rate; the network picks its own.

Build and run the program with `make`. The placeholder `train()` from Exercise 4.8 returns `true` straight away, so the program runs to the end already. You should get the following output (the dense layers are still stubs, so the prediction is always 0.5):

```
--------------------------------------------------------------------------------
Predictions before training:
Input: 0 0, predicted output: 0.5
Input: 0 1, predicted output: 0.5
Input: 1 0, predicted output: 0.5
Input: 1 1, predicted output: 0.5
--------------------------------------------------------------------------------
Predictions after training:
Input: 0 0, predicted output: 0.5
Input: 0 1, predicted output: 0.5
Input: 1 0, predicted output: 0.5
Input: 1 1, predicted output: 0.5
--------------------------------------------------------------------------------
```

EXERCISE SET 2

Training the Neural Network

You'll complete the class `ml::neural_network::Shallow` you just built by implementing a full training method: first the precision it measures itself by, then the rule that turns that measurement into a learning rate, and finally the training loop that uses both.

Exercise 4.10 Precision calculation

CODE

Implement the private method `precision()` in `source/ml/neural_network/shallow.cpp`. It measures how close the network currently is to its training data, the same mean absolute error (MAE) the models in Chapter 2 measure, computed over vectors rather than single values.

Start with a helper function named `averageAbs()`, in an anonymous namespace at the top of `shallow.cpp`:

```
[[nodiscard]] double averageAbs(const Matrix1d& x, const Matrix1d& y) noexcept
```

- Compute the number of comparisons as the smaller of `x.size()` and `y.size()`.
- Return `0.0` if there's nothing to compare.
- Otherwise, sum `std::abs(x[i] - y[i])` for every index `i` below that count, and return the sum divided by the count, i.e. the average absolute difference between the two vectors.

Include `<cmath>` in `shallow.cpp` for `std::abs`.

Then implement `precision()` itself:

- Declare an error sum starting at zero.
- For each training set:
  - Call `predict()` with the set's input and keep the returned reference.

  - Add `averageAbs(reference, prediction)` to the error sum, where `reference` is the set's training output.
- Compute the MAE by dividing the error sum by the number of training sets, `myTrainOrder.size()`.
- Return `1.0 - MAE`.

Why average each set first? Chapter 2 divides by the number of training sets, because a linear regression model produces exactly one value per set. Here a set holds one value per output node, so summing the raw errors and dividing by the set count would report an error that grows with the width of the output layer. Averaging within each set first keeps every set's contribution on the same scale, whatever the width, so `1.0 - MAE` reads the same for a network with one output node as for one with ten. Since every set holds the same number of values (see below), the mean of the per-set means is exactly the average error *per value*.

Why every set has the same width. `train()` only calls `precision()` at the end of a complete epoch, and every epoch has already passed each training output through `myOutputLayer.backpropagate()`, which returns `false` when the row's length doesn't match the layer's node count. A row of the wrong length therefore ends training before `precision()` is ever reached, so every reference row and every prediction holds exactly one value per output node. `averageAbs()` still compares only up to the shorter of its two vectors, so it can never read out of range even when called from somewhere without that guarantee.

Exercise 4.11 The learning rate rule

CODE

The learning rate is the one training parameter with no sensible default, so the network sets its own, using the precision it just computed. Add a function named `updateLearningRate()` to the anonymous namespace from Exercise 4.10, as Chapter 2 does in `adaptive.cpp`:

```
void updateLearningRate(double& learningRate, double& prevPrecision,
                        const double currentPrecision) noexcept
```

Why not a private method? The rule touches nothing but its arguments, so it has no business in the class, and declaring it in `shallow.hpp` would expose an implementation detail to every file that includes the header. The anonymous namespace makes it visible in `shallow.cpp` only.

It takes the current learning rate and the previous precision by reference, since it updates both, and the precision just measured by value. Give each of the five values it needs a named constant: a maximum learning rate of `0.25`, a minimum of `0.01`, a smallest expected improvement of `0.1`, a step of `0.05`, and a decrease factor of `0.5`.

The rule works on the difference between the two precisions, i.e. how much the network improved since the last evaluation:

- **Improved by at least the expected minimum:** leave the learning rate alone. It's working.

- **Improved, but by less than that:** raise the learning rate by the step. Progress has stalled, so take bigger steps.
- **Didn't improve at all:** halve the learning rate. The network is overshooting, so take smaller steps.

As in Chapter 2, keep the limits out of the branches and apply them once afterwards with `learningRate = std::clamp(learningRate, minLearningRate, maxLearningRate)`. It needs `<algorithm>`, which `shallow.cpp` already includes for the constructor's `std::min()`.

Store the precision just measured in `prevPrecision` before returning, so the next call has something to compare against.

Why halve rather than subtract? Chapter 2 lowers the rate by the same step it raises it by. Here the rate goes up by a fixed step and down by a fixed factor, a scheme known as *additive increase, multiplicative decrease* (AIMD), the same rule TCP uses to adjust how fast it sends data. The cut scales with the rate: from `0.25`, repeated halving eases down through `0.125`, `0.0625` and so on, whereas subtracting `0.05` would take any rate below `0.06` straight to the floor after a single bad evaluation. And since a raise always adds `0.05`, a rate that was halved after one unlucky evaluation has more than recovered after the next one that shows progress.

The minimum differs from Chapter 2 too, which uses `1e-6`. The purpose the minimum is given there is that "a model that keeps missing can still creep towards the line rather than stopping dead", and in a network `1e-6` doesn't deliver it: the parameter updates it allows are far too small to measure, so the precision drifts *down* by a fraction of a millionth from one evaluation to the next, the rule reads that as another failure to improve, and the rate stays at the floor for the rest of the run. A network that sits on a plateau for enough evaluations can halve its way down there, and it never learns anything afterwards. With `0.01` the rate recovers on its own, since a network that is still moving improves sooner or later, and the rule raises the rate again the moment it does.

The initial rate sits at that floor as well, `0.01` rather than the `0.1` of Chapter 2, so the network starts cautiously and only speeds up once it has shown progress.

Exercise 4.12 Training method

CODE

Replace the temporary version of `train()` in `source/ml/neural_network/shallow.cpp` with a full implementation.

Input validation

- Return `false` if `epochCount == 0`, or if `precisionThreshold` falls outside the range `(0.0, 1.0)`:
  - A threshold of `1.0` or more can never be reached, since the precision is `1.0 - MAE` and the mean absolute error can't be negative. Training would always run the full epoch count.
  - A threshold of `0.0` or less accepts a network whose mean absolute error is `1.0` or worse, which defeats the purpose of the check.
- There's no learning rate to validate, since the caller no longer supplies one. The layers still check the rate they're handed on every call to `optimize()`, and the rule above keeps it between `0.01` and `0.25`, well inside the `(0.0, 1.0)` the layers demand.
- The training set count doesn't need checking here. The constructor already guaranteed at least one complete set, so `myTrainOrder` is never empty.
- As in Chapters 1 and 2, `train()` reports invalid arguments through its return value. Only a constructor calls `std::terminate()`, because it has no way to return a failure code to the caller.

Starting from scratch

- Once the arguments are known to be valid, and before the first epoch, reset both layers by calling `myHiddenLayer.initParams()` and `myOutputLayer.initParams()`. The method is part of `ml::dense_layer::Interface` from Chapter 3, so the network can call it without knowing which kind of layer it holds: `Stub` implements it as an empty method, and `Dense` implements it in Chapter 5 by drawing new values.
- Training therefore always starts from freshly initialized parameters. Calling `train()` a second time trains a new network rather than continuing the old one, which is what makes retraining a network that ended up badly trained worth doing at all: with the parameters carried over, a second call would resume from the same bad state it just failed in.
- This is a deliberate departure from `ml::lin_reg::Fixed` in Chapter 1, whose `train()` continues from the current parameters so that a model can be trained in stages. That only makes sense while the caller sets the learning rate. Here the schedule restarts from the initial rate on every call, so the rate the network picks would no longer match the parameters it's applied to.
- Do the reset **after** the argument checks, so a rejected call leaves a trained network exactly as it was.

Training

- Give the evaluation interval (`100`) and the initial learning rate (`0.01`) named constants.
- Declare the learning rate and the previous precision as local variables, initialized to the initial learning rate and `0.0`. Being local means every call starts from the initial rate again, rather than inheriting whatever the previous call ended on.
- Iterate the desired number of epochs with a for loop: `for (std::size_t epoch{}; epoch < epochCount; ++epoch)`.
- At the start of every epoch, call `randomizeTrainOrder()`. As in Chapter 2, reshuffling each epoch keeps the network from learning anything from the order the training data happens to be stored in.
- For each epoch, iterate through the training sets in the order `myTrainOrder` gives rather than sequentially, e.g. with a range-based for loop over `myTrainOrder`. Note that the loop variable is the index into the training data, not the counter itself.
- For each training set index `x`, perform the following three steps:
  1. **Feedforward:** call `feedforward(myTrainInput[x])`. This performs feedforward through both the hidden layer and the output layer.
  2. **Backpropagation:** call `backpropagate(myTrainOutput[x])`. This computes the error in both layers.
  3. **Optimization:** call `optimize(myTrainInput[x], learningRate)`, passing the current learning rate. This updates the parameters in both layers.

Each of the three calls returns `bool`. Return `false` as soon as any of them fails: a dimension mismatch means the network is wired wrong, and training on from there would only produce meaningless numbers.

Replace the temporary bodies of `backpropagate()` and `optimize()` with full implementations:

The method `backpropagate()`

1. Compute the error in the output layer: `myOutputLayer.backpropagate(reference)`.
2. Compute the error in the hidden layer from the output layer's error and weights: `myHiddenLayer.backpropagate(myOutputLayer)`.

The order is crucial: the hidden layer's error is computed from the output layer's, so the output layer has to go first.

The method `optimize()`

1. Optimize the hidden layer: `myHiddenLayer.optimize(input, learningRate)`.
2. Optimize the output layer based on the hidden layer's output: `myOutputLayer.optimize(myHiddenLayer.output(), learningRate)`.

As in `feedforward()`, each method returns `false` as soon as either layer call fails, without making the second call, and `true` otherwise. Each layer call returns `bool` (see Chapter 3).

Evaluating the training progress

- Every hundredth epoch, except the first, compute the precision once and keep it in a local variable:
  - Stop training and return `true` if it reaches or exceeds `precisionThreshold`, printing the achieved precision and the number of epochs it took. Report the number of epochs completed, not the loop index, so the first epoch reads as 1 rather than 0.
  - Otherwise, pass it to `updateLearningRate()`.
- Chapter 2 evaluates every tenth epoch, since a linear regression model reaches its threshold in tens of epochs. A network needs thousands, so a tenth-epoch evaluation would scan the whole training data hundreds of times for a picture that has barely changed. Every hundredth keeps that overhead down without letting the learning rate go unexamined for long.
- Note that the precision is computed once per evaluation and then used twice. Calling `precision()` a second time inside `updateLearningRate()` would feed every training set through the network again for a value that's already in hand.

Return value

`false` on invalid arguments or a failed layer call, otherwise `true` once the threshold is reached or the epochs run out.

Exercise 4.13 Compiling and testing CODE

`main.cpp` needs no changes: it already calls `train()` between the two rounds of predictions, and keeps using `ml::dense_layer::Stub` for both layers until a concrete dense layer implementation exists (see Chapter 5).

Build and run the program again with `make`. The output is the same as in Exercise 4.9, only now the full training loop runs behind it.

Don't worry if the network doesn't predict correctly; the dense layers are still stubs and therefore don't actually train. What matters is that the code compiles and runs without errors; a concrete implementation of `Dense` is added in Chapter 5.

The early-stop line stays absent for the same reason: the stub reports the same output whatever it's fed, so the precision sits at whatever that fixed output happens to score and never moves. Every run trains for the full epoch count. Your `train()` isn't wrong; there's simply nothing there to learn until Chapter 5.

Exercise 4.14 Running the tests CODE

An updated test suite is available in `lectures/L04/exercises/test`. It's cumulative: it carries the stub tests from Chapter 3 over unchanged and adds component tests for `Shallow` on top, so it supersedes the suite from §3.4 entirely.

Once you've implemented the exercises above, build and run it from the exercises directory:

```
make -C test
```

All 30 test cases should pass. The ones worth reading check that a prediction reads the output layer *live* rather than from a stored copy, that training performs exactly one feedforward per training set per epoch, and that the precision is evaluated on every hundredth epoch and no other. Nothing else can tell a correct training loop from one that runs a single pass: both line up dimensionally and both return `true`.

The test suite's README, `lectures/L04/exercises/test/README.md`, has more information, including the mistakes this suite deliberately can't catch and what it would take to catch them.

CHAPTER 5

The Dense Layer

The stub replaced by a layer that learns.

IN THIS CHAPTER

- Implementing a dense layer in software, using the code from Chapter 4:
  - A concrete subclass `Dense`, taking over from the existing stub class.
  - A handful of activation functions.
- Completing the implementation: feedforward, backpropagation, and optimization.

This chapter replaces the stub the network has been wired to since Chapter 3 with a dense layer that computes something. You'll create a concrete subclass that inherits the existing dense layer interface, implement all of its methods, including `initParams()`, `feedforward()`, `backpropagate()`, and `optimize()`, and then test it with the complete neural network from Chapter 4. There's no learning rate to adjust while you do: the network sets its own, and stops as soon as it reaches its precision threshold.

§5.1 describes the dense layer's architecture, and §5.2 maps the feedforward, backpropagation, and optimization equations from §3.1 onto the code. The exercises in §5.4 declare the class and implement it, method by method.

5.1 Dense Layer Architecture

5.1.1 Introduction

- A **dense layer** (*fully connected layer*) is a layer in a neural network where every node receives all inputs from the previous layer.
- The class `ml::dense_layer::Dense` (see §5.4) implements the interface `ml::dense_layer::Interface` and represents such a layer, whether it's used as a hidden layer or output layer in the network.

5.1.2 Structure of a layer

A dense layer with `nodeCount` nodes and `weightCount` weights per node consists of the parts described below. These map directly onto the theory in Chapter 3 (see §3.1):

Member variable	Size	Corresponds to (see §3.1)
`myOutput`	`nodeCount`	y, each node's output (after the activation function)
`myPreActOutput`	`nodeCount`	s, each node's weighted sum (before the activation function)
`myError`	`nodeCount`	Δe, each node's computed error
`myBias`	`nodeCount`	b, each node's bias
`myWeights`	`nodeCount` × `weightCount`	w, each node's weights
`myActFunc`	none	σ, the layer's activation function

5.1.3 Activation function

Each layer is assigned an activation function of type `ActFunc` (`Relu`, `Tanh`, or `None`) at construction. The activation function is applied to every node in the layer during feedforward (implemented in the exercises).

5.1.4 Composing a neural network

A simple neural network (`ml::neural_network::Shallow`, see Chapter 4) consists of two dense layers: a hidden layer and an output layer. The hidden layer's outputs are the output layer's inputs, so the output layer's `weightCount` equals the hidden layer's `nodeCount`.

5.2 From Math to Code

This section summarizes how the equations for feedforward, backpropagation, and optimization from Chapter 3 (see §3.1) map onto the code you write in `Dense` (see §5.4 for the full instructions).

5.2.1 Feedforward

$$s = b + \sum_{i=0}^{n-1} w_i \cdot x_i, \qquad y = \sigma(s)$$

Math	Code
b	`myBias[i]`
w_i	`myWeights[i][j]`
x_i	`input[j]`
s	`sum` (also stored in `myPreActOutput[i]`, see below)
$\sigma(s)$	`actFuncOutput(myActFunc, sum)`
y	`myOutput[i]`

`myPreActOutput[i]` is needed for backpropagation below, since the activation function's derivative y'_p must be computed from s (the weighted sum before the activation function was applied), not from y (the output after the activation function).

5.2.2 Backpropagation

Output layer:

$$\delta = y_{\text{ref}} - y_p, \qquad \Delta e = \delta \cdot y'_p$$

Math	Code
y_{ref}	`reference[i]`
y_p	`myOutput[i]`
δ	`err`
y'_p	`actFuncDelta(myActFunc, myPreActOutput[i])`
Δe	`myError[i]`

Hidden layer:

$$\delta = \sum_{i=0}^{n-1} \left[\Delta e_i \cdot w_i\right], \qquad \Delta e = \delta \cdot y'_p$$

Math	Code
Δe_i (next layer)	`nextLayer.error()[j]`
w_i (next layer)	`nextLayer.weights()[j][i]`
δ	`err`
y'_p	`actFuncDelta(myActFunc, myPreActOutput[i])`
Δe	`myError[i]`

Δe is the *negative* gradient of the cost with respect to the node's weighted sum s, which is why `optimize()` adds it rather than subtracting it. Most literature calls it the *delta* or the *error term*. It's kept as `error()` here because that's what the layer stores: one value per node.

It is not the same thing as the `inputGradients()` you'll meet in Chapter 8 (see the flatten layer in Chapter 10 for the clearest illustration of it), which holds one value per *input* and

is what a layer hands back to the layer before it. A hidden layer here fetches Δe from its neighbour itself with `nextLayer.error()`, so no layer ever needs to publish that second vector.

5.2.3 Optimization

$$\Delta c_n = \Delta e_n \cdot L, \qquad b_n = b_n + \Delta c_n, \qquad w_j = w_j + \Delta c_n \cdot y_j$$

Math	Code
Δe_n	`myError[i]`
L	`learningRate`
b_n	`myBias[i]`
w_j	`myWeights[i][j]`
y_j (previous layer's output)	`input[j]`

5.2.4 Helper functions `initRandGen()` and `randomStartVal()`

These don't correspond to any specific equation, but handle the random initialization of weights and biases described in Chapter 3. Both live in `ml/utils`, alongside the activation-function helpers, rather than inside the layer itself:

- `initRandGen()` ensures the random number generator (`std::rand()`) is seeded exactly once, regardless of how many layers are created. It's the same function you wrote in Chapter 4, which is exactly why it lives in a shared file.
- `randomStartVal()` generates the actual random number in $[-1.0, 1.0]$ that each bias and weight value is initialized with. The range is closed at both ends: the function first divides by `RAND_MAX`, which gives a number in $[0.0, 1.0]$, and then rescales it with `2.0 * x - 1.0`. A draw of exactly `0` therefore gives exactly `-1.0`, and a draw of exactly `RAND_MAX` gives exactly `1.0`.
- The start values are symmetric around zero, with a mean of 0.0. That matters for two reasons:
  - With only positive start values, a larger input always gives a larger weighted sum in every node. A network that must learn XOR, where `{1, 1}` should give `0`, then starts out ranking `{1, 1}` highest, and training must undo that before it can make progress.
  - `Tanh` layers with all positive start values go straight into saturation, where the derivative is close to zero and training is slow.
- Note this is still a deliberately simple choice for the course. Production networks also scale the range by the number of inputs (Xavier or He initialization).

5.3 Review

What you should be able to do now

After this chapter, you should:

- Have created a concrete subclass that inherits the existing dense layer interface.
- Have implemented all methods, including `initParams()`, `feedforward()`, `backpropagate()`, and `optimize()`.
- Have created a fully functioning dense layer implementation.
- Have tested the implementation with a complete neural network (the one created in the exercises in Chapter 4).

Questions to test yourself

You should be able to explain:

1. What's the difference between `ml::dense_layer::Interface` and your concrete class, and why is this split beneficial?
2. Which activation functions does your implementation support, and when is each one suitable?
3. Can you connect the mathematical formulas for backpropagation to your implementation; what does each computation step in the code represent?
4. How do you test that the implementation is correct, and how do you know when the network is sufficiently trained?

Looking ahead

Chapter 6 begins Part II of the book, and introduces convolutional neural networks (CNNs) for image classification.

5.4 Exercises

How to work these. Both sets extend the `ml` codebase from §4.4: carry it forward into the lecture's exercises directory, `lectures/L05/exercises` in the companion repository. Set 1 declares the `Dense` class, and Set 2 implements its methods, so your network finally runs on a layer that computes something rather than the stub it's been wired to since Chapter 3. See §5.1 for a walkthrough of the dense layer's architecture before you start.
How to check your work. The final test suite of Part I is already in `lectures/L05/exercises/test`, and Exercise 5.17 runs it. It's cumulative: it carries the stub tests from Chapter 4 over unchanged, updates the network tests for the new return value of `train()` (see Exercise 5.15),

and adds unit tests for `Dense` and for the helpers in `ml/utils`. It tests the directory above it by default; `make ML_DIR=<path>` points it at a tree of your own elsewhere.
The solution is published in the companion repository after the lecture, and `lectures/L05/README.md` links to it. Try each exercise yourself before looking at it.

EXERCISE SET 1

Declaring the Dense Layer

You'll extend your `ml` codebase from Chapter 4 by implementing a concrete subclass `Dense`, which takes over from `ml::dense_layer::Stub` as the layer your network is built from. The stub itself stays in the codebase; see Exercise 5.16.

Exercise 5.1 The `ActFunc` enum class

CODE

In `ml/types.hpp`, define an enum class named `ActFunc`. This enum class should be usable to select the activation function for a given dense layer. Add the following enumerators:

- `Relu`: for ReLU (*Rectified Linear Unit*), which for a given input x returns x if $x > 0$, otherwise 0.
- `Tanh`: for hyperbolic tangent, which produces output in the range $(-1, 1)$.
- `None`: for no activation at all, i.e. the identity function, which returns x unchanged. Useful on an output layer that has to produce values outside the range a bounded activation function can reach.

Exercise 5.2 Create `dense.hpp`

CODE

Create a new file named `dense.hpp` in `include/ml/dense_layer`. The file path should therefore be `include/ml/dense_layer/dense.hpp`, and the file should be includable as shown below:

```
#include "ml/dense_layer/dense.hpp"
```

Exercise 5.3 Create `dense.cpp`

CODE

Create a new file named `dense.cpp` in `source/ml/dense_layer`. The file path should therefore be `source/ml/dense_layer/dense.cpp`.

Note! Don't forget to add this file to your makefile! The only change is the new `source/ml/dense_layer/dense.cpp` line in `SRC_FILES`:

```
# Target application.
TARGET := neural_network

# C++ compiler.
```

```
CXX_COMPILER := g++

# C++ compiler flags.
CXX_FLAGS := -Wall -Werror -std=c++17 -Iinclude

# Source files.
SRC_FILES := source/main.cpp \
             source/ml/dense_layer/dense.cpp \
             source/ml/neural_network/shallow.cpp \
             source/ml/utils.cpp \

# Build and run the target by default.
default: build run

# Build the target.
build:
    @$(CXX_COMPILER) $(SRC_FILES) -o $(TARGET) $(CXX_FLAGS)

# Run the target.
run:
    @./$(TARGET)

# Clean the target.
clean:
    @rm -f $(TARGET)

.PHONY: default build run clean
```

Exercise 5.4 The `Dense` class: declaration CODE

In the header file `include/ml/dense_layer/dense.hpp`, in the namespace `ml::dense_layer`, add a class named `Dense` that inherits the corresponding interface, see the file `include/ml/dense_layer/interface.hpp`:

- Use public inheritance and mark the class `final` so it can't be inherited further.
- Override every method from the interface, including the destructor.
- Keep `[[nodiscard]]` on every override whose interface method has it. Attributes aren't inherited, so the override needs its own (see Chapter 3).
- That includes `initParams()`, which `Stub` implements as an empty method for want of anything to reset (see Chapter 3). `Dense` has bias values and weights to draw again, so its override does real work; the implementation follows in Exercise 5.10.

Tip: copy the entire contents of the interface and paste it into the new file. Then adapt the code for the new subclass `Dense` (no `virtual` or `= 0`, use `override`, etc.).

Exercise 5.5 Removed constructors and operators CODE

Delete the following:

- The default constructor.
- The copy constructor.
- The move constructor.
- The copy operator.
- The move operator.

Exercise 5.6 Constructor CODE

Create a constructor that can be used to create a dense layer with arbitrary dimensions and a selectable activation function.

- **Takes:**
  - `nodeCount`: the number of nodes in the layer (unsigned integer).
  - `weightCount`: the number of weights per node in the layer (unsigned integer).
  - `actFunc`: the activation function to use (of type `ActFunc`). `ActFunc::Relu` should be used as the default.
- Should be marked `explicit` and `noexcept`.
- If `nodeCount` or `weightCount` equals 0, an error message should be printed and the program terminated by calling `std::terminate()`.

Exercise 5.7 Private member variables CODE

Add the following private member variables to the class:

- `myOutput`: vector holding the nodes' output (floating-point). Should hold `nodeCount` values.
- `myPreActOutput`: vector holding the nodes' weighted sum before the activation function has been applied (floating-point). Should hold `nodeCount` values. Used by `backpropagate()` (implemented in Set 2) to correctly compute the activation function's derivative.
- `myError`: vector holding the nodes' error (floating-point). Should hold `nodeCount` values.
- `myBias`: vector holding the nodes' bias values (floating-point). Should hold `nodeCount` values.
- `myWeights`: two-dimensional vector holding the nodes' weights. Should hold `nodeCount` × `weightCount` values.
- `myActFunc`: the layer's activation function (of type `ActFunc`).

Exercise 5.8 Defining the methods CODE

Define all methods, constructors, etc. that aren't marked `delete` or `default` in the file `source/ml/dense_layer/dense.cpp`.

Constructor

- In the constructor, all member variables should be initialized:
  - `myOutput`, `myPreActOutput`, and `myError` should hold `nodeCount` floating-point values equal to 0.0 at the start.
  - `myBias` should hold `nodeCount` floating-point values randomized in $[-1.0, 1.0]$.
  - `myWeights` should hold `nodeCount` × `weightCount` floating-point values randomized in $[-1.0, 1.0]$.
  - The randomizing itself is written once, in `initParams()`, and called from here; see Exercise 5.10.
  - `myActFunc` should be assigned the given activation function.
- If `nodeCount` or `weightCount` equals 0, the error message below should be printed to standard error, after which the program should terminate by calling `std::terminate()`, as shown in the code after it:

```
Invalid dense layer parameters: nodeCount and weightCount must be greater than 0!
```

```cpp
#include <cstdio>
#include <exception>

if ((0U == nodeCount) || (0U == weightCount))
{
    std::fprintf(stderr, "Invalid dense layer parameters: "
        "nodeCount and weightCount must be greater than 0!\n");
    std::terminate();
}
```

Other methods

- Implement every method from the interface except `feedforward()`, `backpropagate()`, `optimize()`, and `initParams()`.
- Follow the descriptions in `include/ml/dense_layer/interface.hpp`:
  - Methods such as `nodeCount()` and `weightCount()` should return the number of nodes and weights per node in the layer, respectively.
  - Getter methods such as `output()` and `error()` should return references to the corresponding member variables.
  - `feedforward()`, both overloads of `backpropagate()`, and `optimize()` return `bool` (see Chapter 3), so an empty body won't compile. Give each one a placeholder body

of `return false;` until you implement it in Set 2. `false` rather than `true`, so that a method you forget to finish reports failure instead of quietly claiming success.
- `initParams()` returns nothing, so an empty body compiles fine. Leave it empty until Exercise 5.10, where the constructor starts calling it. That's the body the stub keeps for good; for `Dense` it's a placeholder.

EXERCISE SET 2

Implementing the Dense Layer

You'll complete the class `Dense` you just declared by implementing the methods `initParams()`, `feedforward()`, `backpropagate()`, and `optimize()`, then change `Shallow::train()` to report the precision it reached. See §5.2 for an overview of how the math from Chapter 3 maps onto the code below.

Exercise 5.9 Helper functions

CODE

`Dense` needs three helpers that don't belong to any one class. Add them to `ml/utils.hpp` and `source/ml/utils.cpp`, the pair you created in Chapter 4, in the namespace `ml`. Declare each one in the header and define it in the `.cpp`, alongside the `initRandGen()` already there.

- `randomStartVal()`: function to generate and return a random number in the range $[-1.0, 1.0]$, closed at both ends.
  - **Implementation:**
    - First generate a random number in the range $[0.0, 1.0]$ by calling `std::rand()`, which generates a random number in `[0, RAND_MAX]`, and dividing by `RAND_MAX`.
    - One of the operands must be converted to a floating-point number to avoid integer division, e.g. `static_cast<double>(RAND_MAX)`.
    - Then rescale the number to the range $[-1.0, 1.0]$ by multiplying by `2.0` and subtracting `1.0`. A draw of exactly `0` then gives exactly `-1.0`, and a draw of exactly `RAND_MAX` gives exactly `1.0`.
  - Should be marked `[[nodiscard]]` and `noexcept`.
  - **Note:** the start values must be able to go negative. If every weight and bias starts positive, the input `{1, 1}` starts out giving the largest weighted sum in every node, the opposite of what XOR asks for, and training must first drive some of the weights negative before the network gets anywhere. Start values symmetric around zero also start `Tanh` layers in the sensitive part of the function rather than in saturation, where the derivative is close to zero.
  - It belongs next to `initRandGen()`: both wrap `std::rand()`, and calling this one without having called that one first gives the same sequence on every run.
- `actFuncOutput()`: function to compute and return the output (floating-point) of a given activation function.

  - **Takes:**
    - `actFunc`: the activation function to use (of type `ActFunc`).
    - `value`: input value to the activation function (floating-point).
  - **Implementation:**
    - Use a switch statement to compute the output depending on the given activation function:
      - `ActFunc::Relu`: return `value` if `value > 0.0`, otherwise `0.0`.
      - `ActFunc::Tanh`: return `std::tanh(value)` (requires `#include <cmath>`).
      - Default case (`ActFunc::None`): return `value` unchanged, which is the identity function.
  - Should be marked `[[nodiscard]]` and `noexcept`.
- `actFuncDelta()`: function to compute and return the derivative (floating-point) of a given activation function.
  - **Takes:**
    - `actFunc`: the activation function to use (of type `ActFunc`).
    - `value`: input value to the activation function (floating-point).
  - **Implementation:**
    - Use a switch statement to compute the derivative depending on the given activation function:
      - `ActFunc::Relu`: return `1.0` if `value > 0.0`, otherwise `0.0`.
      - `ActFunc::Tanh`: compute `const auto tanhOutput = std::tanh(value)` and return `1.0 - tanhOutput * tanhOutput`.
      - Default case (`ActFunc::None`): return `1.0`, the derivative of the identity function.
  - Should be marked `[[nodiscard]]` and `noexcept`.

Neither activation function needs an error case. `ActFunc` has exactly three enumerators, and all three are handled, so the default case is `None` rather than something that shouldn't happen.

Exercise 5.10 Randomizing bias and weights CODE

Randomize all bias values and weights in `initParams()`, the interface method `Dense` overrides (see Exercise 5.4). It takes no arguments, returns nothing, and should be marked `override` and `noexcept`:

- Iterate through every node in the layer with a for loop: `for (std::size_t i{}; i < nodeCount(); ++i)`.
- For each node `i`:
  - Assign a random number to its bias value by calling `randomStartVal()`: `myBias[i] = randomStartVal()`.
  - Iterate through the node's weights with a nested for loop: `for (std::size_t j{}; j < weightCount(); ++j)`.
  - For each weight `j`:
    - Assign a random number to the weight by calling `randomStartVal()`: `myWeights[i][j] = randomStartVal()`.

In the constructor, first call `initRandGen()` (from `ml/utils.hpp`) to seed the random number generator, then call `initParams()` to fill the two containers. A layer therefore starts out randomized exactly as before; the randomization has simply moved into a method the network can call again.

Who calls it, and why the network needs it. `Shallow::train()` calls `initParams()` on both its layers before the first epoch (see Chapter 4), so every call trains a network from scratch rather than continuing the one the previous call left behind. That matters most when a run ends badly: training on top of a network that has settled into a bad set of parameters mostly keeps it there, while a fresh start draws new ones and can converge on the second attempt. Without a method on the interface, the network couldn't do it at all, since the bias and weights are private to the layer.

Exercise 5.11 The method `feedforward()` CODE

Input validation

- Check that the dimensions of the given input match the number of weights per node in the layer (`input.size() == weightCount()`).
- If the dimensions don't match: print the error message `"Input dimension mismatch: expected X, actual: Y!"` and return `false` without computing anything.

Computation for each node

- Iterate through every node in the layer with a for loop: `for (std::size_t i{}; i < nodeCount(); ++i)`.
- For each node `i`, compute the weighted sum:
  1. Start with the node's bias value: `auto sum{myBias[i]}`.

2. Add each weight multiplied by the corresponding input: `for (std::size_t j{}; j < weightCount(); ++j)` where `sum += myWeights[i][j] * input[j]`.

- Store the weighted sum before the activation function is applied: `myPreActOutput[i] = sum`. This value is needed by `backpropagate()` below to correctly compute the activation function's derivative.
- Apply the activation function to the sum: `myOutput[i] = actFuncOutput(myActFunc, sum)`.

Return value

`true` once every node has been computed.

Exercise 5.12 The method `backpropagate()` (output layer) CODE

Implement `backpropagate()` for output layers (with reference values).

Input validation

- Check that the size of the reference vector matches the number of nodes (`reference.size() == nodeCount()`).
- If the dimensions don't match: print the error message `"Output dimension mismatch: expected X, actual: Y!"` and return `false` without computing anything.

Error computation for each node

- Iterate through every node in the layer: `for (std::size_t i{}; i < nodeCount(); ++i)`.
- For each node `i`:

1. Compute the raw error: `const auto err{reference[i] - myOutput[i]}`.
2. Compute the gradient error: `myError[i] = err * actFuncDelta(myActFunc, myPreActOutput[i])`.
   Note! Use `myPreActOutput[i]` (the weighted sum before the activation function was applied in `feedforward()`), not `myOutput[i]`. `actFuncDelta()` expects the activation function's *input*, not its output; otherwise the derivative comes out wrong for `ActFunc::Tanh` (it happens to work for `ActFunc::Relu` by coincidence).

Return value

`true` once every node's error has been computed.

Exercise 5.13 The method `backpropagate()` (hidden layer) CODE

Implement `backpropagate()` for hidden layers (with error and weights from the next layer).

Input validation

- Check that the next layer's weight count matches this layer's node count (`nextLayer.weightCount() == nodeCount()`).
- If the dimensions don't match: print the error message `"Layer dimension mismatch: expected X, actual: Y!"` and return `false` without computing anything.

Error computation for each node

- Iterate through every node in this layer: `for (std::size_t i{}; i < nodeCount(); ++i)`.
- For each node `i`:
  1. Initialize a variable to store the computed raw error: `double err{}`.
  2. Sum all errors from the next layer:
     `for (std::size_t j{}; j < nextLayer.nodeCount(); ++j)`
     – `err += nextLayer.error()[j] * nextLayer.weights()[j][i]`.
  3. Compute the gradient error:
     `myError[i] = err * actFuncDelta(myActFunc, myPreActOutput[i])`
     (see the note in Exercise 5.12 for why `myPreActOutput[i]` is used instead of `myOutput[i]`).

Return value

`true` once every node's error has been computed.

Exercise 5.14 The method `optimize()` CODE

Input validation

- Check that the learning rate lies inside the range `(0.0, 1.0)`, the same range the stub checked in Chapter 3.
- If the learning rate is invalid: print the error message `"Invalid learning rate X!"` and return `false` without updating anything.
- Check that the input size matches the number of weights per node (`input.size() == weightCount()`).
- If the dimensions don't match: print the error message `"Input dimension mismatch: expected X, actual: Y!"` and return `false` without updating anything.

Parameter update for each node

- Iterate through every node in the layer: `for (std::size_t i{}; i < nodeCount(); ++i)`.
- For each node `i`:
  1. Update the bias: `myBias[i] += myError[i] * learningRate`.
  2. Update all weights: `for (std::size_t j{}; j < weightCount(); ++j)`
     - `myWeights[i][j] += myError[i] * learningRate * input[j]`

Return value

`true` once every node's bias and weights have been updated.

Exercise 5.15 Reporting the precision from `train()` CODE

In Chapter 4, `Shallow::train()` returns `true` whenever its arguments are valid, whether the network learned anything or not. With two stubs that was all there was to say, since nothing could be learned. Now that the network actually trains, the caller needs to know how well it did, for instance to decide whether to train again (see Exercise 5.16). Change `train()` to return the precision it reached instead.

`std::optional`. `train()` now has two kinds of result: a precision, or no result at all when training couldn't be carried out. `std::optional<double>`, from the header `<optional>`, holds exactly that, either a `double` or nothing:

- Returning a `double` from a function declared to return `std::optional<double>` stores it, so `return currPrecision;` works as written.
- `return std::nullopt;` returns an empty optional, i.e. no value.
- The caller checks for a value with `has_value()` (or simply `if (precision)`), and reads it with `*precision` or `precision.value()`. Only read it after checking: `*precision` on an empty optional is undefined behavior, while `value()` throws an exception.

Why not return `-1.0` on invalid arguments? The precision is `1.0 - MAE`, and the mean absolute error has no upper bound. A network whose predictions miss by `2.0` on average has a precision of exactly `-1.0`, so any negative number could be a real result. An empty optional can't be mistaken for a precision.

Declaration

In `ml/neural_network/shallow.hpp`, change the declaration of `train()` to

```
[[nodiscard]] std::optional<double> train(std::size_t epochCount,
                                          double precisionThreshold = 0.999999) ↩
    ↪ noexcept;
```

- Include `<optional>` in `shallow.hpp`.

- Mark it `[[nodiscard]]`: a caller that ignores the result has no idea whether training worked.
- Update the `@return` line of its doc comment to match.

Implementation

In `source/ml/neural_network/shallow.cpp`, update the return type of the definition, then change each `return`:

- Replace every `return false;` with `return std::nullopt;`: the two argument checks, and the three layer calls inside the epoch loop. Each of them means training couldn't be carried out at all.
- At the early stop, return the precision you just measured, `currPrecision`, instead of `true`. The early stop line printed there stays as it is.
- After the epoch loop, replace `return true;` with `return precision();`. Measure once more rather than returning the last evaluated precision: that value is up to 99 epochs old, and a run of 100 epochs or fewer has never been evaluated at all.

Return value

`std::nullopt` on invalid arguments or a failed layer call. Otherwise the precision the network reached: at least `precisionThreshold` if training stopped early, and whatever the final measurement gives if the epochs ran out.

Exercise 5.16 Compiling and testing CODE

Update `main.cpp` to use your new layer:

- Replace the two `ml::dense_layer::Stub` instances with `ml::dense_layer::Dense`, keeping the same wiring: the output layer's weight count must still match the hidden layer's node count.
- Make the hidden layer wider than the task strictly needs, 15 nodes, and leave its activation function at the default `ActFunc::Relu`. Give the output layer `ActFunc::Tanh`. A ReLU output node that dies takes the whole network with it, since every prediction is then `0`. Keep the 2-bit XOR training data from Chapter 4.
- Give the training parameters named constants in `trainAndTest()`: an epoch count of `1000000`, a target precision of `0.9995`, and a retry count of `10`.
- Pass the target precision to `train()` explicitly, as its precision threshold. The default of `0.999999` is practically never reached, so every call would run all 1000000 epochs and still fall short of it.
- In `printMatrix()`, print each value with two decimals (`"%.2f"`) instead of `"%g"`. The trained predictions differ from their targets by a few thousandths, which `"%g"` prints as long tails such as `0.000214783`.

Then train in a loop, calling `train()` at most once per retry:

- Call `train()` and keep the returned optional: `const auto precision = network.train(epochCount, targetPrecision)`.
- If it holds no value, print `"Training failed!"` to standard error and return `false`. The arguments are invalid or the network is wired wrong, so another call would fail the same way.
- If the precision reaches the target, print the predictions after training as before and return `true`.
- Otherwise, print the precision it reached, e.g. `"Precision 0.75 is below target, retrying..."`, and go round again.
- If every retry falls short, print `"Failed to train the network with 10 retries!"` to standard error (using the constant rather than a literal) and return `false`.

There's no learning rate to pass: the network sets its own, and stops as soon as it reaches the precision threshold, so the epoch count is an upper bound rather than a number to tune. Each call to `train()` resets both layers first (see Chapter 4), so every attempt starts from newly drawn parameters rather than the ones the previous attempt got stuck in.

Don't delete `stub.hpp`. `Dense` replaces the stub in your *network*, not in your codebase. The test suite still compiles both, and still uses the stub to test `Shallow`, since a layer that computes nothing is what makes the network's wiring visible. See Exercise 5.17.

Compile and test-run the program. Unlike Chapter 4, where every prediction was the stub's fixed output, each prediction should now land within a few thousandths of its training target, so it prints as `0.00` or `1.00`. A `-0.00` is fine too: the `Tanh` output can land a hair below zero. The exact values behind them differ from run to run, because the bias and weights start randomized.

The early stop line from `train()` should appear above the predictions, reporting a precision and an epoch count well short of the 1000000 you asked for. Both vary from run to run: the starting parameters are random, and so is the training order. A run that stops after about 120000 epochs and one that needs 850000 are both normal; the average is around 250000, which takes a fraction of a second.

When an attempt doesn't converge. Roughly two calls to `train()` in a hundred settle into a set of parameters they can't improve on, typically with one or two of the four predictions well off target however long they train. That's what the retry loop is for. A second call starts from scratch and nearly always gets there: in 1000 runs of the program, not one used up all ten retries. That's the reset from Chapter 4 earning its keep, and it's what the test suite's convergence test relies on as well.

Worth trying: switch the output layer to `ActFunc::Relu` as well and shrink the hidden layer to 2 nodes. Roughly six calls to `train()` in seven then fail to learn XOR at all, however long they train, and even with ten retries about one run of the program in five ends with `"Failed to train the network with 10 retries!"`. A node whose weighted sum is negative for every input has a derivative of exactly zero: it stops learning and can't recover. Since `randomStartVal()` draws from $[-1.0, 1.0]$, a node can start out that way, and with only three ReLU nodes in the network, about two networks in three contain one before training

has even begun. That's the dying ReLU problem from Chapter 3, and it's why the test suite trains with `Tanh`.

Exercise 5.17 Running the tests CODE

The final test suite is available in `lectures/L05/exercises/test`. It's cumulative: it carries the stub tests from Chapter 4 over unchanged, updates the network tests for the new return value of `train()` (see Exercise 5.15), and adds unit tests for `Dense` and for the helpers in `ml/utils`.

Once you've carried your code forward into this lecture's exercises directory, build and run it:

```
make -C test
```

All 64 test cases should pass. Because `Dense` randomizes its own bias and weights, none of its output can be predicted from the constructor arguments, so the tests recover the bias from the layer's output instead. Without an activation function, a vector of zeros is enough: every weight term drops out and the weighted sum is the bias alone. `Tanh` tests feed the same zeros and invert the output with `std::atanh()`. ReLU would clamp a negative bias to zero, so the ReLU tests instead lift each node's weighted sum by a known amount, far above any start value, and subtract it again. That makes every expected value exact rather than approximate.

`InitParamsDrawsNewValues` is the first test in the course where `initParams()` has anything to do. The stub tests only check that calling it changes nothing, which is all an empty override can be asked for; here the values it writes are read straight back out of `weights()` and out of a zero-input feedforward.

The one to read is `BackpropagateUsesPreActivationDerivative`. It checks the note in Exercises 5.12 and 5.13, and only `ActFunc::Tanh` can catch that mistake: for `Relu` the two values agree, since `max(0.0, s)` is positive exactly when `s` is.

The test suite's README, `lectures/L05/exercises/test/README.md`, has more information, including how the convergence threshold was chosen.

PART II

Convolutional Neural Networks

Chapters 6 to 10, taught as lectures L06 to L10. Why a network built only from dense layers handles images badly, and the convolutional, pooling and flatten layers that handle them well; each worked through by hand, written once as a simple struct, and then rebuilt behind an interface in a polymorphic pipeline. The part ends with every layer real, a network that tells four hand-drawn digits apart, a review of the whole book, and what changes when a network like it has to run on a microcontroller.

CHAPTER 6

Convolutional Neural Networks

Why an image is not just a long vector, and the layers that treat it as an image.

IN THIS CHAPTER

- Why networks built only from dense layers aren't well suited to image classification.
- The structure of a convolutional neural network (CNN): convolutional, pooling, flatten, and dense layers.
- Feature extraction via kernels.
- Feeding an input through a small CNN by hand, then backpropagating through it and optimizing its kernel.
- Starting a convolutional layer in software: the `ml::ConvLayer` struct.

This chapter opens the second part of the book, and the network it builds towards is one that classifies images. It explains why a regular neural network, built only from dense layers, handles images badly, and how a convolutional neural network is structured instead: which layer types it is built from, what each of them is for, and how a kernel slides across an image to extract its features. By the end of it you should be able to feed an input through a small CNN by hand, and to have started implementing a convolutional layer in software.

Read §6.1 and §6.2 first. The first exercise set in §6.4 then works a small CNN through by hand, feedforward first and then backpropagation and optimization, and the second starts the struct `ml::ConvLayer`, which runs the same numbers in C++. You don't need to finish that struct in this chapter: Chapter 7 finishes it, compiles it, and runs it for the first time.

6.1 Why Regular Neural Networks Struggle with Images

A regular neural network (consisting only of dense layers) can technically be used to process images, but in practice it rarely works well. Below we go through why, and what convolutional neural networks (CNNs) do differently.

6.1.1 Spatial structure is lost

An image is two- or three-dimensional, but has to be flattened into a one-dimensional vector to be fed into a regular neural network, which means the information about which pixels sat next to each other is lost.

6.1.2 The number of parameters explodes

A 1024 × 1024 pixel image requires over a billion weights already in a single hidden layer of 1000 nodes, resulting in a network that's large, memory-hungry, and slow.

6.1.3 Generalization gets worse

Since each pixel is treated as a completely separate input, the network doesn't understand that the same pattern can appear in different places in the image, which requires more training data and results in worse generalization to new images.

6.1.4 What CNNs do instead

Convolutional neural networks are built specifically for image data and solve this by preserving the image's spatial structure, using small local filters (kernels), and sharing the same weights across the entire image.

6.2 Convolutional Neural Networks (CNN)

6.2.1 Overview

A typical CNN consists of four types of layers:

1. **Convolutional layers** (hereafter *conv layers*) identify and extract local patterns and features in an image.
2. **Pooling layers** downsample the image, reducing its size while preserving the most important patterns.
3. **Flatten layers** flatten the two-dimensional image into a one-dimensional representation that can be fed into regular neural layers.
4. **Dense layers** make the final prediction based on the extracted features.

The first three layer types extract features from the image, while the dense layer handles the final interpretation.

6.2.2 Convolutional layers (conv layers)

A convolutional layer applies a *kernel*, a small filter of weights, across the entire input image. The same kernel is reused at every position in the image, which keeps the number of parameters down and lets the same pattern be recognized regardless of where in the image

it appears. Each kernel looks for a specific local pattern, for example an edge or a corner; in practice, several kernels are often used per conv layer to capture different patterns at once, but to keep things simple we only use a single kernel per conv layer in this book.

Two properties distinguish a conv layer from a regular dense layer:

- The computations happen locally, on a small window of the image at a time.
- The same weights (the kernel) are reused across the entire image instead of each node having its own.

Beyond that, it works exactly like a regular layer, with a weighted sum, a bias, and an activation function (usually ReLU). The layer's output is called a *feature map*, and the process of extracting these patterns is called *feature extraction*.

Feedforward: the kernel slides across the input image one step at a time (usually with `stride = 1`, i.e. one pixel at a time). For each position:

1. A small window is selected from the image (e.g. 2×2).
2. The values are multiplied by the corresponding kernel weights.
3. The results are summed together with a bias value.
4. The activation function is applied to the result.

The value at each position in the output therefore shows how strongly the kernel reacted to that particular spot in the image.

Backpropagation: just as with dense layers, the error is propagated backward through the layer to compute gradients, i.e. how much and in which direction each weight and bias value should be adjusted. The difference is that since the same kernel was used at every position in the image, the gradients from all those positions must be accumulated before the kernel's weights are updated. In other words, each kernel weight has been influenced by many different parts of the image, and backpropagation computes the combined contribution.

Optimization: works identically to dense layers; each kernel weight and the bias value are adjusted along their gradient, scaled by the learning rate. The gradient is added rather than subtracted, since the deviation is computed as $y_{\text{ref}} - y_p$ and so already points in the direction that reduces the error. The only difference is that there are far fewer parameters involved, since a conv layer consists of just a small kernel and a bias value rather than a unique weight per node and input.

6.2.3 Pooling layers

Pooling layers downsample the image so the network has less data to process, which both simplifies training and prediction and makes the network more robust to small variations in the input. Pooling layers have no trainable parameters at all. The input is divided into blocks (e.g. 2×2), and a single value is picked out of each block:

- **Max pooling:** the largest value in the block. By far the most common, and what we use in this book.
- **Average pooling:** the average of the block.

Feedforward: the largest value in each pooling block is passed forward. The most prominent features in each region are preserved, at the cost of detail; much like when an image is downsampled: it looks largely the same, but if you enlarge it again, you can see that detail has been lost.

Backpropagation: the gradients are only sent back to the positions that actually held the maximum value during feedforward; the remaining positions get a gradient of 0, since they didn't contribute to the output.

6.2.4 Flatten layers

After convolution and pooling, the data is still two-dimensional, while dense layers require one-dimensional input. The flatten layer solves this by reshaping the data from 2D to 1D. It performs no computations and has no weights of its own; it just changes the shape of the data. During backpropagation, it performs the corresponding operation in reverse, reshaping the gradient from 1D back to 2D.

6.2.5 Dense layers in a CNN

The dense layer works exactly as in a regular neural network. The only difference is that the input now already consists of extracted, compressed features rather than raw data, and the layer weighs these features together to make the final classification or regression.

6.2.6 CNNs in a nutshell

A CNN can be seen as a regular neural network with a few extra preprocessing layers specialized for spatial data:

- The conv layer looks for local patterns.
- The pooling layer keeps the most prominent ones and discards the rest.
- The flatten layer reshapes the data into a form the dense layer understands (one dimension).
- The dense layer finally performs the actual interpretation of the information.

Backpropagation follows the same chain in reverse:

- The dense layer adjusts how features are weighted.
- The flatten layer reshapes the gradients.
- The pooling layer routes the gradients to the correct positions.
- The conv layer finally adjusts the weights for the patterns that actually contributed to the prediction.

You'll see this whole pipeline wired together for the first time in Chapter 8's `cnn_work` (§8.1), running on stub layers at first; the real implementations are built into it across Chapters 8 to 10. Exercises 6.1 and 6.2 work the whole pipeline through by hand, on a small example.

6.3 Review

What you should be able to do now

After this chapter, you should:

- Know why regular neural networks aren't well suited to image classification.
- Know at a high level how convolutional layers are structured.
- Be able to extract features from images via kernels.
- Be able to feed an input through a small CNN by hand (feedforward).
- Have started implementing a conv layer in software (`ml::ConvLayer`).

Questions to test yourself

You should be able to explain:

1. Why are regular neural networks (dense layers only) less suited to image classification than a CNN?
2. Can you, in your own words, describe the purpose of each layer type in a CNN: convolutional, pooling, flatten, and dense?
3. Without looking at §6.2, can you describe how a kernel is applied across an image during feedforward in a conv layer?
4. Which parts of your `ml::ConvLayer` so far correspond to feedforward?

Looking ahead

Chapter 7 finishes the conv layer, then implements max pooling and flatten layers in software.

6.4 Exercises

How to work these. Set 1 is a calculation: work it on paper, Exercise 6.1 first, since Exercise 6.2 backpropagates through the values it produces. Set 2 is a program, and it isn't new: the file `lectures/L06/conv_layer/cpp/conv_demo.cpp` in the companion repository already holds a skeleton of the struct you write, its helper functions, and the `main()` that uses it, with a Makefile beside it. Get as far as you can with it here; Chapter 7 finishes it.

How to check your work. The worked solutions to Set 1 are in Appendix A. Set 2 has no test suite; instead, the demo runs the numbers of Set 1 and prints each result next to the value the hand calculation arrives at, so your implementation is checked against your own arithmetic. The same layer is implemented in the repository twice more, as references: in Python, in `lectures/L06/conv_layer/python`, and in C, in `lectures/L06/conv_layer/c`, where the layer is encapsulated behind an opaque struct whose accessor functions are only exposed through its header. Try the exercise yourself before looking at them.

EXERCISE SET 1

Hand-Training a CNN

We have the following convolutional neural network:

- Input: 4×4.
- Conv layer:
  - Kernel: 2×2.
  - Stride $= 1$ (the kernel moves one pixel at a time).
  - Zero-padding (we add zeros around the image so its size is preserved despite the extraction).
  - Activation: **ReLU** (all negative values are replaced with zero).
  - Learning rate: $LR = 0.001$, i.e. 0.1%.
- Max pool:
  - Pool size $= 2$.
  - Stride $= 2$ (the pools are non-overlapping).
- Flatten: $2 \times 2 \rightarrow 1 \times 4$.

The two exercises below work the whole training process through, step by step: feedforward through all three layers, then backpropagation and optimization.

Exercise 6.1 Feedforward

CALCULATION

Assume we have the following input image, resembling the digit 0 made up of ones:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

We use a kernel (a filter of weights) and a bias, with simple, fixed values chosen to make the calculations easy to follow:

$$\text{kernel} = \begin{bmatrix} 0.2 & 0.4 \\ 0.6 & 0.8 \end{bmatrix}, \qquad \text{bias} = 0.5$$

a) Padding the input image

The first step is *zero-padding*: we add zeros around the image so its size is preserved even after the kernel filter has been applied. This way, the conv layer's output has the same dimensions as the input, 4×4 in this case.

The number of zeros n we add on each side is computed as

$$n = \text{kernel_size}/2,$$

where `kernel_size` is the size of the kernel, 2 in this example, i.e. one zero on each side. Give the padded image, which is 6×6.

In general, sliding a $K \times K$ kernel with stride s over an $N \times N$ image padded by p on each side gives an output of size $\lfloor (N + 2p - K)/s \rfloor + 1$. For an **odd** kernel that lands exactly on N, which is the case this padding rule is designed for. For the **even** 2×2 kernel used here it comes out one larger, 5×5, because `n = kernel_size / 2` pads symmetrically for a kernel that has no centre. The layer keeps only the first 4×4 window positions, i.e. i and j both run from 0 to 3, which is what makes the output match the input size. Work out that 4×4 below and you have exactly what the code produces.

b) Feedforward through the conv layer

For each position in the image, compute the sum of the element-wise multiplication between the kernel and the corresponding part of the padded image, plus the bias, then apply ReLU (all negative values are replaced with zero):

```
sum = a * k00 + b * k01 + c * k10 + d * k11 + bias
output = ReLU(sum)
```

where:

- **a, b, c, d** are pixel values from the current 2×2 section of the image.
- **k** stands for kernel (the filter), e.g. `k00` means the value at row 0, column 0 of the kernel matrix, i.e. the top-left corner.
- **bias** is a constant value added to the sum.

Apply the kernel and bias across the entire image, and give the conv output (4×4).

c) The max pooling layer (feedforward)

The max pooling layer looks for the largest value in each 2×2 pool (non-overlapping) of the conv layer's output. The purpose is to downsample the image: we remove detail while keeping the most important features, which reduces the amount of data and makes the model less sensitive to small variations.

Split the conv layer's output into its four pools, and give the largest value of each, and the position it was found at. Where a pool holds its maximum more than once, pass forward the first instance. Then give the max pooling layer's output (2×2).

d) Flatten (feedforward)

Flatten the max pooling layer's output into a vector, so it can be fed to the next layer.

The worked solution is on page 164.

Exercise 6.2 Backpropagation and optimization

CALCULATION

Assume the dense layer sends back the following gradients:

$$[10, 20, 30, 40]$$

a) Flatten (backpropagation)

Reshape the gradients from the dense layer back into a matrix.

b) Max pool (backpropagation)

Propagate the gradients back to the correct positions in the max pooling layer, i.e. to the spots where the max values were located in each pool. If a pool holds its max value more than once, the gradient is propagated back to the first position; the remaining positions get a gradient of 0. Give the gradient for each pool, and then all four assembled into a single 4×4 matrix.

c) Conv layer (backpropagation)

Now compute the gradient for the bias and the kernel from the error propagated back.

One step is invisible here. Strictly, each incoming gradient is first multiplied by the activation function's derivative at that position, $\sigma'(s)$, where s is the pre-activation sum from feedforward. Every sum in this example is positive and the activation is ReLU, so every derivative is 1 and the multiplication changes nothing. It is not optional in code: the `Conv` class you write in Chapter 8 applies it explicitly, in the `backpropagate()` specified in §8.3.

Bias gradient: the sum of all values in the gradient matrix.

Kernel gradients: for each kernel element, sum the product of the corresponding patch in the padded input image and the gradient matrix. For each position (i, j) in the gradient matrix, the corresponding 2×2 patch is extracted from the padded input image P, and each kernel element is multiplied by the corresponding value in the patch and summed over all positions:

$$\text{kernelGradient}[a][c] = \sum_{i,j} P[i+a][j+c] \cdot \text{grad}[i][j]$$

Most gradients are zero, so only the positions where the gradient is non-zero need computing.

Input gradients (padded input): compute how the error propagates back to the input by, for each position in the gradient matrix, "spreading" the kernel's weights multiplied by the gradient value at the right spot into a new 6×6 matrix, and summing overlapping positions. Then remove the padding, which leaves a 4×4 matrix matching the original image: the gradient with respect to the input, which is passed further back through the network.

d) Optimization

Finally, update the kernel and bias using the learning rate *LR*:

```
kernel = kernel + LR * kernelGradient
bias   = bias   + LR * biasGradient
```

Give the updated kernel and the updated bias.

The worked solution is on page 165.

EXERCISE SET 2

A Simple Conv Layer in C++

Exercise 6.3 The `ml::ConvLayer` struct

CODE

A struct named `ml::ConvLayer` should be added to `lectures/L06/conv_layer/cpp/conv_demo.cpp` to implement a simple conv layer. To keep things as simple as possible, we implement a struct and skip get/set methods, deletion of copy and move constructors, and so on.

You don't need to finish this now: get as far as you can with `feedforward()` and `backpropagate()`; you'll finish the struct (and compile and run it for the first time) in Chapter 7. See Exercise 7.1 for the wrap-up steps once you're done.

`ConvLayer` zero-pads its input (`pad = kernelSize / 2`) so the output is the **same size as the input**, not smaller. This is why `outputGradients` in the example below is a 4×4 matrix, matching the 4×4 `input`: your `feedforward()` and `backpropagate()` need to pad and unpad internally to make that size match up. The struct skeleton in `conv_demo.cpp` sketches two private helper methods for exactly that, along with every member variable you need.

The general output size is $\lfloor (N + 2p - K)/s \rfloor + 1$, which lands exactly on N for an odd kernel. The 2×2 kernel used here is even, so the padded image admits 5×5 window positions; loop `i` and `j` from 0 to `inputSize - 1` and the extra row and column simply go unused. Exercise 6.1 works the same point through by hand.

The demo runs the *same* numbers you worked through by hand in Exercises 6.1 and 6.2: the same input image, the same kernel and bias, and the same gradients coming back from the max pooling layer. It prints each result next to the value the hand calculation arrives at, so you can check your implementation against your own hand calculations rather than guessing whether the output looks reasonable.

That's also why the kernel and bias are assigned right after the layer is created: the constructor randomizes them, and the demo overwrites them so every run prints the same numbers.

Study the code in the `main()` function. Your implementation should be written so this code works:

```
// Create a convolutional layer: 4x4 input, 2x2 kernel.
constexpr std::size_t inputSize{4U};
constexpr std::size_t kernelSize{2U};
ml::ConvLayer convLayer{inputSize, kernelSize};

// Use appendix B's kernel and bias instead of the randomized ones.
convLayer.kernel = Matrix2d{{0.2, 0.4}, {0.6, 0.8}};
convLayer.bias   = 0.5;

// Input image from appendix B, resembling the digit 0 made up of ones.
const Matrix2d input{{1, 1, 1, 1},
                     {1, 0, 0, 1},
                     {1, 0, 0, 1},
                     {1, 1, 1, 1}};

// Perform feedforward (convolution). Expect the conv output from appendix B.
convLayer.feedforward(input);

// Gradients the max pooling layer sends back in appendix B.
const Matrix2d outputGradients{{0, 10, 20, 0},
                               {0,  0,  0, 0},
                               {0,  0,  0, 0},
                               {0, 30,  0, 40}};

// Perform backpropagation. Expect the input gradients from appendix B.
convLayer.backpropagate(outputGradients);

// Perform optimization. Expect the updated kernel and bias from appendix B.
convLayer.optimize(0.001);
```

The comments name appendix B of the lecture, which is Set 1 here. Each of `feedforward()`, `backpropagate()` and `optimize()` returns a `bool`; the demo checks it and aborts with a message on failure.

CHAPTER 7

Convolution, Pooling and Flatten Layers

Three layer types, each written once as a simple struct.

IN THIS CHAPTER

- Finishing the conv layer struct, `ml::ConvLayer`, started in Chapter 6.
- A max pooling layer struct, `ml::MaxPoolLayer`.
- A flatten layer struct, `ml::FlattenLayer`.
- Weight sharing: the same kernel across the entire image.
- Reference implementations in Python and C, and encapsulation via opaque structs.

This chapter turns the three layer types that extract features from an image into code. After it, you should be able to implement a simple conv, max pooling, and flatten layer in software. Each is a struct in a demo program of its own, kept as simple as possible: public member variables, no getters or setters, and no deleted copy and move operations, so that nothing stands between you and the arithmetic of §6.2 and Exercises 6.1 and 6.2. The next three chapters rebuild the same layers as classes behind an interface, in a real network.

The exercises in §7.2 are the whole of the chapter. The first set finishes the `ml::ConvLayer` you started in Exercise 6.3 and runs it for the first time; the second and third add `ml::MaxPoolLayer` and `ml::FlattenLayer`. The companion repository holds a reference implementation of each layer in Python, which you can test-run with the command `python3 <name>_demo.py` in its directory, and a more developed one in C, which demonstrates encapsulation via an opaque struct whose accessor functions are only exposed through the corresponding header file.

7.1 Review

What you should be able to do now

After this chapter, you should:

- Be able to implement a simple conv, max pooling, and flatten layer in software.

Questions to test yourself

You should be able to explain:

1. Which parts of your `ml::ConvLayer` implementation correspond to feedforward and backpropagation, respectively?
2. Why do we use the same kernel (the same weights) across the entire image instead of unique weights per position?
3. How does your `ml::MaxPoolLayer` implementation know which positions the gradients should be propagated back to during backpropagation?
4. Why is a flatten layer needed between the convolutional and pooling layers and the dense layer?

Looking ahead

Chapter 8 implements the conv layer in the real, polymorphic CNN pipeline, `cnn_work`, replacing the stub it starts out with.

7.2 Exercises

How to work these. Each set is one demo program that is already in the companion repository, with a skeleton of the struct you write, its helper functions, the `main()` that uses it, and a Makefile: `lectures/L06/conv_layer/cpp/conv_demo.cpp` for Set 1, which you started in Exercise 6.3, `lectures/L07/max_pool_layer/cpp/max_pool_demo.cpp` for Set 2, and `lectures/L07/flatten_layer/cpp/flatten_demo.cpp` for Set 3. Until you switch the body of `main()` on, as each set's last exercise describes, that body is compiled out, and with it every use of the struct, so the program builds while the struct is still incomplete.

How to check your work. These three have no test suite. The conv layer's demo prints each result next to the value Exercises 6.1 and 6.2 arrive at by hand. For the other two, run the Python reference beside your program: it feeds the same input through the same layer and prints the same matrices, so compare the numbers rather than the formatting.

The reference implementations are published in the repository beside each C++ demo, in Python in `python` and in C in `c`; the C versions keep each layer behind an opaque struct whose accessor functions are only exposed through its header. Try each exercise yourself before looking at them.

EXERCISE SET 1

Finishing the Conv Layer

You started implementing `ml::ConvLayer` in Exercise 6.3: finish it here. The full task description and the target `main()` usage are both in Exercise 6.3; this set only covers the wrap-up steps for compiling and running it for the first time, in `lectures/L06/conv_layer/cpp/conv_demo.cpp`.

Exercise 7.1 Compiling and running the program

CODE

As usual, the program can be run by typing the command `make` in the terminal:

```
make
```

Once the implementation is complete, uncomment the compiler flag `CXX_FLAGS` in the Makefile, `lectures/L06/conv_layer/cpp/Makefile`. That is, change the following:

```
# TODO: Uncomment this line once the implementation is finished.
#CXX_FLAGS := -std=c++17 -Wall -Werror -DCONV_LAYER_IMPLEMENTED
```

to

```
CXX_FLAGS := -std=c++17 -Wall -Werror -DCONV_LAYER_IMPLEMENTED
```

You can then also remove the conditional-compilation guard `CONV_LAYER_IMPLEMENTED` from `conv_demo.cpp` if you like. In that case, change the following:

```
/**
 * @brief Create and demonstrate a simple convolutional layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) ↪
    ↪ once the
//        implementation is finished.
#ifdef CONV_LAYER_IMPLEMENTED

    // Function content.
    return 0;

//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) ↪
    ↪ once the
```

```
//        implementation is finished.
#endif /** CONV_LAYER_IMPLEMENTED */
}
```

to

```
/**
 * @brief Create and demonstrate a simple convolutional layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    // Function content.
    return 0;
}
```

EXERCISE SET 2

A Simple Max Pooling Layer

Exercise 7.2 The `ml::MaxPoolLayer` struct

CODE

A struct named `ml::MaxPoolLayer` should be added to `lectures/L07/max_pool_layer/cpp/max_pool_demo.cpp` to implement a simple max pooling layer. To keep things as simple as possible, we implement a struct and skip get/set methods, deletion of copy and move constructors, and so on.

Study the code in the `main()` function. Your implementation should be written so this code works to create and use a max pooling layer named `poolLayer`:

```
// Create a max pooling layer: 4x4 input, 2x2 pooling regions, produces 2x2 output.
constexpr std::size_t inputSize{4U};
constexpr std::size_t poolSize{2U};
ml::MaxPoolLayer poolLayer{inputSize, poolSize};

// Example 4x4 input matrix (could represent an image or feature map).
const Matrix2d input{{2, 1, 6, 1},
                     {3, 0, 4, 6},
                     {1, 2, 4, 5},
                     {3, 4, 7, 7}};

// Perform feedforward (pooling).
poolLayer.feedforward(input);
```

```
// Example output gradients (same shape as pooling output, used for backpropagation demo).
const Matrix2d outputGradients{{1, 2},
                               {3, 4}};
// Perform backpropagation.
poolLayer.backpropagate(outputGradients);
```

Exercise 7.3 Compiling and running the program

CODE

As usual, the program can be run by typing the command `make` in the terminal:

```
make
```

Once the implementation is complete, uncomment the compiler flag `CXX_FLAGS` in the Makefile, `lectures/L07/max_pool_layer/cpp/Makefile`. That is, change the following:

```
# TODO: Uncomment this line once the implementation is finished.
#CXX_FLAGS := -std=c++17 -Wall -Werror -DMAX_POOL_LAYER_IMPLEMENTED
```

to

```
CXX_FLAGS := -std=c++17 -Wall -Werror -DMAX_POOL_LAYER_IMPLEMENTED
```

You can then also remove the conditional-compilation guard `MAX_POOL_LAYER_IMPLEMENTED` from `max_pool_demo.cpp` if you like. In that case, change the following:

```
/**
 * @brief Create and demonstrate a simple max pooling layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) once the
//        implementation is finished.
#ifdef MAX_POOL_LAYER_IMPLEMENTED

    // Function content.
    return 0;
```

```
//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) ↪
    ↪ once the
//        implementation is finished.
#endif /** MAX_POOL_LAYER_IMPLEMENTED */
}
```

to

```
/**
 * @brief Create and demonstrate a simple max pooling layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    // Function content.
    return 0;
}
```

EXERCISE SET 3

A Simple Flatten Layer

Exercise 7.4 The `ml::FlattenLayer` struct

CODE

A struct named `ml::FlattenLayer` should be added to `lectures/L07/flatten_layer/cpp/flatten_demo.cpp` to implement a simple flatten layer. To keep things as simple as possible, we implement a struct and skip get/set methods, deletion of copy and move constructors, and so on.

Study the code in the `main()` function. Your implementation should be written so this code works to create and use a flatten layer named `flattenLayer`:

```
// Create a flatten layer: 4x4 input, produces 1x16 output.
constexpr std::size_t inputSize{4U};
ml::FlattenLayer flattenLayer{inputSize};

// Example 4x4 input matrix (could represent an image or feature map).
const Matrix2d input{{2, 1, 6, 1},
                     {3, 0, 4, 6},
                     {1, 2, 4, 5},
                     {3, 4, 7, 7}};

// Perform feedforward (flatten the input).
```

```
flattenLayer.feedforward(input);

// Example output gradients (same shape as flattened output, used for ↪
    ↪ backpropagation demo).
const Matrix1d outputGradients{1, 2, 3, 4, 8, 7, 6, 5, 0, 2, 4, 8, 9, 7, 5, 3};

// Perform backpropagation (unflatten the output gradients).
flattenLayer.backpropagate(outputGradients);
```

Exercise 7.5 Compiling and running the program CODE

As usual, the program can be run by typing the command `make` in the terminal:

```
make
```

Once the implementation is complete, uncomment the compiler flag `CXX_FLAGS` in the Makefile, `lectures/L07/flatten_layer/cpp/Makefile`. That is, change the following:

```
# TODO: Uncomment this line once the implementation is finished.
#CXX_FLAGS := -std=c++17 -Wall -Werror -DFLATTEN_LAYER_IMPLEMENTED
```

to

```
CXX_FLAGS := -std=c++17 -Wall -Werror -DFLATTEN_LAYER_IMPLEMENTED
```

You can then also remove the conditional-compilation guard `FLATTEN_LAYER_IMPLEMENTED` from `flatten_demo.cpp` if you like. In that case, change the following:

```
/**
 * @brief Create and demonstrate a simple flatten layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) ↪
    ↪ once the
//        implementation is finished.
#ifdef FLATTEN_LAYER_IMPLEMENTED
    // Function content.
    return 0;
```

```
//! @todo Remove this guard (and/or uncomment the compiler flags in the makefile) ↩
    ↪ once the
//        implementation is finished.
#endif /** FLATTEN_LAYER_IMPLEMENTED */
}
```

to

```
/**
 * @brief Create and demonstrate a simple flatten layer.
 *
 * @return 0 on success, -1 on failure.
 */
int main()
{
    // Function content.
    return 0;
}
```

CHAPTER 8

The Convolutional Layer

The conv layer, rebuilt behind an interface in a real pipeline.

IN THIS CHAPTER

- The polymorphic `cnn_work` pipeline, and how its layers connect.
- A concrete class `ml::conv_layer::Conv` implementing `conv_layer::Interface`.
- Its `feedforward()`, `backpropagate()` and `optimize()`.
- Replacing `ConvStub` in `factory.cpp`.
- The split between an interface and the concrete class behind it.

This chapter implements a real, working convolutional layer, `ml::conv_layer::Conv`, in the polymorphic CNN pipeline `cnn_work`, replacing the stub the pipeline currently uses for it. The math is the one you worked through by hand in §6.4 and wrote as the struct `ml::ConvLayer` in Chapters 6 and 7; what is new is the design around it. After this chapter you should have created a concrete subclass `Conv` that inherits `ml::conv_layer::Interface`, and implemented its `feedforward()`, `backpropagate()` and `optimize()`.

§8.1 introduces `cnn_work`, which is wired together from end to end but still runs on stub layers. Build and run it with `make` before you change anything, to see the meaningless output the stubs produce, then start replacing them with real layers in the exercises in §8.3. It is your own copy from here on, and you keep building it up through Chapters 9 and 10. The max pooling and flatten layers are still stubs at the end of this chapter, so the program won't produce meaningful predictions until every layer is real, at the end of Chapter 10. In the meantime, `dense.hpp` and `dense.cpp` are left in the project already fully implemented, as a working example of what a finished layer class looks like.

8.1 The CNN Pipeline

`cnn_work`, in `lectures/L08/cnn_work`, is the working copy of a C++17 convolutional neural network (CNN), built up one layer at a time across Chapters 8, 9 and 10. It's trained to classify four hand-drawn 4×4 pixel patterns: the digits 0 to 3.

The four training patterns, from `source/main.cpp`, where `1` is a filled pixel and `0` is empty, are shown below:

```
Digit 0:      Digit 1:      Digit 2:      Digit 3:
1 1 1 1       0 1 0 0       1 0 0 1       0 0 1 0
1 0 0 1       0 1 0 0       0 1 1 0       0 0 1 0
1 0 0 1       0 1 0 0       0 1 1 0       1 1 1 1
1 1 1 1       0 1 0 0       1 0 0 1       0 0 1 0
```

8.1.1 Current status

Not every layer is real yet: `source/ml/factory/factory.cpp` decides, per layer type, whether to build the real class or a `Stub`, and each one not yet built is marked with a `//! @todo` comment there:

- **`Conv`** (`ml/conv_layer/conv.hpp` and `conv.cpp`): placeholder, see the exercises in §8.3.
- **`MaxPool`** (`ml/conv_layer/max_pool.hpp` and `max_pool.cpp`): placeholder, see the exercises in §9.2.
- **`Flatten`** (`ml/flatten_layer/flatten.hpp` and `flatten.cpp`): placeholder, see the exercises in §10.4.
- **`Dense`**: already real (built separately back in Chapter 5).

The project builds and runs as-is at every stage: with any layer still stubbed, `make` builds, trains and prints predictions without error; they just won't be meaningful until every layer is real, at the end of Chapter 10. That's expected, not a bug.

8.1.2 Structure

```
cnn_work/
├── Makefile
├── source/
│   ├── main.cpp
│   └── ml/
│       ├── cnn/cnn.cpp
│       ├── conv_layer/{conv,max_pool}.cpp
│       ├── dense_layer/dense.cpp
│       ├── flatten_layer/flatten.cpp
│       ├── factory/factory.cpp
│       ├── random/generator.cpp
│       └── utils.cpp
└── include/ml/
    ├── types.hpp, utils.hpp
    ├── act_func/{interface,relu,tanh,none,type}.hpp
    ├── cnn/{interface,cnn}.hpp
```

```
    ├── conv_layer/{interface,conv,max_pool,stub}.hpp
    ├── dense_layer/{interface,dense,stub}.hpp
    ├── flatten_layer/{interface,flatten,stub}.hpp
    ├── factory/{interface,factory,stub}.hpp
    └── random/{interface,generator}.hpp
```

Each layer type (`conv_layer`, `dense_layer`, `flatten_layer`) follows the same pattern: an `Interface` that defines the contract, a concrete class that does the real computation (`Conv`, `MaxPool`, `Dense`, `Flatten`), and a `Stub` variant that only validates dimensions and returns `true` or `false` without computing anything. `ml::factory::Factory` picks, layer by layer, which one to build; see `factory.cpp`.

Data types (`ml/types.hpp`): `Matrix1d = std::vector<double>`, `Matrix2d = std::vector<Matrix1d>`, `Matrix3d = std::vector<Matrix2d>`.

8.1.3 `dense_layer::Dense`

The one layer that's already real. A regular fully connected layer with weights and a bias.

- **`feedforward()`:** `sum = bias[i] + Σ weight[i][j] * input[j]` per node, followed by the activation function.
- **`backpropagate()`:** computes

  ```
  error[i] = (outputGradients[i] - myOutput[i])
           * activationFunction.delta(preActivationSum[i])
  ```

  See the note in §8.1.8 on how `outputGradients` is interpreted differently depending on whether the layer is the output layer or a hidden layer.
- **`optimize()`:** updates the weights and bias with `error * learningRate * input`.

8.1.4 `ml::cnn::Cnn`: how the layers connect

The constructor creates and connects the layers via the injected factory:

1. A `Conv` layer and a `MaxPool` layer are added to the same list, `myConvLayers` (in that order).
2. A `Flatten` layer is created based on the max pooling layer's output size.
3. A `Dense` layer is created based on the flatten layer's output size.

`feedforward(input)`: each layer is fed the previous layer's `output()`:

```
input → Conv → MaxPool → Flatten → Dense → prediction
```

`backpropagate(target)`: goes the same path in reverse. The target vector (`target`) is passed directly into the last dense layer's `backpropagate()`: that's where `(outputGradients[i] - myOutput[i])` actually becomes `(target[i] - prediction[i])`.

Each preceding layer is then fed the next layer's `inputGradients()`: the dense layers in reverse order, then the flatten layer, then `MaxPool`, and finally `Conv`.

`optimize(learningRate)`: updates every layer with the gradients already computed by `backpropagate()`. The dense layers also need the previous layer's `output()` (the value actually used during feedforward) as input for the weight update.

`train(trainIn, trainOut, epochCount, learningRate)`: pure stochastic gradient descent (batch size 1, no mini-batching). The training order is reshuffled every epoch. If `feedforward()`, `backpropagate()`, or `optimize()` fails for any example, training is aborted immediately.

`predict(input)`: runs `feedforward()` and returns the last dense layer's `output()`.

8.1.5 Activation functions

Function	`output(x)`	`delta(x)`
`Relu`	`x` if `x > 0`, otherwise `0`	`1` if `x > 0`, otherwise `0`
`Tanh`	`tanh(x)`, range $(-1, 1)$	`1 - tanh(x)²`
`None`	`x` (identity)	`1`

`delta()` is always computed from the pre-activation sum (the value before the activation function was applied), not from the already-activated output value.

8.1.6 Random number generator

`random::Generator` is a singleton that initializes `std::srand` once, on first use. It's used both to randomize initial weight and bias values (via `randomStartVal()` in `Conv` and `Dense`), and to reshuffle the training order every epoch.

8.1.7 Building and running

```
make
```

This builds and runs the program directly (the `build` target followed by the `run` target). `clean` removes the built binary.

8.1.8 Implementation details worth knowing

`Dense::backpropagate()` reuses the same formula for the output layer and hidden layers. For the last dense layer, `outputGradients` is literally the target vector, so `outputGradients[i] - myOutput[i]` becomes `target - prediction`, which is the deviation the book defines. For earlier dense layers, what's passed in instead is the next layer's already-computed `inputGradients()`, which is a gradient, not a target. Subtracting `myOutput[i]` from it adds a term that isn't part of the gradient of any loss, so for a hidden layer this is a deliberate

simplification rather than the exact derivative. The networks in this book still train, because the term is small next to the one that carries the signal, and keeping one formula keeps the layer readable. Worth knowing before you compare this code against a textbook, or against a framework, and find they don't agree.

8.2 Review

What you should be able to do now

After this chapter, you should:

- Have created a concrete subclass `Conv` that inherits `ml::conv_layer::Interface`.
- Have implemented `feedforward()`, `backpropagate()`, and `optimize()` for `Conv`.

Questions to test yourself

You should be able to explain:

1. What's the difference between `ml::conv_layer::Interface` and your concrete class `Conv`, and why is this split beneficial?
2. Can you connect the mathematical formulas from the hand-training exercise in §6.4 to each computation step in your `feedforward()` and `backpropagate()`?
3. How does your `ml::conv_layer::Conv` compare to the `ml::ConvLayer` struct you wrote in Chapters 6 and 7: same math, but what's different about the design?
4. Why do `Conv` and `MaxPool` (Chapter 9) share a single interface, `conv_layer::Interface`?

Looking ahead

Chapter 9 implements the max pooling layer in the same CNN pipeline.

8.3 Exercises

> **How to work these.** The three sets are one class, written in `cnn_work` (`lectures/L08/cnn_work`, see §8.1), which is where you write your code from here on. There's nothing to copy: `conv.hpp` and `conv.cpp` are already in the project as placeholders, and already built by its Makefile.
> **How to check your work.** The test suite in `lectures/L08/exercises/test` already points at `cnn_work`; Exercise 8.10 runs it, and `make ML_DIR=<path>` points it at a tree of your own elsewhere. Run `cnn_work` itself as well, once Exercise 8.9 has wired the class in.
> The finished reference implementation of this chapter's class, and of the next two chapters' too, is published in the companion repository as `lectures/L10/cnn_demo`; see the warning below.

Spoiler warning. `lectures/L10/cnn_demo` is the finished reference copy of this project, and it contains working `Conv`, `MaxPool` and `Flatten` classes; i.e. the answers to this chapter's exercises and the next two chapters'. It's there for Chapter 10, where you compare your own pipeline against it. Write your own version first; reading it now costs you the exercise.

EXERCISE SET 1

Declaring the Conv Layer

You'll extend the `cnn_work` codebase, implementing a concrete class `Conv`, replacing the temporary stub class `ml::conv_layer::ConvStub` currently used for the conv layer in `factory.cpp`. This is the same math you already worked through by hand in §6.4 and implemented as the standalone struct `ml::ConvLayer` in Chapters 6 and 7: here you're adapting that same logic to satisfy `cnn_work`'s polymorphic `conv_layer::Interface` instead.

Exercise 8.1 The `Conv` class, declaration

CODE

In the header file `include/ml/conv_layer/conv.hpp` (currently a placeholder), add a class named `Conv` that inherits `ml::conv_layer::Interface`, from `include/ml/conv_layer/interface.hpp`:

- Use public inheritance and mark the class `final` so it can't be inherited further.
- Override every method from the interface, including the destructor.
- Keep `[[nodiscard]]` on every override whose interface method has it. Attributes aren't inherited, so the override needs its own (see Chapter 3).

The interface is typeset below, straight from the repository:

lectures/L08/cnn_work/include/ml/conv_layer/interface.hpp

```
/**
 * @file Convolutional layer interface.
 */
#pragma once

#include <cstddef>
#include <memory>

#include "ml/types.hpp"

namespace ml::conv_layer
{
/**
 * @brief Convolutional layer interface.
 */
```

```
class Interface
{
public:
    /**
     * @brief Destructor.
     */
    virtual ~Interface() noexcept = default;

    /**
     * @brief Get the input size of the layer.
     *
     * @return The input size of the layer.
     */
    [[nodiscard]] virtual std::size_t inputSize() const noexcept = 0;

    /**
     * @brief Get the output size of the layer.
     *
     * @return The output size of the layer.
     */
    [[nodiscard]] virtual std::size_t outputSize() const noexcept = 0;

    /**
     * @brief Get the output of the layer.
     *
     * @return Matrix holding the output of the layer.
     */
    [[nodiscard]] virtual const Matrix2d& output() const noexcept = 0;

    /**
     * @brief Get the input gradients of the layer.
     *
     * @return Matrix holding the input gradients of the layer.
     */
    [[nodiscard]] virtual const Matrix2d& inputGradients() const noexcept = 0;

    /**
     * @brief Perform feedforward operation.
     *
     * @param[in] input Matrix holding input data.
     *
     * @return True on success, false on failure.
     */
    virtual bool feedforward(const Matrix2d& input) noexcept = 0;
```

```
    /**
     * @brief Perform backpropagation.
     *
     * @param[in] outputGradients Matrix holding gradients from the next layer.
     *
     * @return True on success, false on failure.
     */
    virtual bool backpropagate(const Matrix2d& outputGradients) noexcept = 0;

    /**
     * @brief Perform optimization.
     *
     * @param[in] learningRate Learning rate to use.
     *
     * @return True on success, false on failure.
     */
    virtual bool optimize(double learningRate) noexcept = 0;
};
} // namespace ml::conv_layer
```

Tip: Copy the entire contents of the interface and paste it into the new file, then adapt it (no `virtual` or `= 0`, add `override` everywhere; including on `inputGradients()`, which is easy to miss).

Exercise 8.2 Removed constructors and operators CODE

Delete the default constructor, copy constructor, move constructor, copy assignment operator, and move assignment operator.

Exercise 8.3 Private member variables CODE

Add the following private member variables to `Conv`:

- **`myInputPadded`**: the zero-padded input (`Matrix2d`).
- **`myInputGradientsPadded`**: input gradients, padded (`Matrix2d`).
- **`myInputGradients`**: input gradients, unpadded; what `inputGradients()` returns (`Matrix2d`).
- **`myKernel`**: the kernel weights (`Matrix2d`).
- **`myKernelGradients`**: kernel gradients (`Matrix2d`).
- **`myOutput`**: the layer's output (feature map); what `output()` returns (`Matrix2d`).
- **`myPreActOutput`**: the sum before the activation function is applied, needed later by `backpropagate()` to compute the activation's derivative (`Matrix2d`).
- **`myBias`**: the bias value (`double`).

- **`myBiasGradient`**: the bias gradient (`double`).
- **`myActFunc`**: pointer to the activation function (`ActFuncPtr`, see `ml/act_func/interface.hpp`).

Feel free to add private helper methods as needed: the reference implementation uses one to validate the constructor's parameters, one to (re-)build the zero-padded input, and one to strip the padding back off the input gradients, but you're free to structure this however you like.

Exercise 8.4 Constructor

CODE

Implement the constructor:

```
explicit Conv(std::size_t inputSize, std::size_t kernelSize,
              act_func::Type actFunc = act_func::Type::None) noexcept;
```

- Print an error message and call `std::terminate()` if `kernelSize` is outside `[1, 11]`, or if `inputSize < kernelSize`.
- Compute the padding offset (`kernelSize / 2`) and the padded size (`inputSize + 2 * offset`).
- Size `myInputPadded` and `myInputGradientsPadded` to the padded size, `myInputGradients`, `myOutput` and `myPreActOutput` to `inputSize`, and `myKernel` and `myKernelGradients` to `kernelSize` (see `ml::initMatrix()` in `ml/utils.hpp`).
- Randomize the bias and every kernel weight using `ml::randomStartVal()`.
- Get the activation function: `factory::Factory factory{}; myActFunc = factory.actFunc(actFunc);` (see `ml/factory/factory.hpp`; this is the same pattern the already finished `Dense` layer uses in `dense.cpp`, if you want a working example to compare against).

Exercise 8.5 `feedforward()`

CODE

Implement `bool feedforward(const Matrix2d& input) noexcept`:

- Return `false` if the size of `input` doesn't match the size of `myOutput` (`ml::matchDimensions()`), or if padding the input fails (see below).
- Zero-pad `input` into `myInputPadded`: for every position `(i, j)` in `input`, write it to `myInputPadded[i + offset][j + offset]`, where `offset = myKernel.size() / 2`. Return `false` if `input` isn't square (`ml::isMatrixSquare()`).
- For each output position `(i, j)`: sum `myBias` plus the element-wise product of `myKernel` and the corresponding `kernelSize × kernelSize` window of `myInputPadded` starting at `(i, j)`. Store the sum in `myPreActOutput[i][j]`, then store `myActFunc->output(sum)` in `myOutput[i][j]`.
- Return `true`.

Exercise 8.6 `backpropagate()` CODE

Implement `bool backpropagate(const Matrix2d& outputGradients) noexcept`:

- Return `false` if the size of `outputGradients` doesn't match the size of `myOutput`, or if it isn't square.
- Reset `myInputGradientsPadded`, `myInputGradients` and `myKernelGradients` to zero, and `myBiasGradient` to `0.0`.
- For each output position `(i, j)`, compute

  ```
  delta   = myActFunc->delta(myPreActOutput[i][j])
  outGrad = outputGradients[i][j] * delta
  ```

  and add `outGrad` to `myBiasGradient`. Then, for each kernel position `(ki, kj)`: add `myInputPadded[i+ki][j+kj] * outGrad` to `myKernelGradients[ki][kj]`, and add `myKernel[ki][kj] * outGrad` to `myInputGradientsPadded[i+ki][j+kj]`.
- Strip the padding border back off `myInputGradientsPadded` into `myInputGradients` (the exact inverse of the padding step in `feedforward()`).
- Return `true`.

Exercise 8.7 `optimize()` CODE

Implement `bool optimize(double learningRate) noexcept`:

- Return `false` if the learning rate is invalid (`ml::checkLearningRate()`).
- `myBias += myBiasGradient * learningRate`.
- For every kernel position: `myKernel[i][j] += myKernelGradients[i][j] * learningRate`.
- Return `true`.

Exercise 8.8 Trivial accessors CODE

Implement the four accessors from the interface:

- **`inputSize()`**: returns `myInputGradients.size()`.
- **`outputSize()`**: returns `myOutput.size()`.
- **`output()`**: returns `myOutput`.
- **`inputGradients()`**: returns `myInputGradients`.

Add two more accessors, which are **not** part of `conv_layer::Interface`. The tests call them on a `const Conv`, so both must be `const`:

- **`kernel()`**: `[[nodiscard]] const Matrix2d& kernel() const noexcept;`, returns `myKernel`.
- **`bias()`**: `[[nodiscard]] double bias() const noexcept;`, returns `myBias`.

These two exist for the test suite. Because the constructor randomizes the bias and the kernel, nothing the layer computes can be predicted from the constructor arguments alone; the tests read these back and redo the arithmetic by hand to check the layer got the same answer. They're declared on `Conv` rather than on the interface because the max pooling layer implements that same interface in Chapter 9 and has neither a kernel nor a bias to report.

EXERCISE SET 2

Wiring It In

Exercise 8.9 Wiring it in

CODE

1. Make sure `source/ml/conv_layer/conv.cpp` is listed in `cnn_work`'s `Makefile` under `SRC_FILES`.
2. In `source/ml/factory/factory.cpp`, find `Factory::convLayer()`: it currently returns a `conv_layer::ConvStub` (marked with a `//! @todo` comment). Replace that with your new class:

```
return std::make_unique<conv_layer::Conv>(inputSize, kernelSize, actFunc);
```

3. Run `make`. The conv layer in `cnn_work` is now real; though the rest of the pipeline (max pooling, flatten) is still stubbed, so the output of `predict()` won't be meaningful yet. That happens once every layer is real, at the end of Chapter 10.

EXERCISE SET 3

Running the Tests

Exercise 8.10 Running the tests

CODE

A test suite for `Conv` is available in `lectures/L08/exercises/test`. It's the first suite of Part II and covers this chapter's layer only; Chapters 9 and 10 add their own.

Build and run it from this lecture's `appendix` directory:

```
make -C exercises/test
```

You don't need to copy anything: unlike the suites of Chapters 1 to 5, the suite's `ML_DIR` already points at `cnn_work`, where you're writing the code. All 23 test cases should pass.

Until `Conv` exists the build fails with `'Conv' in namespace 'ml::conv_layer' does not name a type`; the suite is the specification, so it doesn't compile until the class does. Note also that the tests construct `Conv` directly, so they pass whether or not you've done the `factory.cpp` wiring step in Exercise 8.9; run `cnn_work` itself to check that.

The test suite's README, `lectures/L08/exercises/test/README.md`, has more information, including why the layer needs `kernel()` and `bias()` at all.

CHAPTER 9

The Max Pooling Layer

A layer with nothing to learn, and gradients to route.

IN THIS CHAPTER

- A concrete class `ml::conv_layer::MaxPool`, on the same `conv_layer::Interface` as `Conv`.
- Its `feedforward()`, `backpropagate()` and `optimize()`.
- Gradient routing in max pooling.
- Layers with and without trainable parameters.
- Replacing `MaxPoolStub` in `factory.cpp`.

This chapter implements a real, working max pooling layer, `ml::conv_layer::MaxPool`, in the polymorphic `cnn_work` pipeline from Chapter 8, replacing the `MaxPoolStub` it currently uses. Keep working in the same project, `lectures/L08/cnn_work`. After this chapter you should have created a concrete subclass `MaxPool` that inherits `ml::conv_layer::Interface`, and implemented its `feedforward()`, `backpropagate()` and `optimize()`.

The logic is the one you worked through by hand in §6.4 and wrote as the struct `ml::MaxPoolLayer` in Chapter 7. What makes the class worth a chapter is what it doesn't have: no trainable parameters and no activation function, and still the same interface as `Conv`, printed in Exercise 8.1. The exercises in §9.2 are where you write it.

9.1 Review

What you should be able to do now

After this chapter, you should:

- Have created a concrete subclass `MaxPool` that inherits `ml::conv_layer::Interface`.
- Have implemented `feedforward()`, `backpropagate()`, and `optimize()` for `MaxPool`.

Questions to test yourself

You should be able to explain:

1. How does your `ml::conv_layer::MaxPool` implementation know which positions the gradients should be propagated back to during backpropagation?
2. What differences exist between a max pooling layer and a conv or dense layer, in terms of trainable parameters?
3. Why can `Conv` and `MaxPool` share a single interface (`conv_layer::Interface`) despite one having trainable parameters and the other not?

Looking ahead

Chapter 10 implements the flatten layer, and wires the complete CNN together as the book's capstone.

9.2 Exercises

How to work these. The three sets are one class, written in the `cnn_work` codebase you started in Chapter 8, `lectures/L08/cnn_work`. Keep working in the same project: `max_pool.hpp` and `max_pool.cpp` are already in it as placeholders.
How to check your work. The test suite in `lectures/L09/exercises/test` points back at `cnn_work`; Exercise 9.10 runs it, and `make ML_DIR=<path>` points it at a tree of your own elsewhere. The finished reference implementation is published in the companion repository as `lectures/L10/cnn_demo`; see the warning below.

Spoiler warning. `lectures/L10/cnn_demo` is the finished reference copy of this project, and it contains working `Conv`, `MaxPool` and `Flatten` classes; i.e. the answers to this chapter's exercises, the previous chapter's and the next. It's there for Chapter 10, where you compare your own pipeline against it. Write your own version first; reading it now costs you the exercise.

EXERCISE SET 1

Declaring the Max Pooling Layer

You'll continue extending `cnn_work`, implementing a concrete class `MaxPool`, replacing the temporary stub class `ml::conv_layer::MaxPoolStub` currently used for the max pooling layer in `factory.cpp`. This is the same logic you already worked through by hand in §6.4 and implemented as the standalone struct `ml::MaxPoolLayer` in Chapter 7: here you're adapting that same logic to satisfy `cnn_work`'s polymorphic `conv_layer::Interface` instead. `MaxPool` uses the *same* interface as `Conv` from Chapter 8; a max pooling layer just has no trainable parameters and no activation function.

Exercise 9.1 The `MaxPool` class, declaration

CODE

In the header file `include/ml/conv_layer/max_pool.hpp` (currently a placeholder), add a class named `MaxPool` that inherits `ml::conv_layer::Interface`, the same interface `Conv` implements, printed in Exercise 8.1:

- Use public inheritance and mark the class `final`.
- Override every method from the interface, including the destructor (which can be `= default`).
- Keep `[[nodiscard]]` on every override whose interface method has it. Attributes aren't inherited, so the override needs its own (see Chapter 3).

Exercise 9.2 Removed constructors and operators

CODE

Delete the default constructor, copy constructor, move constructor, copy assignment operator, and move assignment operator.

Exercise 9.3 Private member variables

CODE

Add the following private member variables to `MaxPool`:

- **`myInput`**: the most recently fed-forward input, needed later by `backpropagate()` to relocate which position held the max value in each pool (`Matrix2d`).
- **`myInputGradients`**: input gradients; what `inputGradients()` returns (`Matrix2d`).
- **`myOutput`**: the pooled output; what `output()` returns (`Matrix2d`).

Note this differs from the standalone `ml::MaxPoolLayer` struct you wrote in Chapter 7, which cached the max *index* for each pool. Here we cache the input and re-find the max during backpropagation, which keeps the member list smaller. Both give the same answer, and both break ties toward the first occurrence in row-major order.

Feel free to add a private helper method that computes the pool size from `myInput` and `myOutput` (`myInput.size() / myOutput.size()`) rather than storing it separately: that's what the reference implementation does, since it's always derivable from the two matrices you already have.

Exercise 9.4 Constructor

CODE

Implement the constructor:

```
explicit MaxPool(std::size_t inputSize, std::size_t poolSize) noexcept;
```

- Print an error message and call `std::terminate()` if `inputSize` is `0`, `poolSize` is `0`, `inputSize < poolSize`, or `inputSize` isn't evenly divisible by `poolSize`.
- Size `myInput` and `myInputGradients` to `inputSize`, and `myOutput` to `inputSize / poolSize` (see `ml::initMatrix()`).

Exercise 9.5 `feedforward()`

CODE

Implement `bool feedforward(const Matrix2d& input) noexcept`:

- Return `false` if the size of `input` doesn't match the size of `myInput` (`ml::matchDimensions()`), or if `input` isn't square (`ml::isMatrixSquare()`).
- For each output position `(i, j)`, find the largest value in the corresponding `poolSize × poolSize` block of `input` (the block starting at `(i * poolSize, j * poolSize)`) and store it in `myOutput[i][j]`. If a block has more than one occurrence of the max value, it doesn't matter which one `feedforward()` treats as "the" max; but `backpropagate()` (below) must be consistent with whichever one it picks, so route to the *first* occurrence (which matches the convention of the hand-training exercise in §6.4).
- Store `input` in `myInput` (needed by `backpropagate()`).
- Return `true`.

Exercise 9.6 `backpropagate()`

CODE

Implement `bool backpropagate(const Matrix2d& outputGradients) noexcept`:

- Return `false` if the size of `outputGradients` doesn't match the size of `myOutput`, or if it isn't square.
- Reset `myInputGradients` to zero.
- For each output position `(i, j)`: re-locate which position in the corresponding block of `myInput` held the max value (first occurrence, same rule as `feedforward()`), and add `outputGradients[i][j]` to `myInputGradients` at that position. Every other position in the block gets no gradient (stays `0`).
- Return `true`.

Exercise 9.7 `optimize()` CODE

Implement `bool optimize(double learningRate) noexcept`. The layer has no trainable parameters, so there's nothing to update: this is a genuine no-op, not a placeholder. Ignore `learningRate` and return `true`.

Exercise 9.8 Trivial accessors CODE

Implement the four accessors from the interface:

- **`inputSize()`**: returns `myInputGradients.size()`.
- **`outputSize()`**: returns `myOutput.size()`.
- **`output()`**: returns `myOutput`.
- **`inputGradients()`**: returns `myInputGradients`.

EXERCISE SET 2

Wiring It In

Exercise 9.9 Wiring it in CODE

1. Make sure `source/ml/conv_layer/max_pool.cpp` is listed in `cnn_work`'s `Makefile` under `SRC_FILES`. It already is, as it has been since Chapter 8: the placeholder compiles to nothing.
2. In `source/ml/factory/factory.cpp`, find `Factory::maxPoolLayer()`: it currently returns a `conv_layer::MaxPoolStub` (marked with a `//! @todo` comment). Replace that with your new class:

   ```
   return std::make_unique<conv_layer::MaxPool>(inputSize, poolSize);
   ```

3. Run `make`. Both `Conv` and `MaxPool` are now real; only the flatten layer is still stubbed, so the output of `predict()` still won't be meaningful. That's the next chapter.

EXERCISE SET 3

Running the Tests

Exercise 9.10 Running the tests

CODE

The test suite is available in `lectures/L09/exercises/test`. It's cumulative: it carries Chapter 8's conv layer tests over unchanged and adds unit tests for `MaxPool`.

Build and run it from this lecture's `appendix` directory:

```
make -C exercises/test
```

There's nothing to copy: `ML_DIR` already points back at the `cnn_work` codebase you started in Chapter 8. All 45 test cases should pass.

Because the pooling layer randomizes nothing, its expected values are written out in full rather than recomputed the way the conv layer's are; and the main worked case is the hand-training example from §6.4, the same pooling and gradient routing you did on paper.

The test worth knowing about before you start is `FeedforwardHandlesNegativeValues`. It feeds an all-negative image, which catches starting the search for the max at `0.0` instead of at the block's first value. Every non-negative input passes either way, so it's an easy bug to ship, and it bites for real once the conv layer in front is using `Tanh`.

The test suite's README, `lectures/L09/exercises/test/README.md`, has more information.

CHAPTER 10

The Flatten Layer and the Complete Network

Every layer real, and what changes off the laptop.

In this chapter

- A real, working flatten layer, `ml::flatten_layer::Flatten`, replacing the `flatten_layer::Stub` in the `cnn_work` pipeline.
- Wiring the complete CNN together: switching `factory.cpp` from stubs to the real classes, and seeing every layer type work together for the first time.
- A review of the whole book: linear regression, neural networks, and CNNs are all the same underlying pattern (feedforward, backpropagation, optimization), applied to increasingly structured layers.
- What changes when a network like this has to run on a resource-constrained target instead of a laptop.

This chapter finishes `cnn_work`. The flatten layer is the last one still stubbed, and a flatten layer has no trainable parameters, so its interface declares no `optimize()` at all: there are only `feedforward()` and `backpropagate()` to write. With it in place, every layer in the pipeline is real and the network runs end to end, `input → Conv → MaxPool → Flatten → Dense → prediction`, telling the four hand-drawn digits apart. Run `make` once the wiring is done, and compare its now meaningful predictions against the placeholder output you first saw in Chapter 8.

The chapter then steps back. §10.1 compares every layer type built in the book, from `Fixed` to `Flatten`, in terms of trainable parameters, computation, and role in the network, and shows the shared feedforward, backpropagate and optimize pattern they all follow. §10.2 closes the book with what happens to that network on a microcontroller: memory footprint, numeric precision, and real-time constraints. The exercises in §10.4 implement the flatten layer, run the final test suite, the first that can test the network as a whole, and then review the whole book on paper.

10.1 Course Review

Every layer built in this book, from Chapter 1's linear regression up through the CNN layers of Chapters 6 to 10, follows the same three-step pattern: **feedforward** (compute a prediction from an input), **backpropagate** (compute how wrong that prediction was, and route the blame backward), and **optimize** (adjust trainable parameters to reduce that error next time). What changes from layer to layer is *what* the parameters are, *how* the feedforward sum is computed, and *whether* there are trainable parameters at all.

10.1.1 Every layer type, side by side

The first table says what each layer type is and where the book builds it; the second what each of its three methods does.

Layer	Chapters	Trainable parameters
`Fixed` (linear regression)	1, 2	weight k, bias m
`Dense`	3 to 5	weights and a bias per node
`Conv`	6 to 8	one shared kernel and a bias
`MaxPool`	6, 7, 9	none
`Flatten`	6, 7, 10	none

Layer	Feedforward	Backpropagate	Optimize
`Fixed`	`y = k*x + m`	`δ = y_ref - y_p`	`k,m += δ * LR * (x or 1)`
`Dense`	`y = σ(b + Σ w*x)`	`Δe = δ * σ'(s)` per node	`w += Δe * LR * input, b += Δe * LR`
`Conv`	same as `Dense`, but local (a small window) and shared (the same kernel at every position)	gradients accumulated across every position the kernel visited, then applied once	`kernel,bias += gradient * LR`
`MaxPool`	passes the largest value in each block forward	routes the incoming gradient only to the position that held the max value; everywhere else gets `0`	no-op: nothing to update
`Flatten`	reshapes 2D to 1D	reshapes 1D to 2D (the exact inverse)	doesn't exist in the interface: there's nothing to optimize

A few things worth noticing across the rows:

- **Only `Conv` and `Dense` have trainable parameters.** `MaxPool` and `Flatten` are purely structural: they reshape or downsample data, but never learn anything themselves.

- **`Conv` is `Dense` with two constraints added**: locality (each output only looks at a small window of the input) and weight sharing (the same kernel is reused everywhere instead of a unique weight per input and output pair). That's the entire conceptual jump from Chapter 5 to Chapters 6 and 7: see §6.1 if you want the full "why" behind those two constraints.
- **`Fixed` is a `Dense` layer with one node, no activation function, and no hidden layer around it.** The same *error, adjustment, update* shape from Chapter 1 reappears, unchanged in spirit, all the way through the CNN layers.

10.1.2 The full pipeline

`ml::cnn::Cnn`, in `lectures/L10/cnn_demo`, the finished reference implementation, wires four of these layer types together into one working classifier. This is the same pipeline you built up yourself as `cnn_work` across Chapters 8 to 10; worth comparing the two if anything in your own version doesn't behave the way you expect:

```
input → Conv → MaxPool → Flatten → Dense → prediction
```

Feedforward runs left to right, each layer consuming the previous layer's `output()`.

Backpropagation runs right to left: the target vector goes into the `backpropagate()` of `Dense` first, and each earlier layer is then fed the next layer's `inputGradients()`: `Dense → Flatten → MaxPool → Conv`.

Optimization updates every layer's parameters (`Conv` and `Dense` only) using the gradients already computed by `backpropagate()`.

`lectures/L10/cnn_demo/README.md` has the full method-by-method breakdown, including the exact formulas used in the C++ implementation; §8.1 is the same breakdown for `cnn_work`.

10.2 Constraints on Resource-Constrained Targets

Everything built in this book has run on a general-purpose computer, where memory is effectively unlimited and every CPU core has a hardware floating-point unit (FPU). Deploying the same kind of network to a microcontroller or other resource-constrained target introduces constraints that never show up on a laptop. This section surveys the main ones, and the last exercises in §10.4 work through two of them by hand: memory footprint and fixed-point arithmetic.

10.2.1 Memory footprint

Every weight and bias your network learned during training has to be stored somewhere on the target device, typically in flash (read-only after deployment) or RAM (if the network

keeps learning on the device). The `std::vector<double>` used throughout this book stores each value as an 8-byte `double`.

Consider the small hidden and output layer pair many of the exercises built (3 nodes with 2 weights each in the hidden layer, 1 node with 3 weights in the output layer):

```
Hidden layer: 3 * 2 weights + 3 biases  =  9 values
Output layer: 1 * 3 weights + 1 bias    =  4 values
Total: 13 doubles * 8 bytes             =  104 bytes
```

That's negligible on any target. But the examples in this book are intentionally tiny. A single hidden layer sized for 28×28 pixel images (784 inputs) with 128 nodes (a very ordinary size for a first hidden layer) tells a different story:

```
Weights: 128 * 784 = 100,352
Biases:  128
Total:   100,480 values

As double (8 bytes each): 803,840 bytes  ≈ 785 KB
As float  (4 bytes each): 401,920 bytes  ≈ 392.5 KB
As int8   (1 byte each):  100,480 bytes  ≈ 98.125 KB
```

That's **one layer**. Many microcontrollers have on the order of tens of KB of RAM; mid-range parts often have a few hundred KB; only higher-end parts reach into the multiple-MB range. The `double` version of this single layer already exceeds what a lot of embedded targets have available for their *entire* program; and a real network has several layers. This is why embedded machine learning almost always trains at full precision on a workstation and then **quantizes** the trained weights down to a smaller type before deployment (see §10.2.3), rather than training or storing at `double` precision on the target itself.

10.2.2 Precision: floating-point vs. fixed-point

Many microcontrollers (especially smaller Cortex-M0 and M0+ parts, or 8- and 16-bit MCUs) have no hardware FPU. Floating-point operations on those targets are emulated in software, which can be an order of magnitude or more slower than an integer add or multiply.

Fixed-point arithmetic sidesteps this by representing a real number as a scaled integer instead of a `float` or `double`. A common convention is Qm.n notation: m integer bits, n fractional bits. In **Q16.16** format, a 32-bit signed integer represents a real number scaled by 2^{16} (65536):

```
using Fixed16_16 = std::int32_t;

constexpr int kFractionalBits{16};
constexpr Fixed16_16 kOne{1 << kFractionalBits};  // 65536
```

```
// Convert a floating-point value to Q16.16 fixed-point.
Fixed16_16 toFixed(const double value) noexcept
{
    return static_cast<Fixed16_16>(value * kOne);
}

// Convert a Q16.16 fixed-point value back to floating-point.
double toDouble(const Fixed16_16 value) noexcept
{
    return static_cast<double>(value) / kOne;
}
```

Addition of two fixed-point values is just integer addition: no special handling needed, since both operands are scaled by the same factor. Multiplication needs more care: multiplying two Q16.16 values naively overflows a 32-bit integer almost immediately (two values scaled by 2^{16} multiply to a result scaled by 2^{32}), so the multiply has to happen in a wider type before scaling back down:

```
// Multiply two Q16.16 fixed-point values.
Fixed16_16 fixedMultiply(const Fixed16_16 a, const Fixed16_16 b) noexcept
{
    const auto product = static_cast<std::int64_t>(a) * static_cast<std::int64_t>(b)
    ↪ ;
    return static_cast<Fixed16_16>(product >> kFractionalBits);
}
```

The trade-off: fixed-point trades dynamic range and some precision (Q16.16 can represent roughly $\pm 32{,}767$ with a resolution of about 0.0000153) for arithmetic that's fast and deterministic on targets without an FPU, with no software floating-point emulation involved.

10.2.3 Quantization

Even on targets *with* an FPU, storing weights as 8-bit integers instead of 32-bit floats cuts memory four times, and many embedded cores can execute int8 operations faster than float32 ones. The common approach, **post-training quantization**, trains the network at full precision as done throughout this book, then converts the trained weights to a smaller integer type for deployment, typically alongside a per-layer scale factor (conceptually similar to the fixed-point conversion above, but chosen per tensor rather than a single fixed format for the whole network). A full quantization pipeline is out of scope for this book, but it's worth recognizing the idea: *train* at high precision, *deploy* at low precision.

10.2.4 Real-time and latency constraints

A conv layer performs on the order of `inputSize² × kernelSize²` multiply-adds per feedforward pass. On a laptop that's negligible; inside a control loop with a hard deadline, e.g. a motor controller running at 1 kHz, giving the entire loop (sensor read, inference, actuation) a 1 ms budget, it can dominate the loop's execution time. For embedded systems, the *worst-case* execution time matters far more than the average case: a network that's usually fast but occasionally spikes in latency can violate a real-time deadline in exactly the way that doesn't show up during casual testing.

10.2.5 Dynamic memory allocation

The `Dense`, `Conv`, `MaxPool`, and `Flatten` implementations built in this book, and the standalone `ml::ConvLayer`, `ml::MaxPoolLayer` and `ml::FlattenLayer` structs from Chapters 6 and 7, all use `std::vector`, which allocates on the heap. On many embedded targets, heap allocation is avoided or tightly controlled: long-running systems risk heap fragmentation over time, allocation latency isn't constant-time, and some bare-metal targets have no heap at all. A common alternative is sizing everything at compile time with `std::array` (or plain C arrays) instead of `std::vector`, trading the convenience of runtime-sized layers for fully static, more predictable memory use.

10.3 Review

What you should be able to do now

After this chapter, you should:

- Have created a concrete subclass `Flatten` that inherits `ml::flatten_layer::Interface`.
- Have implemented `feedforward()` and `backpropagate()` for `Flatten` (note: no `optimize()`; the interface has none, since a flatten layer has no trainable parameters).
- Have a fully working, end-to-end CNN: `input → Conv → MaxPool → Flatten → Dense → prediction`.
- Recognize the shared feedforward, backpropagate and optimize pattern common to every layer type built in this book.
- Be able to compare the layer types built in this book (`Fixed`, `Dense`, `Conv`, `MaxPool`, `Flatten`) in terms of trainable parameters, computation, and role in the network.
- Be able to reason about memory footprint, numeric precision, and real-time constraints when deploying a trained network outside of a general-purpose computer.

Questions to test yourself

You should be able to explain:

1. Can you trace a single training example through your finished `cnn_work` pipeline, from feedforward through backpropagation to weight updates, describing what happens at each layer?
2. What do all the layer types built in this book have in common in their interface (feedforward, backpropagate, optimize), and where do they genuinely differ?
3. Which layer types have trainable parameters, and which don't? Why?
4. If you were designing a new CNN for a different image classification task, how would you decide how many conv and pooling layers to use, and why?
5. Why might a network that runs comfortably on a laptop fail to fit, or fail to meet a deadline, on a microcontroller?
6. What's the trade-off between fixed-point and floating-point arithmetic, and why does it matter more on some targets than others?

Looking ahead

This is the last chapter, and with it the book's main text is complete. What remains is to work the exercises in §10.4, and to compare the Calculation and Reflection exercises with their worked solutions in Appendix A. After that, the two written papers in Appendices B to F are there to test, on paper and without a compiler, how much of all ten chapters you can reconstruct on your own.

10.4 Exercises

How to work these. Sets 1 to 3 finish `cnn_work`, `lectures/L08/cnn_work`: keep working in the same project, where `flatten.hpp` and `flatten.cpp` are already in place as placeholders. Sets 4 to 6 are for self-assessment, reviewing the whole book: Set 4 tests whether you can recognize a layer type from its algorithm alone, Set 5 tests implementation-level understanding (shapes, parameter counts, and what has to be remembered for backpropagation), and Set 6 is a short set of exam-style conceptual questions. Everything you need to answer them is in the earlier chapters. Set 7 takes the network off the laptop, as §10.2 describes: one exercise on paper, one in code.

How to check your work. The test suite in `lectures/L10/exercises/test` points back at `cnn_work`; Exercise 10.9 runs it, and `make ML_DIR=<path>` points it at a tree of your own elsewhere. The worked solutions to Sets 4 to 6 and to Exercise 10.18 are in Appendix A.

The finished reference implementation of the whole pipeline is published in the companion repository as `lectures/L10/cnn_demo`, and this is the chapter it's there for: compare your `cnn_work` against it once your own version runs.

EXERCISE SET 1

Declaring the Flatten Layer

You'll finish `cnn_work` (from Chapters 8 and 9), implementing a concrete class `Flatten`, replacing the temporary stub class `ml::flatten_layer::Stub` currently used for the flatten layer in `factory.cpp`. This is the same logic you already worked through by hand in §6.4 and implemented as the standalone struct `ml::FlattenLayer` in Chapter 7: here you're adapting that same logic to satisfy `cnn_work`'s polymorphic `flatten_layer::Interface`. Once this is done, every layer in `cnn_work` is real, and you'll wire the whole pipeline together for the first time.

Exercise 10.1 The `Flatten` class, declaration

CODE

In the header file `include/ml/flatten_layer/flatten.hpp` (currently a placeholder), add a class named `Flatten` that inherits `ml::flatten_layer::Interface`, typeset below straight from the repository:

- Use public inheritance and mark the class `final`.
- Override every method from the interface, including the destructor (which can be `= default`).
- Keep `[[nodiscard]]` on every override whose interface method has it. Attributes aren't inherited, so the override needs its own (see Chapter 3).

lectures/L08/cnn_work/include/ml/flatten_layer/interface.hpp

```
/**
 * @file Flatten layer interface.
 */
#pragma once

#include <cstddef>

#include "ml/types.hpp"

namespace ml::flatten_layer
{
/**
 * @brief Flatten layer interface.
 */
class Interface
{
public:
    /**
     * @brief Destructor.
     */
```

```
    virtual ~Interface() noexcept = default;

    /**
     * @brief Get the input size of the layer.
     *
     * @return The input size of the layer.
     */
    [[nodiscard]] virtual std::size_t inputSize() const noexcept = 0;

    /**
     * @brief Get the output size of the layer.
     *
     * @return The output size of the layer.
     */
    [[nodiscard]] virtual std::size_t outputSize() const noexcept = 0;

    /**
     * @brief Get the input gradients of the layer.
     *
     * @return Matrix holding the input gradients of the layer.
     */
    [[nodiscard]] virtual const Matrix2d& inputGradients() const noexcept = 0;

    /**
     * @brief Get the flattened output of the layer.
     *
     * @return Matrix holding the output of the layer.
     */
    [[nodiscard]] virtual const Matrix1d& output() const noexcept = 0;

    /**
     * @brief Flatten the input from 2D to 1D.
     *
     * @param[in] input Matrix holding input data.
     *
     * @return True on success, false on failure.
     */
    virtual bool feedforward(const Matrix2d& input) noexcept = 0;

    /**
     * @brief Unflatten the output gradients from 1D to 2D.
     *
     * @param[in] outputGradients Matrix holding output gradients.
     *
```

```
     * @return True on success, false on failure.
     */
    virtual bool backpropagate(const Matrix1d& outputGradients) noexcept = 0;
};
} // namespace ml::flatten_layer
```

Note: Unlike `Conv` and `MaxPool`, `flatten_layer::Interface` declares **no `optimize()` method at all**: not because it was left out, but because a flatten layer has nothing to optimize and the interface simply doesn't ask for it. Don't add one.

Exercise 10.2 Removed constructors and operators CODE

Delete the default constructor, copy constructor, move constructor, copy assignment operator, and move assignment operator.

Exercise 10.3 Private member variables CODE

Add the following private member variables to `Flatten`:

- **`myInputGradients`**: the unflattened gradients; what `inputGradients()` returns (`Matrix2d`).
- **`myOutput`**: the flattened output; what `output()` returns (`Matrix1d`, not `Matrix2d`; note the interface's `output()` returns a 1D vector here, unlike `Conv` and `MaxPool`).

Exercise 10.4 Constructor CODE

Implement the constructor:

```
explicit Flatten(std::size_t inputSize) noexcept;
```

- Print an error message and call `std::terminate()` if `inputSize` is `0`.
- Size `myInputGradients` to `inputSize` (a square `inputSize × inputSize` matrix), and `myOutput` to `inputSize * inputSize` (see `ml::initMatrix()`).

Exercise 10.5 `feedforward()` CODE

Implement `bool feedforward(const Matrix2d& input) noexcept`:

- Return `false` if the size of `input` doesn't match the size of `myInputGradients` (`ml::matchDimensions()`), or if `input` isn't square (`ml::isMatrixSquare()`).
- Row-major flatten: for every position `(i, j)` in `input`, write it to `myOutput[i * inputSize + j]`, where `inputSize = myInputGradients.size()`.
- Return `true`.

Exercise 10.6 `backpropagate()` CODE

Implement `bool backpropagate(const Matrix1d& outputGradients) noexcept`:

- Return `false` if the size of `outputGradients` doesn't match the size of `myOutput`.
- The exact inverse of `feedforward()`: set `myInputGradients[i][j] = outputGradients[i * inputSize + j]` for every position `(i, j)`.
- Return `true`.

Exercise 10.7 Trivial accessors CODE

Implement the four accessors from the interface:

- **`inputSize()`**: returns `myInputGradients.size()`.
- **`outputSize()`**: returns `myOutput.size()`.
- **`inputGradients()`**: returns `myInputGradients`.
- **`output()`**: returns `myOutput`.

EXERCISE SET 2

Wiring It In

Exercise 10.8 Wiring the CNN together CODE

1. Make sure `source/ml/flatten_layer/flatten.cpp` is listed in `cnn_work`'s `Makefile` under `SRC_FILES`. It already is, as it has been since Chapter 8: the placeholder compiles to nothing.
2. In `source/ml/factory/factory.cpp`, find `Factory::flattenLayer()`: it currently returns a `flatten_layer::Stub` (marked with a `//! @todo` comment). Replace that with your new class:

   ```
   return std::make_unique<flatten_layer::Flatten>(inputSize);
   ```

3. Run `make`. Every layer in `cnn_work` (`Conv`, `MaxPool`, `Flatten`, and `Dense`, already done back in Chapter 5) is now real. The output of `predict()` is finally meaningful, and `train()` actually trains a working CNN end to end. Compare the predictions against the placeholder output you first saw from `cnn_work` in Chapter 8.
4. Compare notes against `lectures/L10/cnn_demo`, the finished reference implementation sitting right alongside this lecture's material: if your `cnn_work` doesn't train the way you expect, this is a good place to check your work against.

EXERCISE SET 3

Running the Tests

Exercise 10.9 Running the tests CODE

The final test suite is available in `lectures/L10/exercises/test`. It's cumulative: it carries the conv and max pooling layer tests over unchanged and adds unit tests for `Flatten`, plus **component tests for the network as a whole**.

Build and run it from this lecture's `appendix` directory:

```
make -C exercises/test
```

There's nothing to copy: `ML_DIR` already points back at the `cnn_work` codebase you started in Chapter 8. All 74 test cases should pass, in about a second.

The component tests are new in this chapter, and they're the reason this suite couldn't exist earlier: they build a whole network through the real factory and train it, so they only pass once `Conv`, `MaxPool` and `Flatten` are all real. `LearnsToRecognizeAllFourDigits` trains on the digits 0 to 3 and checks the right output node wins for each; the closest thing this book has to a final exam for your code.

Two things worth knowing before you start:

- The one flatten layer test to read is `BackpropagateUndoesFeedforward`. What really has to hold is that `backpropagate()` undoes exactly what `feedforward()` did; if the two disagree, gradients land on the wrong pixels and the conv layer quietly learns from noise, so the network still runs, it just never gets good. Note though that Exercise 10.5 mandates **row-major** order, and `FeedforwardIsRowMajor` and `BackpropagateIsRowMajor` check it, so a self-consistent column-major implementation still fails those two.
- If `LearnsToRecognizeAllFourDigits` fails but every unit test passes, the layers are individually right and something about how they're wired together is wrong. Start with `BackpropagationReachesTheConvLayerThroughEveryLayer`, which chains the four layers by hand and checks the gradients survive the trip back.

The test suite's README, `lectures/L10/exercises/test/README.md`, has more information, including how the convergence thresholds were measured.

EXERCISE SET 4

Recognize the Layer

Exercise 10.10 Six feedforward algorithms

REFLECTION

Below are six feedforward algorithms, written as pseudocode. For each one, decide which layer type it describes: a linear regression layer, a dense layer, a conv layer, a max pooling layer, or a flatten layer.

A

```
output = k * input + m
```

B

```
for n in nodes:
    sum = bias[n]
    for i in inputs:
        sum += weight[n][i] * input[i]
    output[n] = activation(sum)
```

C

```
for row in range(outputHeight):
    for col in range(outputWidth):
        sum = bias
        for kr in range(kernelSize):
            for kc in range(kernelSize):
                sum += kernel[kr][kc] * input[row + kr][col + kc]
        output[row][col] = activation(sum)
```

D

```
for row in range(outputHeight):
    for col in range(outputWidth):
        window = input[row * poolSize : row * poolSize + poolSize,
                       col * poolSize : col * poolSize + poolSize]
        output[row][col] = max(window)
```

E

```
output = reshape(input, (1, height * width))
```

F

```
sum = bias[0]
for i in inputs:
    sum += weight[0][i] * input[i]
output[0] = sum   // no activation function applied
```

The worked solution is on page 167.

EXERCISE SET 5

Implementation Questions

Consider the following network, processing a single-channel **8 × 8** input image:

```
input (8×8)
  → conv layer (one 3×3 kernel, stride 1, zero-padded)
  → max pooling layer (2×2, stride 2)
  → flatten layer
  → dense layer (10 nodes)
```

Exercise 10.11 Tracing shapes

CALCULATION

What is the shape of the data after each layer (conv, max pooling, flatten, dense)?

The worked solution is on page 168.

Exercise 10.12 Counting trainable parameters

CALCULATION

How many trainable parameters does each layer have? What is the total for the whole network?

The worked solution is on page 169.

Exercise 10.13 What max pooling has to remember

REFLECTION

The max pooling layer has no trainable parameters, yet it still needs to store something during `feedforward()` so that `backpropagate()` works correctly. What does it need to remember, and why?

The worked solution is on page 169.

Exercise 10.14 Conv vs. dense, parameter count

CALCULATION

A dense layer connecting the flattened 8×8 image (64 values) directly to a single output node would need 64 weights + 1 bias = 65 trainable parameters just to combine the whole image once. The conv layer in the network above combines local pixel neighborhoods using only how many trainable parameters? Why is the difference so large?

The worked solution is on page 170.

Exercise 10.15 Flatten in reverse

REFLECTION

During `feedforward()`, a flatten layer reshapes a $\mathbf{3 \times 4}$ matrix into a $\mathbf{1 \times 12}$ vector. During `backpropagate()`, it receives a $\mathbf{1 \times 12}$ gradient vector from the next layer. What shape does it reshape that gradient vector back into, and why?

The worked solution is on page 170.

EXERCISE SET 6

Exam-Style Questions

Exercise 10.16 True or false?

REFLECTION

For each statement, decide if it's true or false, and briefly justify your answer.

1. A max pooling layer has trainable parameters that are updated during `optimize()`.
2. The same kernel (the same weights) is reused at every spatial position in a conv layer.
3. A flatten layer's `backpropagate()` performs the exact same reshape operation as its `feedforward()`.
4. A dense layer with exactly one node, no activation function, and no hidden layer around it behaves identically to a linear regression layer.
5. Adding more filters (kernels) to a conv layer increases its trainable parameter count, but doesn't by itself change the width or height of its output.

The worked solution is on page 170.

Exercise 10.17 Short answer

REFLECTION

6. In one or two sentences, explain why swapping a dense layer for a conv layer generally means far fewer trainable parameters for the same size of input image.
7. A student claims: *"Max pooling layers slow down training because they have so many weights to update."* Is this correct? Why or why not?
8. Most layer types in this book implement the same three methods: `feedforward()`, `backpropagate()`, and `optimize()`. What does `optimize()` do for a max pooling layer, which has no trainable parameters? And why does the flatten layer's interface not declare `optimize()` at all, rather than declaring it and leaving it empty?

The worked solution is on page 171.

EXERCISE SET 7

Off the Laptop

Two of the constraints from §10.2, worked through by hand: memory footprint on paper, and fixed-point arithmetic in code.

Exercise 10.18 Memory footprint

CALCULATION

Using the layer sizes from the network you built in Chapters 4 and 5 (or any other layer configuration of your choice):

1. Compute the total number of trainable values (weights and biases) across all layers.
2. Compute the memory footprint at `double` (8 bytes), `float` (4 bytes), and `int8` (1 byte) per value.
3. Compare your numbers against the example in §10.2.1 of a layer sized for 28×28 pixel images (785 KB, 392.5 KB and 98.125 KB). Which of the three precisions would plausibly fit into a microcontroller with 64 KB of RAM? With 512 KB?

The worked solution is on page 172.

Exercise 10.19 Fixed-point conversion

CODE

Take the `predict()` method from Chapter 1's `ml::lin_reg::Fixed` class ($y = k \cdot x + m$):

1. Add `toFixed()`, `toDouble()`, and `fixedMultiply()` as shown in §10.2.2 (an addition helper isn't needed: plain integer + already works for Q16.16 values).
2. Implement a free function `Fixed16_16 predictFixed(Fixed16_16 weight, Fixed16_16 bias, Fixed16_16 input)` that computes the same prediction as `predict()`, but entirely in Q16.16 fixed-point using `fixedMultiply()` for the multiplication.
3. In `main()`, pick a few `(weight, bias, input)` triples, convert them to fixed-point, run both `predict()` (floating-point) and `predictFixed()` (fixed-point), convert the fixed-point result back with `toDouble()`, and print both side by side.
4. Confirm the two results agree to within roughly 0.0001 for each test case. If they don't, double-check the `>>` shift in `fixedMultiply()`: it's the step most likely to be off by a factor of `kOne`.

APPENDIX A

Worked Solutions

This appendix holds the solutions to the exercises that are answered on paper: the Calculation exercises, where a model is trained or an image traced by hand, and the Reflection exercises, where the answer is a sentence or two of reasoning. Each solution is headed with the number of the exercise it answers and the page that exercise is on, and every step is shown, so a slip can be traced to the line it was made on.

The solutions to the Code exercises are not here. They are published in the companion repository, lecture by lecture, and each chapter's exercises say where.

Work each exercise before you read its solution. Most of these calculations are short, and all of them are much easier to follow than to produce; the point of doing one is finding out which step you could not have produced.

A.1 Chapter 1: Linear Regression

Exercise 1.1 Training by hand

SOLUTION

The exercise is on page 17.

Five training sets defined by $y = 3x + 2$:

x	y
0	2
1	5
2	8
3	11
4	14

The model starts out with both parameters at zero, and is trained for one epoch with a learning rate of 10%:

$$\begin{cases} k = 0 \\ m = 0 \end{cases}, \qquad LR = 0.1$$

The update rule is the one from §1.3, applied to every training set, including the one at $x = 0$:

$$\delta = y_{\text{ref}} - y_p, \quad \Delta e = \delta \cdot LR, \quad m = m + \Delta e, \quad k = k + \Delta e \cdot x$$

The training sets are used one at a time, in the order listed above.

Training set 1

From the first training set we get input $x = 0$ and reference value $y_{\text{ref}} = 2$:

$$\begin{cases} x = 0 \\ y_{\text{ref}} = 2 \end{cases}$$

Both parameters are zero at the start, so the predicted output y_p is zero as well:

$$y_p = k \cdot x + m = 0 \cdot 0 + 0 = 0$$

The deviation δ is therefore two:

$$\delta = y_{\text{ref}} - y_p = 2 - 0 = 2$$

For a learning rate LR of 10%, the adjustment amount Δe is 0.2:

$$\Delta e = \delta \cdot LR = 2 \cdot 0.1 = 0.2$$

The m-value is increased directly by the adjustment amount Δe:

$$m = m + \Delta e = 0 + 0.2 = 0.2$$

The k-value is increased by Δe multiplied by x, which when $x = 0$ leaves it untouched:

$$k = k + \Delta e \cdot x = 0 + 0.2 \cdot 0 = 0$$

After the first round of training:

$$\begin{cases} k = 0 \\ m = 0.2 \end{cases}$$

Training set 2

$$\begin{cases} x = 1 \\ y_{\text{ref}} = 5 \end{cases}$$

$$\begin{aligned} y_p &= k \cdot x + m = 0 \cdot 1 + 0.2 = 0.2 \\ \delta &= y_{\text{ref}} - y_p = 5 - 0.2 = 4.8 \\ \Delta e &= \delta \cdot LR = 4.8 \cdot 0.1 = 0.48 \\ m &= m + \Delta e = 0.2 + 0.48 = 0.68 \\ k &= k + \Delta e \cdot x = 0 + 0.48 \cdot 1 = 0.48 \end{aligned}$$

After the second round of training:

$$\begin{cases} k = 0.48 \\ m = 0.68 \end{cases}$$

Training set 3

$$\begin{cases} x = 2 \\ y_{\text{ref}} = 8 \end{cases}$$
$$y_p = k \cdot x + m = 0.48 \cdot 2 + 0.68 = 1.64$$
$$\delta = y_{\text{ref}} - y_p = 8 - 1.64 = 6.36$$
$$\Delta e = \delta \cdot LR = 6.36 \cdot 0.1 = 0.636$$
$$m = m + \Delta e = 0.68 + 0.636 = 1.316$$
$$k = k + \Delta e \cdot x = 0.48 + 0.636 \cdot 2 = 1.752$$

After the third round of training:

$$\begin{cases} k = 1.752 \\ m = 1.316 \end{cases}$$

The deviation has grown for three sets in a row, simply because the reference values grow faster than the parameters have managed to so far.

Training set 4

$$\begin{cases} x = 3 \\ y_{\text{ref}} = 11 \end{cases}$$
$$y_p = k \cdot x + m = 1.752 \cdot 3 + 1.316 = 6.572$$
$$\delta = y_{\text{ref}} - y_p = 11 - 6.572 = 4.428$$
$$\Delta e = \delta \cdot LR = 4.428 \cdot 0.1 = 0.4428$$
$$m = m + \Delta e = 1.316 + 0.4428 = 1.7588$$
$$k = k + \Delta e \cdot x = 1.752 + 0.4428 \cdot 3 = 3.0804$$

After the fourth round of training:

$$\begin{cases} k = 3.0804 \\ m = 1.7588 \end{cases}$$

The deviation has decreased for the first time, and the k-value has passed its target of 3.

Training set 5

$$\begin{cases} x = 4 \\ y_{\text{ref}} = 14 \end{cases}$$
$$y_p = k \cdot x + m = 3.0804 \cdot 4 + 1.7588 = 14.0804$$
$$\delta = y_{\text{ref}} - y_p = 14 - 14.0804 = -0.0804$$

The deviation is negative for the first time: the prediction overshoots the reference value, so both parameters are adjusted downwards.

$$\Delta e = \delta \cdot LR = -0.0804 \cdot 0.1 = -0.00804$$
$$m = m + \Delta e = 1.7588 - 0.00804 = 1.75076$$
$$k = k + \Delta e \cdot x = 3.0804 - 0.00804 \cdot 4 = 3.04824$$

After the fifth and final round of training:

$$\begin{cases} k = 3.04824 \\ m = 1.75076 \end{cases}$$

After a single epoch, the model predicts according to the following formula:

$$y_p = 3.04824 \cdot x + 1.75076$$

Prediction

Predicted output y_p and reference values y_{ref} for input x in the range $[-5, 5]$, with the predictions rounded to two decimal places:

x	y_p	y_{ref}
−5	−13.49	−13
−4	−10.44	−10
−3	−7.39	−7
−2	−4.35	−4
−1	−1.30	−1
0	1.75	2
1	4.80	5
2	7.85	8
3	10.90	11
4	13.94	14
5	16.99	17

The weight is already within 2% of its target after one epoch, while the bias is still 0.25 short, which is what tilts every prediction in the table. The bias converges more slowly here

because it's only ever adjusted by Δe, whereas the weight is adjusted by $\Delta e \cdot x$, i.e. up to four times as much per training set.

The `x = 0` shortcut used by the code in Exercise 1.13 would have assigned $m = 2$ directly on the first training set, and the run would have ended somewhere else entirely. That's why the exercise asks for the rule from §1.3 here: it's the rule the hand calculations in §1.3.3 demonstrate, and it converges on both parameters from the training data alone.

A.2 Chapter 3: Neural Networks

Exercise 3.1 Odd parity, first training set

SOLUTION

The exercise is on page 54.

All calculations are shown below. Every node uses ReLU, so $\sigma(s) = s$ for a positive weighted sum, and its derivative $\sigma'(s)$ is 1 for every positive sum and 0 otherwise.

Feedforward

$$
\begin{aligned}
x_1 &= X_1 = 0, \quad x_2 = X_2 = 1 \\
y_1 &= \sigma(b_1 + x_1 w_1 + x_2 w_2) = \sigma(0.2 + 0 \cdot 0.5 + 1 \cdot 0.6) = \sigma(0.8) = 0.8 \\
y_2 &= \sigma(b_2 + x_1 w_3 + x_2 w_4) = \sigma(0.4 + 0 \cdot 0.3 + 1 \cdot 0.8) = \sigma(1.2) = 1.2 \\
y_3 &= \sigma(b_3 + y_1 w_5 + y_2 w_6) = \sigma(0.6 + 0.8 \cdot 0.1 + 1.2 \cdot 0.9) = \sigma(1.76) = 1.76 \\
Y &= y_3 = 1.76
\end{aligned}
$$

Backpropagation

$$
\begin{aligned}
\delta_3 &= Y_{\text{train}} - y_3 = 1 - 1.76 = -0.76 \\
e_3 &= \delta_3 \cdot \sigma'(s_3) = -0.76 \cdot 1 = -0.76 \\
\delta_1 &= e_3 w_5 = -0.76 \cdot 0.1 = -0.076 \\
\delta_2 &= e_3 w_6 = -0.76 \cdot 0.9 = -0.684 \\
e_1 &= \delta_1 \cdot \sigma'(s_1) = -0.076 \cdot 1 = -0.076 \\
e_2 &= \delta_2 \cdot \sigma'(s_2) = -0.684 \cdot 1 = -0.684
\end{aligned}
$$

Optimization

$$\begin{aligned}
\Delta c_3 &= e_3 \cdot LR = -0.76 \cdot 0.1 = -0.076 \\
b_3 &= b_3 + \Delta c_3 = 0.6 + (-0.076) = 0.524 \\
w_5 &= w_5 + \Delta c_3 \cdot y_1 = 0.1 + (-0.076) \cdot 0.8 = 0.0392 \\
w_6 &= w_6 + \Delta c_3 \cdot y_2 = 0.9 + (-0.076) \cdot 1.2 = 0.8088
\end{aligned}$$

$$\begin{aligned}
\Delta c_1 &= e_1 \cdot LR = -0.076 \cdot 0.1 = -0.0076 \\
\Delta c_2 &= e_2 \cdot LR = -0.684 \cdot 0.1 = -0.0684
\end{aligned}$$

$$\begin{aligned}
b_1 &= 0.2 + (-0.0076) = 0.1924 \\
w_1 &= 0.5 + (-0.0076) \cdot 0 = 0.5 \\
w_2 &= 0.6 + (-0.0076) \cdot 1 = 0.5924
\end{aligned}$$

$$\begin{aligned}
b_2 &= 0.4 + (-0.0684) = 0.3316 \\
w_3 &= 0.3 + (-0.0684) \cdot 0 = 0.3 \\
w_4 &= 0.8 + (-0.0684) \cdot 1 = 0.7316
\end{aligned}$$

Verification (via feedforward)
With the new parameters and the same inputs $X_1X_2 = 01$:

$$\begin{aligned}
y_1 &= \sigma(0.1924 + 0 \cdot 0.5 + 1 \cdot 0.5924) = \sigma(0.7848) = 0.7848 \\
y_2 &= \sigma(0.3316 + 0 \cdot 0.3 + 1 \cdot 0.7316) = \sigma(1.0632) = 1.0632 \\
y_3 &= \sigma(0.524 + 0.7848 \cdot 0.0392 + 1.0632 \cdot 0.8088) \approx \sigma(1.41) = 1.41 \\
Y &= y_3 \approx 1.41, \quad \delta_3 = 1 - 1.41 = -0.41
\end{aligned}$$

The deviation decreased from -0.76 to -0.41. With further rounds of training and a lower learning rate (e.g. 0.01), the deviation could be brought much closer to zero.

Exercise 3.2 Inverting two inputs

SOLUTION

The exercise is on page 55.

All calculations are shown below. Every weighted sum in this example is positive, so every ReLU derivative is 1 and each computed error e equals its deviation δ.

Feedforward

$$
\begin{aligned}
&x_1 = 1, \quad x_2 = 0\\
&y_1 = \sigma(0.1 + 1 \cdot 0.2 + 0 \cdot 0.9) = 0.3\\
&y_2 = \sigma(0.1 + 1 \cdot 0.3 + 0 \cdot 0.8) = 0.4\\
&y_3 = \sigma(0.7 + 1 \cdot 0.5 + 0 \cdot 0.4) = 1.2\\
&y_4 = \sigma(0.1 + 0.3 \cdot 0.1 + 0.4 \cdot 0.1 + 1.2 \cdot 1.0) = \sigma(1.37) = 1.37\\
&y_5 = \sigma(0.6 + 0.3 \cdot 0.3 + 0.4 \cdot 0.0 + 1.2 \cdot 0.6) = \sigma(1.41) = 1.41\\
&Y_1 = y_4 = 1.37, \quad Y_2 = y_5 = 1.41
\end{aligned}
$$

Backpropagation

$$
\begin{aligned}
&\delta_4 = 0 - 1.37 = -1.37, \quad e_4 = -1.37\\
&\delta_5 = 1 - 1.41 = -0.41, \quad e_5 = -0.41\\
&\delta_1 = e_4 w_7 + e_5 w_{10} = -1.37 \cdot 0.1 + (-0.41) \cdot 0.3 = -0.26\\
&\delta_2 = e_4 w_8 + e_5 w_{11} = -1.37 \cdot 0.1 + (-0.41) \cdot 0.0 = -0.137\\
&\delta_3 = e_4 w_9 + e_5 w_{12} = -1.37 \cdot 1.0 + (-0.41) \cdot 0.6 = -1.616\\
&e_1 = -0.26, \quad e_2 = -0.137, \quad e_3 = -1.616
\end{aligned}
$$

Optimization

$$
\begin{aligned}
\Delta c_4 = -1.37 \cdot 0.1 = -0.137, \quad \Delta c_5 = -0.41 \cdot 0.1 = -0.041\\
b_4 = 0.1 + (-0.137) = -0.037, \quad b_5 = 0.6 + (-0.041) = 0.559
\end{aligned}
$$

Output-layer weights, scaled by the hidden layer's outputs $y_1 = 0.3$, $y_2 = 0.4$ and $y_3 = 1.2$:

$$
\begin{aligned}
w_7 &= 0.1 + (-0.137) \cdot 0.3 = 0.0589, & w_{10} &= 0.3 + (-0.041) \cdot 0.3 = 0.2877\\
w_8 &= 0.1 + (-0.137) \cdot 0.4 = 0.0452, & w_{11} &= 0.0 + (-0.041) \cdot 0.4 = -0.0164\\
w_9 &= 1.0 + (-0.137) \cdot 1.2 = 0.8356, & w_{12} &= 0.6 + (-0.041) \cdot 1.2 = 0.5508
\end{aligned}
$$

Hidden-layer biases:

$$
\begin{aligned}
\Delta c_1 &= -0.026, & \Delta c_2 &= -0.0137, & \Delta c_3 &= -0.1616\\
b_1 &= 0.074, & b_2 &= 0.0863, & b_3 &= 0.5384
\end{aligned}
$$

Hidden-layer weights, scaled by the inputs $x_1 = 1$ and $x_2 = 0$, so the three weights on x_2 keep their values:

$$
\begin{aligned}
w_1 &= 0.2 + (-0.026) \cdot 1 = 0.174, & w_2 &= 0.9 + (-0.026) \cdot 0 = 0.9\\
w_3 &= 0.3 + (-0.0137) \cdot 1 = 0.2863, & w_4 &= 0.8 + (-0.0137) \cdot 0 = 0.8\\
w_5 &= 0.5 + (-0.1616) \cdot 1 = 0.3384, & w_6 &= 0.4 + (-0.1616) \cdot 0 = 0.4
\end{aligned}
$$

Verification (via feedforward)
With the new parameters and $X_1 X_2 = 10$:

$$
\begin{aligned}
&y_1 \approx 0.248, \quad y_2 \approx 0.3726, \quad y_3 \approx 0.8768 \\
&Y_1 = y_4 = \sigma(-0.037 + 0.248 \cdot 0.0589 + 0.3726 \cdot 0.0452 + 0.8768 \cdot 0.8356) \approx 0.727 \\
&Y_2 = y_5 = \sigma(0.559 + 0.248 \cdot 0.2877 + 0.3726 \cdot (-0.0164) + 0.8768 \cdot 0.5508) \approx 1.107 \\
&\delta_4 \approx 0 - 0.727 = -0.727, \quad \delta_5 \approx 1 - 1.107 = -0.107
\end{aligned}
$$

The deviations are lower than before (from -1.37 and -0.41). With further rounds of training and a lower learning rate (e.g. 0.01), the deviation could be brought much closer to zero.

Exercise 3.3 Odd parity, after one epoch

SOLUTION

The exercise is on page 56.

All calculations are shown below.

Feedforward

$$
\begin{aligned}
x_1 &= X_1 = 1, \quad x_2 = X_2 = 1 \\
y_1 &= \sigma(b_1 + x_1 w_1 + x_2 w_2) = \sigma(0.1 + 1 \cdot 0.5 + 1 \cdot 0.4) = \sigma(1.0) = 1.0 \\
y_2 &= \sigma(b_2 + x_1 w_3 + x_2 w_4) = \sigma(-0.4 + 1 \cdot (-0.2) + 1 \cdot 0.1) = \sigma(-0.5) = 0 \\
y_3 &= \sigma(b_3 + y_1 w_5 + y_2 w_6) = \sigma(-0.2 + 1.0 \cdot 0.7 + 0 \cdot 0.8) = \sigma(0.5) = 0.5 \\
Y &= y_3 = 0.5
\end{aligned}
$$

Backpropagation

$$
\begin{aligned}
\delta_3 &= Y_{\text{train}} - y_3 = 0 - 0.5 = -0.5 \\
e_3 &= \delta_3 \cdot \sigma'(s_3) = -0.5 \cdot 1 = -0.5 \\
\delta_1 &= e_3 w_5 = -0.5 \cdot 0.7 = -0.35 \\
\delta_2 &= e_3 w_6 = -0.5 \cdot 0.8 = -0.4 \\
e_1 &= \delta_1 \cdot \sigma'(s_1) = -0.35 \cdot 1 = -0.35 \\
e_2 &= \delta_2 \cdot \sigma'(s_2) = -0.4 \cdot 0 = 0
\end{aligned}
$$

Note that $s_2 = -0.5$, which means $\sigma'(s_2) = 0$ since the derivative of ReLU is zero for weighted sums ≤ 0. Node 2 therefore doesn't contribute to the error and won't be updated in this step.

Optimization

$$\begin{aligned}
\Delta c_3 &= e_3 \cdot LR = -0.5 \cdot 0.1 = -0.05 \\
b_3 &= b_3 + \Delta c_3 = -0.2 + (-0.05) = -0.25 \\
w_5 &= w_5 + \Delta c_3 \cdot y_1 = 0.7 + (-0.05) \cdot 1.0 = 0.65 \\
w_6 &= w_6 + \Delta c_3 \cdot y_2 = 0.8 + (-0.05) \cdot 0 = 0.8 \\
\Delta c_1 &= e_1 \cdot LR = -0.35 \cdot 0.1 = -0.035 \\
\Delta c_2 &= e_2 \cdot LR = 0 \cdot 0.1 = 0 \\
b_1 &= 0.1 + (-0.035) = 0.065 \\
w_1 &= 0.5 + (-0.035) \cdot 1 = 0.465 \\
w_2 &= 0.4 + (-0.035) \cdot 1 = 0.365 \\
b_2 &= -0.4 + 0 = -0.4 \\
w_3 &= -0.2 + 0 \cdot 1 = -0.2 \\
w_4 &= 0.1 + 0 \cdot 1 = 0.1
\end{aligned}$$

Since $\Delta c_2 = 0$, b_2, w_3, and w_4 remain completely unchanged.

Verification (via feedforward)
With the new parameters and the same inputs $X_1 X_2 = 11$:

$$\begin{aligned}
y_1 &= \sigma(0.065 + 1 \cdot 0.465 + 1 \cdot 0.365) = \sigma(0.895) = 0.895 \\
y_2 &= \sigma(-0.4 + 1 \cdot (-0.2) + 1 \cdot 0.1) = \sigma(-0.5) = 0 \\
y_3 &= \sigma(-0.25 + 0.895 \cdot 0.65 + 0 \cdot 0.8) = \sigma(0.332) = 0.332 \\
Y &= y_3 \approx 0.332, \quad \delta_3 = 0 - 0.332 = -0.332
\end{aligned}$$

Did the error decrease?
The deviation decreased from -0.5 to -0.332, so the error got smaller. The node that produced $y_2 = 0$, however, remained completely unchanged since the derivative of ReLU is zero where the input is ≤ 0, an example of the so-called "dying ReLU" problem. If this kept happening over several consecutive epochs, that node could never be relearned. To counteract this, **Leaky ReLU** could be used instead of ReLU, since it allows a weak signal (and therefore a non-zero derivative) even when the input is negative.

A.3 Chapter 6: Convolutional Neural Networks

Exercise 6.1 Feedforward

SOLUTION

The exercise is on page 108.

The whole feedforward pass is worked through below, step by step.

a) Padding the input image

The kernel is 2×2, so $n = 2/2 = 1$: one zero on each side. The padded image P is 6×6:

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

b) Feedforward through the conv layer

Output cell (i, j) is computed from the 2×2 window of P whose top-left corner is at (i, j), for i and j from 0 to 3. Two cells as examples, the top-left and the bottom-right:

$$\text{sum}(0,0) = 0 \cdot 0.2 + 0 \cdot 0.4 + 0 \cdot 0.6 + 1 \cdot 0.8 + 0.5 = 1.3$$
$$\text{sum}(3,3) = 0 \cdot 0.2 + 1 \cdot 0.4 + 1 \cdot 0.6 + 1 \cdot 0.8 + 0.5 = 2.3$$

Once the kernel and bias have been applied across the entire image, we get the following conv output (4×4):

$$\begin{bmatrix} 1.3 & 1.9 & 1.9 & 1.9 \\ 1.7 & 1.7 & 1.1 & 1.9 \\ 1.7 & 1.3 & 0.5 & 1.7 \\ 1.7 & 2.1 & 1.9 & 2.3 \end{bmatrix}$$

Every sum is positive, the smallest being 0.5, so ReLU passes each of them through unchanged: the pre-activation sums and the output are the same matrix here.

c) The max pooling layer (feedforward)

We split the conv layer's output into four pools.

Top-left corner: max value 1.9, at $(0, 1)$:

$$\begin{bmatrix} 1.3 & 1.9 \\ 1.7 & 1.7 \end{bmatrix}$$

Top-right corner: the max value 1.9 occurs in three places; we pass forward the first instance, at $(0, 2)$:

$$\begin{bmatrix} 1.9 & 1.9 \\ 1.1 & 1.9 \end{bmatrix}$$

Bottom-left corner: max value 2.1, at $(3, 1)$:

$$\begin{bmatrix} 1.7 & 1.3 \\ 1.7 & 2.1 \end{bmatrix}$$

Bottom-right corner: max value 2.3, at $(3, 3)$:

$$\begin{bmatrix} 0.5 & 1.7 \\ 1.9 & 2.3 \end{bmatrix}$$

The max pooling layer's output is therefore:

$$\begin{bmatrix} 1.9 & 1.9 \\ 2.1 & 2.3 \end{bmatrix}$$

d) Flatten (feedforward)

The max pooling layer's output is flattened, row by row, into a vector so it can be fed to the next layer:

$$[1.9, 1.9, 2.1, 2.3]$$

Exercise 6.2 Backpropagation and optimization

SOLUTION

The exercise is on page 110.

The dense layer sends back the gradients $[10, 20, 30, 40]$.

a) Flatten (backpropagation)

The gradients from the dense layer are reshaped back into a matrix, row by row, the exact inverse of the flatten in Exercise 6.1:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

b) Max pool (backpropagation)

Each gradient goes to the position its pool's maximum came from, found in Exercise 6.1, and the remaining positions get 0.

Top-left corner: gradient 10 is propagated to the max value 1.9 (top right):

$$\begin{bmatrix} 0 & 10 \\ 0 & 0 \end{bmatrix}$$

Top-right corner: gradient 20 is propagated to the first occurrence of the max value 1.9 (top left):

$$\begin{bmatrix} 20 & 0 \\ 0 & 0 \end{bmatrix}$$

Bottom-left corner: gradient 30 is propagated to the max value 2.1 (bottom right):

$$\begin{bmatrix} 0 & 0 \\ 0 & 30 \end{bmatrix}$$

Bottom-right corner: gradient 40 is propagated to the max value 2.3 (bottom right):

$$\begin{bmatrix} 0 & 0 \\ 0 & 40 \end{bmatrix}$$

Assembling all the pools' gradients into a single matrix gives us the gradient matrix that arrives at the conv layer's output:

$$\text{grad} = \begin{bmatrix} 0 & 10 & 20 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 30 & 0 & 40 \end{bmatrix}$$

c) Conv layer (backpropagation)

Every pre-activation sum from Exercise 6.1 is positive and the activation is ReLU, so $\sigma'(s) = 1$ at every position, and multiplying each incoming gradient by it changes nothing.

Bias gradient: the sum of all values in the gradient matrix:

$$\text{biasGradient} = 0 + 10 + 20 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 30 + 0 + 40 = 100$$

Kernel gradients: only four gradients are non-zero, at $(0,1)$, $(0,2)$, $(3,1)$ and $(3,3)$, so each kernel element gets four terms, one per non-zero gradient, reading the padded input P from part (a) at that kernel element's offset:

$$\begin{aligned} k_{00} &= P[0][1] \cdot 10 + P[0][2] \cdot 20 + P[3][1] \cdot 30 + P[3][3] \cdot 40 \\ &= 0 \cdot 10 + 0 \cdot 20 + 1 \cdot 30 + 0 \cdot 40 = 30 \\ k_{01} &= P[0][2] \cdot 10 + P[0][3] \cdot 20 + P[3][2] \cdot 30 + P[3][4] \cdot 40 \\ &= 0 \cdot 10 + 0 \cdot 20 + 0 \cdot 30 + 1 \cdot 40 = 40 \\ k_{10} &= P[1][1] \cdot 10 + P[1][2] \cdot 20 + P[4][1] \cdot 30 + P[4][3] \cdot 40 \\ &= 1 \cdot 10 + 1 \cdot 20 + 1 \cdot 30 + 1 \cdot 40 = 100 \\ k_{11} &= P[1][2] \cdot 10 + P[1][3] \cdot 20 + P[4][2] \cdot 30 + P[4][4] \cdot 40 \\ &= 1 \cdot 10 + 1 \cdot 20 + 1 \cdot 30 + 1 \cdot 40 = 100 \end{aligned}$$

The kernel gradients are therefore:

$$\begin{bmatrix} 30 & 40 \\ 100 & 100 \end{bmatrix}$$

Input gradients (padded input): each non-zero gradient spreads the kernel's weights, multiplied by that gradient, over the 2×2 patch of a new 6×6 matrix dX it came from. Only four gradients are non-zero, so only their contributions need computing:

- From `grad[0,1] = 10`: `dX(0,1) += 2`, `dX(0,2) += 4`, `dX(1,1) += 6`, `dX(1,2) += 8`.
- From `grad[0,2] = 20`: `dX(0,2) += 4`, `dX(0,3) += 8`, `dX(1,2) += 12`, `dX(1,3) += 16`.
- From `grad[3,1] = 30`: `dX(3,1) += 6`, `dX(3,2) += 12`, `dX(4,1) += 18`, `dX(4,2) += 24`.
- From `grad[3,3] = 40`: `dX(3,3) += 8`, `dX(3,4) += 16`, `dX(4,3) += 24`, `dX(4,4) += 32`.

After summing all contributions, we get the padded input gradient matrix:

$$dX = \begin{bmatrix} 0 & 2 & 8 & 8 & 0 & 0 \\ 0 & 6 & 20 & 16 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 12 & 8 & 16 & 0 \\ 0 & 18 & 24 & 24 & 32 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Removing the padding, the outermost row and column on every side, leaves a 4×4 matrix matching the original image; this is the gradient with respect to the input, which is passed further back through the network:

$$\begin{bmatrix} 6 & 20 & 16 & 0 \\ 0 & 0 & 0 & 0 \\ 6 & 12 & 8 & 16 \\ 18 & 24 & 24 & 32 \end{bmatrix}$$

d) Optimization

Finally, the kernel and bias are updated using the learning rate $LR = 0.001$:

$$k_{00} = 0.2 + 0.001 \cdot 30 = 0.23 \qquad k_{01} = 0.4 + 0.001 \cdot 40 = 0.44$$
$$k_{10} = 0.6 + 0.001 \cdot 100 = 0.70 \qquad k_{11} = 0.8 + 0.001 \cdot 100 = 0.90$$
$$\text{bias} = 0.5 + 0.001 \cdot 100 = 0.6$$

Updated kernel:

$$\begin{bmatrix} 0.23 & 0.44 \\ 0.70 & 0.90 \end{bmatrix}$$

Updated bias: 0.6.

These are the values the demo in Exercise 6.3 prints in its *expected* column: the conv output from Exercise 6.1, and the input gradients, the kernel and the bias from this one.

A.4 Chapter 10: The Flatten Layer and the Complete Network

Exercise 10.10 Six feedforward algorithms

SOLUTION

The exercise is on page 151.

What gives each layer away is its parameters and the shape of its loop: what is summed, over what, and whether the same weights come back at every position.

A: a linear regression layer

One input, one weight k, one bias m, and no activation function: $y = kx+m$, exactly `predict()` in Chapter 1.

B: a dense layer

Every node `n` has its own bias and its own weight for every input, the weighted sum over *all* inputs starts at the bias, and the activation function is applied to it: the feedforward of §5.2.

C: a conv layer

The loop runs over output positions, and each output is the weighted sum of a small `kernelSize × kernelSize` window of the input, starting at `(row, col)`, plus a bias, passed through the activation function. Two things separate it from B: the sum only covers a local window (locality), and `kernel` and `bias` carry no index for the output position, so the same weights are used at every position (weight sharing). The `input` it reads is the zero-padded one, which is why `input[row + kr][col + kc]` never runs off its edge.

D: a max pooling layer

Each output is the largest value in a non-overlapping `poolSize × poolSize` block: the block steps by `poolSize`, so the stride equals the pool size. There is no weight or bias anywhere, and nothing is summed.

E: a flatten layer

Nothing is computed at all; the data is only given a new shape, from `height × width` to one row of `height * width` values.

F: a dense layer with one node and no activation function

It has the dense layer's structure, a bias and one weight per input summed over all the inputs, but only for node 0, and with the activation function left out (`ActFunc::None`). That makes it linear regression generalised to several inputs, $y = m + \sum_i k_i x_i$: with a single input it is A written out as a loop. It isn't A as written, because it takes a vector of inputs rather than one, and it isn't B in general, because B has any number of nodes and applies an activation function. §10.1 makes the same point from the other side: `Fixed` is a `Dense` layer with one node and no activation function. Either name earns the answer, as long as the reason is given.

Exercise 10.11 Tracing shapes

SOLUTION

The exercise is on page 152.

After	Shape
conv layer	8×8
max pooling layer	4×4
flatten layer	1×16
dense layer	1×10

- **Conv.** The padding is `kernelSize / 2` $= 3/2 = 1$ on each side, so with stride 1 the output is $\lfloor (8 + 2 \cdot 1 - 3)/1 \rfloor + 1 = 8$ wide: the same size as the input, which is what the padding is for.
- **Max pooling.** Non-overlapping 2×2 blocks halve each side: $8/2 = 4$.
- **Flatten.** $4 \cdot 4 = 16$ values in one row.
- **Dense.** One output per node: 10 values.

Exercise 10.12 Counting trainable parameters

SOLUTION

The exercise is on page 152.

Layer	Trainable parameters
conv	$3 \cdot 3 + 1 = 10$
max pooling	0
flatten	0
dense	$10 \cdot 16 + 10 = 170$
total	180

The conv layer has one 3×3 kernel and one bias, whatever the size of the image. The dense layer has 10 nodes, each with one weight per input (the 16 values from the flatten layer) and a bias of its own. Max pooling and flatten are purely structural: one picks values that are already there, the other changes their shape, and neither has anything to learn. More than 94% of the network's parameters are in the dense layer at the end.

Exercise 10.13 What max pooling has to remember

SOLUTION

The exercise is on page 152.

It has to remember **which position in each block held the maximum**, or enough to find it again. The standalone `ml::MaxPoolLayer` of Chapter 7 stores the index of each block's maximum; `MaxPool` in Chapter 9 stores the input itself, `myInput`, and searches each block again.

The reason is how the gradient is routed. Only the position that produced a pooled value influenced the output, so `backpropagate()` must hand that position the whole incoming gradient and every other position in the block zero. The output alone can't say where the maximum was: a block holding `1.9` could have held it in any of its four cells. And the search in `backpropagate()` has to break ties exactly as `feedforward()` did, towards the first occurrence in row-major order, or the gradient lands on a pixel that didn't produce the output.

Exercise 10.14 Conv vs. dense, parameter count

SOLUTION

The exercise is on page 153.

10: nine kernel weights and one bias, against the dense layer's 65.

The difference comes from the two constraints a conv layer adds to a dense one:

- **Locality.** Each output looks at a 3×3 window, 9 pixels, rather than at all 64.
- **Weight sharing.** The same kernel is reused at every one of the 64 positions, instead of every output having weights of its own.

The comparison even flatters the dense layer. Its 65 parameters produce *one* output; the conv layer's 10 produce 64 of them, one per position. A dense layer producing the same 64 outputs from the same 64 inputs would need $64 \cdot 64 + 64 = 4160$ parameters. And the gap grows with the image: the dense layer's count grows with the number of pixels, while the conv layer's stays at 10 for any image size, because it only depends on the kernel.

Exercise 10.15 Flatten in reverse

SOLUTION

The exercise is on page 153.

Back into a $\mathbf{3 \times 4}$ matrix: the shape of the input the layer received, not of the output it produced.

`backpropagate()` computes the gradients for the layer's *input*, and hands them to the layer before it, which produced a 3×4 matrix and expects exactly one gradient per value it output. The reshape also has to be the exact inverse of the one in `feedforward()`: in row-major order, element (i, j) went to index $4i + j$ of the vector, so the gradient at index $4i + j$ has to come back to (i, j), the position whose value it belongs to. Any other order puts each gradient on a value it has nothing to do with, and the layers behind learn from noise.

The flatten layer in Chapter 10 only accepts square inputs, but the rule is the same for any shape.

Exercise 10.16 True or false?

SOLUTION

The exercise is on page 153.

1. **False.** A max pooling layer has no trainable parameters at all: it only selects the largest value in each block. Its `optimize()` exists because the interface it shares with `Conv` declares one, and it's a genuine no-op that returns `true`.
2. **True.** That is weight sharing: one kernel slides across the whole image, which is what keeps the parameter count down and lets the same pattern be recognized wherever it appears.
3. **False.** It performs the *inverse* reshape: `feedforward()` goes from 2D to 1D, and `backpropagate()` goes from 1D back to 2D, so each gradient lands on the value it came from.
4. **True**, for a single input. Both predict $y = wx+b$, and both update with the same rule: with no activation function the derivative is 1, so $\Delta e = \delta$, and b moves by $\delta \cdot LR$ and w by $\delta \cdot LR \cdot x$, exactly as m and k do in Chapter 1. Two differences in the book's implementations don't change the model, only where training starts and one shortcut: `ml::lin_reg::Fixed` starts at zero while `Dense` draws random start values, and `Fixed::optimize()` sets the bias directly when $x = 0$ (Exercise 1.13). With several inputs, the dense node is linear regression over several inputs.
5. **True.** Each extra kernel adds its own weights (and, usually, a bias of its own), so the parameter count grows. Each kernel produces its own feature map of the same width and height, so the output gains maps, or channels, not size; width and height are set by the input size, the kernel size, the padding and the stride. This book only ever uses one kernel per conv layer, to keep things simple.

Exercise 10.17 Short answer

SOLUTION

The exercise is on page 153.

6. A dense layer gives every output its own weight for every input pixel, so its parameter count grows with the size of the image; a conv layer combines only a small local window (locality) and reuses the same kernel at every position (weight sharing), so its parameter count is fixed by the kernel size alone.
7. **Incorrect.** A max pooling layer has no weights at all, so there is nothing for it to update; its `optimize()` does nothing. If anything it makes training *cheaper*: it shrinks the data every layer after it has to process, a 2×2 pool by a factor of four, which means fewer inputs, and fewer weights, in the dense layer behind it.
8. **`MaxPool::optimize()`** is a genuine no-op, not a placeholder: it ignores the learning rate and returns `true`. It has to exist because `MaxPool` shares `conv_layer::Interface` with `Conv`, which does need `optimize()`, and sharing the interface is what lets the network hold both in one list and call the same three methods on each.
   `flatten_layer::Interface` belongs to flatten layers alone, and a flatten layer can never have trainable parameters, so there is nothing an `optimize()` could ever do. Declaring it

and leaving it empty would promise something no flatten layer does, and would leave it to whoever writes the class to remember that the method must do nothing. Leaving it out moves that fact into the type system: the network simply doesn't call `optimize()` on its flatten layer, and a `Flatten` that tried to optimize something wouldn't compile.

Exercise 10.18 Memory footprint

SOLUTION

The exercise is on page 154.

The network from Chapters 4 and 5

Chapter 5 trains a hidden layer of 8 nodes over the 2 inputs of the XOR pattern, and an output layer of 1 node over those 8:

$$\begin{aligned} \text{hidden layer} &= 8 \cdot 2 \text{ weights} + 8 \text{ biases} = 24 \text{ values} \\ \text{output layer} &= 1 \cdot 8 \text{ weights} + 1 \text{ bias} = 9 \text{ values} \\ \text{total} &= 33 \text{ values} \end{aligned}$$

Type	Bytes each	Footprint
`double`	8	$33 \cdot 8 = 264$ B
`float`	4	$33 \cdot 4 = 132$ B
`int8`	1	$33 \cdot 1 = 33$ B

The smaller network of Chapter 4, 3 hidden nodes over 2 inputs and 1 output node over 3, is the one §10.2.1 counts: 13 values, so 104, 52 and 13 bytes.

Against the layer sized for 28 × 28 images

The course's networks fit comfortably at any of the three precisions on any target: even as `double`, a few hundred bytes. The layer of 128 nodes over 784 inputs is another matter, with 64 KB = 65,536 bytes and 512 KB = 524,288 bytes:

- **64 KB of RAM: none of the three fits.** Even at `int8`, the layer's 100,480 bytes (98.125 KB) are more than the whole RAM.
- **512 KB of RAM: `float` and `int8` fit, `double` does not.** 401,920 bytes (392.5 KB) and 100,480 bytes fit; 803,840 bytes (785 KB) do not.

"Plausibly" is the right word, because the RAM has to hold more than the weights: the layers' outputs, errors and gradients, the stack, and the rest of the program's data. `float` in 512 KB leaves about 120 KB for all of that, and a real network has more than one layer. This is the argument for training at full precision and deploying at `int8`, from §10.2.3.

APPENDIX B

The Self-Assessment Papers

This appendix and the four after it hold two complete four-hour papers for *Machine Learning*, with worked solutions. Appendices C and D are the papers, questions only; Appendices E and F are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. What you get instead is a test suite for the code exercises of Chapters 1 to 5 and 8 to 10, the conv demo of Chapter 6 printing its results next to the numbers you worked out by hand in that chapter's hand-training exercise, the worked solutions in Appendix A, and the self-assessment exercises in §10.4. Every one of those checks a component you built or a number you computed beside it.

These papers do not replace that. **They are here to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them and `make test` does not know they exist. They are useful where a written result is wanted anyway, for a certifying employer or a formal course credit, and useful on their own for finding out what you can reconstruct with nothing in front of you.

Take one after the last chapter. Every paper draws on all ten chapters, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood. The intended point is after Chapter 10, once your `cnn_work` trains end to end.

The two papers

Both cover the whole book, Chapters 1 to 10, and share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

They do **not** weight the book identically, and that is deliberate rather than accidental. Paper A spends more on the convolutional layer itself (Chapter 6 is worth roughly three times as much there), Paper B spends more on the finished network and its constraints (Chapter 10 is worth twice as much). Two topics appear on one paper only: the precision

and early stopping of Chapter 2 are examined on **B alone** (Q2c, Q2d), and the fixed-point and real-time material from §10.2 is likewise **B alone** (Q9b, Q9c). Paper A is also the only one that asks a candidate to write a method from scratch. If you are using the papers as a sitting and a resit, that asymmetry is worth knowing; if you need the two to examine exactly the same ground, sit both.

Paper A

Question	Topic	Chapters	Marks
1	Learning the rules instead of writing them	1, 2	12
2	The training loop that trains nothing	2, 4	12
3	A network trained by hand	3	14
4	The activation function, and the argument it is given	3, 5	10
5	The dense layer, written out	5	12
6	Why an image is not just a long vector	6	10
7	A convolutional layer, traced by hand	6, 7, 8	14
8	Max pooling, and the value it forgets	7, 9	8
9	The finished network, and the target it has to run on	10	8
			100

Paper B

Question	Topic	Chapters	Marks
1	The model, and the rate that destroys it	1	10
2	Order, precision, and knowing when to stop	2, 4	10
3	Two output nodes	3	14
4	The network class, and what a stub can prove	3, 4	12
5	The dense layer, and the mistake only Tanh can catch	5	12
6	Padding, sharing, and the kernel's gradient	6, 7, 8	14
7	A pooling layer that runs and returns true	7, 9	12
8	Every layer, side by side	10	8
9	Off the laptop	10	8
			100

Paper A leans towards **tracing an algorithm forwards**: one epoch of linear regression by hand, a full `feedforward()`/`backpropagate()`/`optimize()` step through a small network, a 4×4 image pushed through conv, pooling and flatten, and the gradients routed back. Paper B leans towards **the conditions under which the algorithm stops working**: the learning rate that diverges inside the range `train()` permits, the derivative that is right for ReLU and wrong for Tanh, the even kernel whose padding does not line up, and a pooling layer whose faults never produce an error message.

Code on a written paper

Unlike the papers for some other QAcademy courses, these two **do ask a candidate to write C++**. That is deliberate. This book teaches machine learning by building it: the specification of every layer is prose in a chapter and a test suite that tells you whether what you wrote satisfies it, and a paper that avoided code entirely would be examining a different book.

The code questions come in two shapes, and neither of them is a typing exercise:

- **Write one method.** `Dense::feedforward()`, `backpropagate()` and `optimize()` are asked for in Paper A; the answers are ten to twenty lines each. What carries the marks is the algorithm, namely the validation, the weighted sum starting at the bias, the derivative taken at the right argument; not the syntax.
- **Find what is wrong with the algorithm.** Both papers hand over a listing that compiles, runs, and returns `true`, and ask what it does instead of what it should. The faults are the ones the course's own test-suite READMEs single out: a shuffle nobody reads, backpropagation in the wrong order, a max search that starts at `0.0`, a gradient matrix that is never cleared.

Both papers state in their rubric that answers may be written in C++17 **or in unambiguous pseudocode**, and that nothing is marked on syntax. A missing semicolon costs nothing. A missing bounds check costs everything.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **The book's sign convention.** The deviation is $\delta = y_{\text{ref}} - y_p$ and every parameter is *increased* by its adjustment, so every update reads `parameter += gradient * learningRate` with no minus sign. A candidate who flips the sign consistently and says so is not penalized twice.
- **The activation derivative is evaluated at the weighted sum** s, never at the output y. Both papers have a question about what happens when it is not.
- **Conv layers zero-pad** with `pad = kernelSize / 2`, so the output is the same size as the input.
- **Max pooling breaks ties toward the first occurrence** in row-major order, and **flatten is row-major**, matching the hand-training exercise of Chapter 6 and the shipped implementations.
- **Only the values nobody could be expected to carry are supplied**: a handful of `tanh()` values, 2^{16} on Paper B, and that a KB is 1024 bytes. Parameter counts, output shapes and padding sizes are derived, because deriving them is the examinable skill.
- **Code questions refer to the `Dense`/`Shallow` API specified in Chapters 3 to 5**, the one the candidate builds, not to the refactored API in `cnn_demo`. The two differ: `cnn_demo`'s

`Dense` reverses the constructor arguments, renames the accessors, collapses the two `backpropagate()` overloads into one, and carries a seventh member variable. Answers written against either are accepted, but the model answers follow Chapters 3 to 5.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write machine learning code, so each carries the reasoning rather than the answer alone. Marks are shown per part in the solutions.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct update rule with an arithmetic slip in it is worth more than a correct number with no working, and both papers are built so that later parts consume earlier answers. Follow through an error rather than penalizing it twice.
- **The named traps are worth full marks on their own.** Several questions exist entirely to see whether a candidate avoids a specific mistake: the activation derivative taken from y instead of s, a gradient that ReLU has already annihilated, `optimize()` given the network's input instead of the previous layer's output, a max search starting at `0.0`, an `optimize()` invented for a layer that has nothing to optimize. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.
- **The discussion parts are not decoration.** "State why" is where the book's actual content is, and a paper marked only on the arithmetic would pass a candidate who has understood none of it.

APPENDIX C

Self-Assessment Paper A

Time: 4 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix E.

Rubric

Sign convention. The book computes a deviation $\delta = y_{\text{ref}} - y_p$ and *adds* the resulting adjustment to every parameter. Every update in this paper therefore reads `parameter += gradient * learningRate`, with no minus sign. An answer that flips the sign consistently and says so is not penalized twice.

The activation derivative is evaluated at the weighted sum s, never at the output y. This matters in more than one question.

$$\sigma_{\text{ReLU}}(s) = \begin{cases} s & s > 0 \\ 0 & s \le 0 \end{cases} \qquad \sigma'_{\text{ReLU}}(s) = \begin{cases} 1 & s > 0 \\ 0 & s \le 0 \end{cases}$$

$$\sigma_{\text{Tanh}}(s) = \tanh(s) \qquad \sigma'_{\text{Tanh}}(s) = 1 - \tanh^2(s) \qquad \sigma_{\text{None}}(s) = s \qquad \sigma'_{\text{None}}(s) = 1$$

Convolutional layers zero-pad with `pad = kernelSize / 2` (integer division), so the output has the same size as the input.

Max pooling breaks ties toward the first occurrence in row-major order, and **flatten is row-major**.

Code answers may be written in C++17 or in unambiguous pseudocode. Nothing on this paper is marked on syntax; a missing semicolon costs nothing and a missing bounds check costs everything.

Show the arithmetic. Where a question asks you to compute, an unsupported number scores nothing, and a correct method with a slip in it scores most of the marks.

Supplied constants

Quantity	Value
$\tanh(1.5)$	0.905148
$\tanh(0.905148)$	0.718795
1 KB	1024 B

Marks

Question	1	2	3	4	5	6	7	8	9
Marks	12	12	14	10	12	10	14	8	8

Question 1 Learning the rules instead of writing them

12 MARKS

(a) Traditional programming and machine learning both involve the same three things: input, rules, and output. They differ in which two are supplied and which one is produced. State the arrangement for each of the two.

Then name the pre-machine-learning paradigm the book contrasts with, give one class of task it handled well and one class it did not, and state the property of the input that separates the two. *(3 marks)*

(b) A linear regression model is to be trained on the training data below, which follows $y = 4x - 1$:

x	y
0	-1
1	3
2	7

Both parameters start at zero, the learning rate is $LR = 0.1$, and one epoch is run with the sets taken in the order shown.

Using the general update rules from Chapter 1, $\Delta e = \delta \cdot LR$ applied to both parameters, with no special case for any input, compute y_p, δ, Δe, k and m for each of the three training sets. *(4 marks)*

(c) Give the trained model's prediction at $x = 5$ and compare it against the reference value.

The worked example in §1.3.3 (which trained on $y = 2x + 1$) landed within a few hundredths of its target after a single epoch. State why this model did not, quoting the specific numbers from (b) that account for the difference. *(2 marks)*

(d) `ml::lin_reg::Fixed::optimize()` treats $x = 0$ as a special case: it assigns the reference value straight to the bias and leaves the weight untouched.

Redo the first training set under that rule, carry the result through the remaining two sets, and compare the final parameters against (b). Then state one situation in which the special case is a bad idea. *(3 marks)*

Question 2 The training loop that trains nothing

12 MARKS

The training method below compiles, runs, and returns `true`. It is the training loop from Chapter 4, and it is wrong in several independent ways.

```
bool Shallow::train(const std::size_t epochCount, const double precisionThreshold) ↪
    ↪ noexcept
{
    constexpr std::size_t evaluationInterval{100U};
    constexpr double initialLearningRate{0.1};

    double learningRate{initialLearningRate};
    double prevPrecision{};

    if ((0U == epochCount) || (0.0 >= precisionThreshold) || (1.0 <= ↪
    ↪ precisionThreshold))
    {
        return false;
    }
    myHiddenLayer.initParams();
    myOutputLayer.initParams();

    randomizeTrainOrder();

    for (std::size_t epoch{}; epoch < epochCount; ++epoch)
    {
        for (std::size_t x{}; x < myTrainOrder.size(); ++x)
        {
            feedforward(myTrainInput[x]);

            myHiddenLayer.backpropagate(myOutputLayer);
            myOutputLayer.backpropagate(myTrainOutput[x]);

            myHiddenLayer.optimize(myTrainInput[x], learningRate);
            myOutputLayer.optimize(myHiddenLayer.output(), learningRate);
        }
        const bool evaluate{(0U != epoch) && (0U == (epoch % evaluationInterval))};
```

```
        if (evaluate)
        {
            const auto currentPrecision = precision();

            if (precisionThreshold <= currentPrecision) { return true; }
            updateLearningRate(learningRate, prevPrecision, currentPrecision);
        }
    }
    return true;
}
```

The parameter reset, the evaluation every hundredth epoch and the learning rate rule are all as specified; the faults below are elsewhere.

(a) The loop was supposed to visit the training sets in a different random order every epoch. Two separate faults prevent it. Name both, and state what order the sets are actually visited in. *(3 marks)*

(b) Backpropagation is performed in the wrong order. State the correct order, state precisely what the hidden layer computes its error from as the code stands, and state what value it computes on the very first training set of the very first epoch. *(3 marks)*

(c) Each of the five calls in the inner loop returns `bool` (the network's own `feedforward()` and the four layer calls), and every one of those return values is discarded. State what the five calls report through that value, name the failure it was meant to catch, and state what this function returns when that failure occurs. *(2 marks)*

(d) The stub from Chapter 3 exposes three methods that are not part of `dense_layer::Interface`. Two of them are `feedforwardCount()` and `setOutput()`. State what each one makes testable.

Then state, for each of (a), (b) and (c) in turn, whether a test built on those two methods catches the fault described there, with the reason. Finally, name one thing you would have to add to the stub to catch the fault in (b). *(4 marks)*

Question 3 A network trained by hand — 14 MARKS

The network below has two inputs, two hidden nodes and one output node. Every node uses ReLU, and the learning rate is $LR = 0.1$.

Hidden layer: $b_1 = 0.3$, $b_2 = -0.2$, with $w_1 = 0.4$ and $w_2 = -0.5$ feeding node 1 from x_1 and x_2, and $w_3 = 0.2$ and $w_4 = 0.3$ feeding node 2 from x_1 and x_2.

Output layer: $b_3 = 0.1$, with $w_5 = 0.6$ from y_1 and $w_6 = -0.7$ from y_2.

Training set: $X_1X_2 = 1\,1$, $Y = 1$.

(a) Perform feedforward. Give the weighted sum s and the output y for all three nodes. *(3 marks)*

(b) Perform backpropagation. Give δ_3, e_3, δ_1, δ_2, e_1 and e_2. State why the output layer has to be done before the hidden layer. *(3 marks)*

(c) Perform optimization. Give all three updated biases and all six updated weights. *(4 marks)*

(d) Feed the same input forward again with the updated parameters. Give the new output and the new deviation, and state by how much the deviation shrank. *(2 marks)*

(e) The **original** parameters are now used to train on a different set: $X_1 X_2 = 0\ 0$, $Y = 0$. Show that node 2's bias and both of its weights leave this step completely unchanged, name the phenomenon, and name the activation function the book recommends as a countermeasure. *(2 marks)*

Question 4 The activation function, and the argument it is given 10 MARKS

(a) `ml::actFuncOutput()` and `ml::actFuncDelta()` each handle exactly three cases. Give the output and the derivative for all three, and state why `ActFunc::None` is the `default` case of the switch rather than an error case. *(3 marks)*

(b) `actFuncDelta()` must be given the weighted sum s, not the output y. A `Tanh` node has $s = 1.5$.

Compute the derivative the correct way and the derivative the wrong way, and give the ratio between them. Then show that for a `Relu` node the two ways agree at $s = 1.5$ and at $s = -0.8$, and state what that agreement costs a test suite. *(4 marks)*

(c) `ml::randomStartVal()` returns values in the range $[-1, 1]$. Explain how that interacts with ReLU to leave a hidden node unable to learn ever again, possibly before the first epoch, and state the two countermeasures the book names. *(3 marks)*

Question 5 The dense layer, written out 12 MARKS

(a) Write `Dense::feedforward()`. Include the input validation.

One member variable is written by this method, is needed later, and is returned by no getter on the interface. Name it and state what it is for. *(4 marks)*

(b) Write the hidden-layer overload of `backpropagate()`, the one declared as `Dense::backpropagate(const Interface& nextLayer)`.

The next layer's weight matrix is read as `nextLayer.weights()[j][i]`. State what j ranges over and what i ranges over, and state what the layer would compute if the two indices were swapped. *(4 marks)*

(c) Write `Dense::optimize()`.

Then state which vector `Shallow::train()` must pass as `input` when it optimizes the **output** layer, and state what would go wrong if it passed the network's own training input instead. (*4 marks*)

Question 6 Why an image is not just a long vector — 10 MARKS

(a) Give the three reasons the book sets out for why a network built only from dense layers handles images badly. (*3 marks*)

(b) A 1024 × 1024 greyscale image is flattened and fed into a single hidden layer of 1000 nodes.

Compute the number of trainable parameters in that one layer, and its memory footprint stored as `double`. Give the footprint in bytes and in GB. (*3 marks*)

(c) Consider a 64 × 64 feature map. A conv layer with one 3 × 3 kernel and same-size padding produces a 64 × 64 output from it. A dense layer producing the same 64 × 64 outputs from the same 64 × 64 inputs is the alternative.

Compute both parameter counts and the ratio between them.

Conv adds two constraints to a dense layer. Name both, then split the ratio you computed into the factor due to each. (*4 marks*)

Question 7 A convolutional layer, traced by hand — 14 MARKS

A conv layer takes a 4 × 4 input, has one 2 × 2 kernel, stride 1, zero-padding with `pad = kernelSize / 2`, and `ActFunc::Relu`. It is followed by a max pooling layer (pool size 2, stride 2) and a flatten layer. The learning rate is 0.01.

Input image (a hand-drawn 7):

```
1 1 1 1
0 0 0 1
0 0 1 0
0 1 0 0
```

Kernel:

```
0.5   1.0
1.0  -0.5
```

Bias: −0.2

(a) Give the padded input, then compute the conv layer's pre-activation matrix and its output. (*4 marks*)

(b) Give the max pooling layer's output and the flatten layer's output. For each pooled value, name the position in the conv output it came from.

Two of the four pooling blocks need the tie-breaking rule. Name them and state the rule. *(3 marks)*

(c) The dense layer sends back the gradients $[4, -2, 6, 8]$. Reshape them, route them back through the pooling layer, and give the 4×4 gradient matrix that arrives at the conv layer's output. *(3 marks)*

(d) Compute the bias gradient and all four kernel gradients, and give the updated kernel and bias.

One of the four gradients from (c) contributes nothing whatsoever to any of those five numbers. Identify it and give the reason. *(4 marks)*

Question 8 Max pooling, and the value it forgets 8 MARKS

A max pooling layer (pool size 2, stride 2) receives the following 4×4 feature map:

```
 0.6   0.2   0.3   0.3
 0.1   0.4   0.1  -0.5
 0.8   0.2  -0.1  -0.6
 0.5   0.9  -0.4  -0.3
```

(a) Give the layer's output and, for each value, the position it came from. One block contains a tie; state which, and which position wins. *(3 marks)*

(b) The gradients $\begin{bmatrix} 5 & -3 \\ 2 & 7 \end{bmatrix}$ arrive from the layer in front. Give the 4×4 gradient matrix the layer passes back, on the first call after construction. *(2 marks)*

(c) A student writes `double maxValue{0.0};` before the search, declares `maxRow` and `maxCol` with `{}` so that both start at 0, and ends the block with `myInputGradients[maxRow][maxCol] = outputGradients[i][j];`.

Give the 4×4 matrix this produces for the same data. Name the **two** distinct things that go wrong with it, and state why a test suite whose feature maps are all non-negative passes anyway. *(3 marks)*

Question 9 The finished network, and the target it has to run on 8 MARKS

A CNN is built as follows:

```
input (16x16) -> conv (one 5x5 kernel, stride 1, same padding, ReLU)
              -> max pooling (4x4, stride 4)
              -> flatten
              -> dense (4 nodes)
```

(a) Give the shape of the data after each of the four layers. *(2 marks)*

(b) Give the trainable-parameter count for each of the four layers and the total for the network. Two of the four contribute nothing; name them and state why. *(3 marks)*

(c) A different network contains a dense layer of 128 nodes over 256 inputs.

Compute its trainable-parameter count, then its footprint stored as `double`, as `float`, and as `int8`. State which of the three fits into a 256 KB flash budget, and show the comparison in bytes rather than in KB. *(3 marks)*

APPENDIX D

Self-Assessment Paper B

Time: 4 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix F.

Rubric

Sign convention. The book computes a deviation $\delta = y_{\text{ref}} - y_p$ and *adds* the resulting adjustment to every parameter. Every update in this paper therefore reads `parameter += gradient * learningRate`, with no minus sign. An answer that flips the sign consistently and says so is not penalized twice.

The activation derivative is evaluated at the weighted sum s, never at the output y. This matters in more than one question.

$$\sigma_{\text{ReLU}}(s) = \begin{cases} s & s > 0 \\ 0 & s \le 0 \end{cases} \qquad \sigma'_{\text{ReLU}}(s) = \begin{cases} 1 & s > 0 \\ 0 & s \le 0 \end{cases}$$

$$\sigma_{\text{Tanh}}(s) = \tanh(s) \qquad \sigma'_{\text{Tanh}}(s) = 1 - \tanh^2(s) \qquad \sigma_{\text{None}}(s) = s \qquad \sigma'_{\text{None}}(s) = 1$$

Convolutional layers zero-pad with `pad = kernelSize / 2` (integer division), so the output has the same size as the input.

Max pooling breaks ties toward the first occurrence in row-major order, and **flatten is row-major**.

Code answers may be written in C++17 or in unambiguous pseudocode. Nothing on this paper is marked on syntax; a missing semicolon costs nothing and a missing bounds check costs everything.

Show the arithmetic. Where a question asks you to compute, an unsupported number scores nothing, and a correct method with a slip in it scores most of the marks.

Supplied constants

Quantity	Value
$\tanh(1.5)$	0.905148
$\tanh(s')$, for the s' you compute in Question 5(b)	0.896887
$\tanh(0.905148)$	0.718795
2^{16}	65536
1 KB	1024 B

Marks

Question	1	2	3	4	5	6	7	8	9
Marks	10	10	14	12	12	14	12	8	8

Question 1 The model, and the rate that destroys it — 10 MARKS

(a) Traditional programming and machine learning both involve input, rules and output, and differ in which two are supplied and which one is produced. State the arrangement for each.

Then state the property of image data that makes the traditional arrangement impractical for image classification. *(2 marks)*

(b) A linear regression model starts at $k = 0$, $m = 0$ and is trained with $LR = 0.5$ on data following $y = 2x + 1$. Three optimization steps are performed, in this order:

$$(x, y_{\text{ref}}) = (3, 7), \quad (4, 9), \quad (3, 7)$$

Compute y_p, δ, Δe, k and m after each step, and state what happens to $|\delta|$ across the three. *(3 marks)*

(c) For a single training set (x, y_{ref}), let δ be the deviation before an optimization step and δ' the deviation for the same input immediately after it.

Derive δ' as a multiple of δ, in terms of LR and x. Hence give the condition on LR under which the deviation shrinks, and evaluate it at $x = 4$. *(3 marks)*

(d) `train()` returns `false` when the learning rate falls outside $(0.0, 1.0)$. Give the reason the book states for each end of that range.

Then use your answer to (c) to state why passing that check does not guarantee the model converges. *(2 marks)*

Question 2 Order, precision, and knowing when to stop 10 MARKS

(a) The training order is reshuffled at the start of every epoch, not once at construction. State what the model risks learning if it is not, and what the shuffle is therefore protecting. *(2 marks)*

(b) `initRandGen()` uses a `static` local variable. State what that variable is for, and what would happen without it.

In Chapter 2 the function first lived in an anonymous namespace inside `fixed.cpp`, and moved to a shared `ml/utils.cpp` once a second model needed it. State exactly what would have broken in the first arrangement once a second `.cpp` file needed the generator. *(3 marks)*

(c) The model's precision is computed as $1.0 - MAE$. A model predicts as follows:

x	y_{ref}	y_p
1	2	1.85
2	4	3.9
3	6	6.3
4	8	8.1

Compute the precision.

The precision threshold must lie strictly inside $(0.0, 1.0)$. Give the reason for each end separately. Then give a set of predictions for the same reference values that makes the precision negative, and state what that reveals about the metric. *(3 marks)*

(d) The precision is computed every tenth epoch rather than every epoch. State what that saves, and give the two prices paid for it. *(2 marks)*

Question 3 Two output nodes 14 MARKS

The network below has two inputs, two hidden nodes (y_1, y_2) and two output nodes (y_3, y_4). Every node uses ReLU, and the learning rate is $LR = 0.1$.

Hidden layer: $b_1 = 0.2$, $b_2 = 0.5$, with $w_1 = 0.3$ and $w_2 = 0.6$ feeding node 1 from x_1 and x_2, and $w_3 = -0.4$ and $w_4 = 0.2$ feeding node 2 from x_1 and x_2.

Output layer: $b_3 = -0.1$, $b_4 = 0.4$, with $w_5 = 0.7$ and $w_6 = 0.5$ feeding node 3 from y_1 and y_2, and $w_7 = -0.2$ and $w_8 = 0.9$ feeding node 4 from y_1 and y_2.

Training set: $X_1X_2 = 1\,0$, $Y_1Y_2 = 0\,1$.

(a) Perform feedforward. Give s and y for all four nodes. *(3 marks)*

(b) Perform backpropagation. Give δ and e for both output nodes and both hidden nodes. State how many terms enter each hidden node's δ, and why. *(4 marks)*

(c) Perform optimization. Give all four updated biases and all eight updated weights. (*4 marks*)

(d) Feed the same input forward again with the updated parameters. Give both new outputs and both new deviations, and state whether the step helped. (*2 marks*)

(e) Two of the eight weights come out of (c) with exactly the values they went in with. Name both and give the reason. (*1 mark*)

Question 4 The network class, and what a stub can prove — 12 MARKS

(a) `Shallow` stores its two layers as references, not by value. Give two consequences of that choice.

Then state what `predict()` returns and why the class needs no member variable to hold it. (*3 marks*)

(b) `train()` performs three steps per training set. Name them in order, and state the two separate ordering constraints that make that the only correct sequence. (*3 marks*)

(c) Among the methods the stub carries that `dense_layer::Interface` does not declare are `setOutput()` and `feedforwardCount()`. State what each one makes testable, and why nothing else in the interface can establish the same thing.

`feedforwardCount()` is incremented before the input size is checked, so a rejected call raises it too. State the argument for that choice. (*4 marks*)

(d) The constructor calls `std::terminate()` on bad arguments; `train()` returns `false` instead. State the rule the book applies and the reason behind it. (*2 marks*)

Question 5 The dense layer, and the mistake only Tanh can catch — 12 MARKS

(a) `Dense` has six member variables. Name each one, give its size, and give the quantity from the theory of Chapter 3 it corresponds to. (*3 marks*)

(b) A `Dense` layer with `ActFunc::Tanh` has one node and two weights:

$$b = 0.3, \quad w_0 = 0.6, \quad w_1 = 0.6$$

It is fed `input = {1.0, 1.0}` with `reference = {0.5}` and a learning rate of 0.2.

Perform feedforward, backpropagation and optimization: give s, y, δ, $\sigma'(s)$, Δe, the updated bias and both updated weights. Then feed the same input forward again and give the new deviation. (*5 marks*)

(c) The layer is now changed to compute `myError[i] = err * actFuncDelta(myActFunc, myOutput[i])`.

Recompute Δe for the node in (b) under that change and give the ratio between the wrong value and the right one.

State why the same mistake is invisible when the layer uses `ActFunc::Relu`, and state what the consequence of that is for how the mistake has to be tested. *(4 marks)*

Question 6 Padding, sharing, and the kernel's gradient — 14 MARKS

A conv layer takes a 4×4 input, has one 3×3 kernel, stride 1, zero-padding with `pad = kernelSize / 2`, and `ActFunc::None`. The learning rate is 0.01.

Input image:

```
0 1 1 0
1 1 1 1
1 1 1 1
0 1 1 0
```

Kernel:

```
 1   0  -1
 1   0  -1
 1   0  -1
```

Bias: 0.0

(a) Give the padded input and compute the layer's full 4×4 output.

State what feature this kernel responds to, and what the sign of an output value tells you. *(3 marks)*

(b) The same 4×4 input is now given to a layer with a 2×2 kernel instead.

Give the padded size in both cases. Then show that with the 2×2 kernel, one row and one column of the padded input are never read by `feedforward()` at all, and state what that implies about how the output lines up with the input. *(3 marks)*

(c) Returning to the 3×3 kernel, the following output gradients arrive:

```
0  2  0  0
0  0  0  4
6  0  0  0
0  0  8  0
```

Compute the bias gradient and the full 3×3 matrix of kernel gradients, then give the updated kernel and bias. *(4 marks)*

(d) Give the number of trainable parameters in this conv layer, and the number a dense layer producing the same 16 outputs from the same 16 inputs would have.

The book says the gradients "from all those positions must be accumulated before the kernel's weights are updated". Using your answer to (c), state how many separate output

positions contributed to the centre kernel gradient, and state why the layer nevertheless applies exactly one update to it. (*4 marks*)

Question 7 A pooling layer that runs and returns true

12 MARKS

`MaxPool::feedforward()` is correct: it scans each block, keeps a value only when it is strictly greater than the best so far, and therefore records the **first** occurrence of the maximum. `backpropagate()` is written as follows:

```
bool MaxPool::backpropagate(const Matrix2d& outputGradients) noexcept
{
    if (!ml::matchDimensions(outputGradients, myOutput)) { return false; }

    const auto poolSize{myInput.size() / myOutput.size()};

    for (std::size_t i{}; i < myOutput.size(); ++i)
    {
        for (std::size_t j{}; j < myOutput.size(); ++j)
        {
            auto maxValue{myInput[i * poolSize][j * poolSize]};
            std::size_t maxRow{i * poolSize};
            std::size_t maxCol{j * poolSize};

            for (std::size_t r{}; r < poolSize; ++r)
            {
                for (std::size_t c{}; c < poolSize; ++c)
                {
                    const auto value{myInput[i * poolSize + r][j * poolSize + c]};

                    if (value >= maxValue)
                    {
                        maxValue = value;
                        maxRow   = i * poolSize + r;
                        maxCol   = j * poolSize + c;
                    }
                }
            }
            myInputGradients[maxRow][maxCol] = outputGradients[i][j];
        }
    }
    return true;
}
```

(a) There are two independent faults in this method's gradient routing. Name both, and for each state exactly what the conv layer behind this one receives that it should not. (*4 marks*)

(b) The layer's most recent `feedforward()` input was:

```
 0.5   0.5  -0.1  -0.4
 0.2   0.3  -0.6  -0.9
 1.0   0.4   0.7   0.2
 0.1   0.9   0.3   0.8
```

The gradients $\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$ arrive, on the first call after construction, so `myInputGradients` still holds the zeros it was built with. Give the matrix this code produces and the matrix it should have produced, and name the block in which they differ. *(3 marks)*

(c) The same class implements `optimize()` as follows:

```
bool MaxPool::optimize(const double learningRate) noexcept
{
    if (!ml::checkLearningRate(learningRate)) { return false; }

    for (std::size_t i{}; i < myInput.size(); ++i)
    {
        for (std::size_t j{}; j < myInput.size(); ++j)
        {
            myInput[i][j] += myInputGradients[i][j] * learningRate;
        }
    }
    return true;
}
```

State what is wrong with this, give the correct body, and give the general rule about layers with no trainable parameters.

`flatten_layer::Interface` handles the same situation differently. State how, and state which of the two designs is the stronger one. *(3 marks)*

(d) Every fault above leaves a network that compiles, runs, and returns `true` from every call. State the symptom a user of the finished CNN actually sees. Then, for each of the three faults, name a test in the test suite of Chapter 10 that fails because of it, or state that none does. *(2 marks)*

Question 8 Every layer, side by side

8 MARKS

(a) Complete the table below for all five layer types built in this book. For each, give the trainable parameters, what `feedforward()` computes, what `backpropagate()` computes, and what `optimize()` does.

Layer	Trainable parameters	`feedforward()`	`backpropagate()`	`optimize()`
`Fixed`				
`Dense`				
`Conv`				
`MaxPool`				
`Flatten`				

(4 marks)

(b) "A conv layer is a dense layer with two constraints added." Name both constraints, and state in one sentence each what they buy and what they cost. *(2 marks)*

(c) A single training example is passed through `input -> Conv -> MaxPool -> Flatten -> Dense`.

Name the method called on each layer, in order, for all three phases of the training step, and state what each call is given as its argument. *(2 marks)*

Question 9 Off the laptop

8 MARKS

(a) A CNN is built as follows:

```
input (32x32) -> conv (one 3x3 kernel, same padding) -> max pooling (2x2, stride 2)
              -> flatten -> dense (64 nodes)
```

Compute the total trainable-parameter count, then the footprint stored as `double`, as `float`, and as `int8`.

The target has 64 KB of RAM available. State which of the three fits, and show the comparison in bytes. *(3 marks)*

(b) The same network is to run on a target without a hardware FPU, using Q16.16 fixed-point.

Convert 0.75 and -2.5 to Q16.16. Multiply them the way `fixedMultiply()` does, giving the product before and after the shift, and the value that represents. Show that the intermediate does not fit in a 32-bit integer.

Give the resolution of Q16.16. *(3 marks)*

(c) Name two constraints of resource-constrained targets other than memory footprint and the absence of an FPU.

Then compute the number of multiply-accumulate operations the conv layer in (a) performs per feedforward pass, and state why the *worst-case* execution time matters more than the average on such a target. (*2 marks*)

APPENDIX E

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct rule with an arithmetic slip in it is worth more than a correct number with no working, and later parts consume earlier answers, so an error should be followed through rather than penalized twice.

Code answers are marked on the algorithm, not on the syntax. A candidate who writes correct pseudocode loses nothing.

Question 1 Learning the rules instead of writing them 12 MARKS

(a) 3 MARKS

Traditional programming is given the **input** and the **rules**, and produces the **output**. The rules are written by hand: the ventilation example in §1.2 is a chain of `if/else if` on a temperature reading.

Machine learning is given the **input** and the **output**, and produces the **rules**. The same ventilation behaviour becomes a table of temperatures paired with fan settings, and the machine finds the relationship itself. (*1 mark for both arrangements*)

The paradigm is **symbolic AI**, the dominant approach before machine learning. (*1 mark*)

Where it worked and where it did not. It worked on complex but *well-defined* tasks: chess is the book's example, where the legal moves and the goal can be stated exactly. It failed on computer vision, speech recognition and language translation.

The property that separates them. How widely the input can vary. Chess has a finite, exactly describable set of positions; two photographs of the same object share almost no pixel values. A task whose input varies without bound cannot be covered by an enumeration of conditions, however long, because there is no finite enumeration to write. (*1 mark*)

(b) 4 MARKS

$y_p = kx + m$, $\delta = y_{\text{ref}} - y_p$, $\Delta e = \delta \cdot LR$, then $m \mathrel{+}= \Delta e$ and $k \mathrel{+}= \Delta e \cdot x$.

Set 1 ($x = 0$, $y_{\text{ref}} = -1$):

$$y_p = 0 \cdot 0 + 0 = 0, \quad \delta = -1 - 0 = -1, \quad \Delta e = -0.1$$
$$m = 0 + (-0.1) = -0.1, \qquad k = 0 + (-0.1)(0) = 0$$

Set 2 ($x = 1$, $y_{\text{ref}} = 3$):

$$y_p = 0 \cdot 1 + (-0.1) = -0.1, \quad \delta = 3 - (-0.1) = 3.1, \quad \Delta e = 0.31$$
$$m = -0.1 + 0.31 = 0.21, \qquad k = 0 + 0.31(1) = 0.31$$

Set 3 ($x = 2$, $y_{\text{ref}} = 7$):

$$y_p = 0.31(2) + 0.21 = 0.83, \quad \delta = 7 - 0.83 = 6.17, \quad \Delta e = 0.617$$
$$m = 0.21 + 0.617 = \mathbf{0.827}, \qquad k = 0.31 + 0.617(2) = \mathbf{1.544}$$

(*1 mark per set, 1 mark for the correct rules stated or clearly used, including k not moving on set 1 because $x = 0$.*)

(c) 2 MARKS

$$y_p(5) = 1.544(5) + 0.827 = \mathbf{8.547} \qquad \text{against } y_{\text{ref}} = 19$$

(*1 mark*)

Why this epoch achieved so much less. The deviations were -1, 3.1, 6.17: they **grew** in magnitude across the whole epoch, where the example in §1.3.3 saw its deviation turn over and start shrinking by the fourth set. Two things cause it.

The target slope is 4 rather than 2, so k has twice as far to travel and every prediction lags further behind. And the target bias is -1 while every deviation after the first is positive, so sets 2 and 3 drive m *upwards*, from -0.1 to $+0.827$, away from where it has to end up. The model can only reach $m = -1$ after k has overshot far enough for the deviations to turn negative, and one epoch of three sets is nowhere near enough for that. (*1 mark; credit any answer that quotes the growing δ sequence and identifies either the larger slope or the wrong-direction bias movement.*)

(d) 3 MARKS

The special case: `if (0 == input) { myBias = output; return; }`.

Set 1 ($x = 0$): m is assigned -1 directly. k is untouched, so $k = 0$. (*1 mark*)

Set 2 ($x = 1$): $y_p = 0(1) + (-1) = -1$, $\delta = 4$, $\Delta e = 0.4$, so $m = -0.6$ and $k = 0.4$.

Set 3 ($x = 2$): $y_p = 0.4(2) + (-0.6) = 0.2$, $\delta = 6.8$, $\Delta e = 0.68$, so $m = \mathbf{0.08}$ and $k = \mathbf{1.76}$.

(*1 mark*)

Against (b)'s $k = 1.544$, $m = 0.827$: the special case puts the bias at its target, -1, at once, so sets 2 and 3 start from a bias 0.9 lower than in (b). The lower bias makes both predictions

lower and both deviations larger, 4 and 6.8 against 3.1 and 6.17, so the weight is pushed further, to 1.76 against 1.544, and the bias is pulled back up by 1.08 rather than 0.927, ending at 0.08 instead of 0.827. Both models are still far from $(4, -1)$.

When it is a bad idea. When the training data is noisy, or contains more than one set at $x = 0$. The bias is a property of the whole data set; the special case snaps it onto a single sample and discards everything every other sample had contributed to it. With more than one $x = 0$ sample the final bias is simply whichever of them was visited last, and since Chapter 2 randomizes the order, the trained model then depends on the shuffle.

(1 mark; any one of: noise, duplicate $x = 0$ samples, order dependence)

Question 2 The training loop that trains nothing

12 MARKS

(a)

3 MARKS

Fault 1: `randomizeTrainOrder()` is called once, outside the epoch loop. It belongs at the top of the epoch body, so a new order is drawn each epoch. As written the order is drawn once and reused for the entire run. *(1 mark)*

Fault 2: the inner loop indexes with the counter, not with the order. It should read `myTrainInput[myTrainOrder[x]]` (and likewise for `myTrainOutput`). As written, `myTrainOrder` is used only for its `size()`; its contents are never read at all. *(1 mark)*

What actually happens. The sets are visited in strict storage order $0, 1, 2, \ldots$, the same every epoch. Fault 2 makes fault 1 invisible: shuffling a vector nobody reads changes nothing, so fixing only fault 1 changes nothing either. Both have to be fixed for the shuffle to have any effect. *(1 mark)*

(b)

3 MARKS

The correct order is output layer first, hidden layer second:

```
myOutputLayer.backpropagate(myTrainOutput[x]);
myHiddenLayer.backpropagate(myOutputLayer);
```

(1 mark)

What the hidden layer reads as written. `myHiddenLayer.backpropagate(myOutputLayer)` computes $\delta_i = \sum_j \Delta e_j \cdot w_{ji}$ from `myOutputLayer.error()` and `myOutputLayer.weights()`. Called first, it reads the error vector the output layer computed on the **previous training set**, against a different reference value, and from a different feedforward pass than the one that just ran. The blame is real, but it belongs to another example. *(1 mark)*

The very first set of the very first epoch. The output layer has never backpropagated, so its error vector still holds the zeros it was constructed with. Every hidden δ is therefore 0, every hidden Δe is 0, and `myHiddenLayer.optimize()` adds zero to every bias and every weight. The hidden layer's first update is a no-op. *(1 mark)*

(c) 2 MARKS

All five report **failure of a range check** through the return value: `false` when the dimensions do not match, and additionally, for `optimize()`, when the learning rate lies outside $(0.0, 1.0)$. The network's `feedforward()` has no check of its own; it passes on the result of the two layer checks it runs. *(1 mark)*

The failure they exist to catch is training data whose shape does not match the layers, namely a training input row that is not `weightCount()` wide, or a training output row that is not `nodeCount()` long. (The layer-to-layer mismatch is already caught: `Shallow`'s constructor terminates when the output layer's weight count does not equal the hidden layer's node count.)

With the values discarded the loop runs to completion regardless, the layers are left holding whatever they held before, and the function **returns `true`**, reporting a successful training run that never happened. *(1 mark)*

(d) 4 MARKS

`feedforwardCount()` makes the *number* of feedforward passes testable: one per training set per epoch. Nothing else can establish it. A loop that runs a single pass instead of every epoch lines up dimensionally and returns `true` exactly as a correct one does, so without a tally the two are indistinguishable.

`setOutput()` makes it testable that `predict()` reads the output layer **live** rather than returning a stored copy. The network holds its layers by reference, so changing a layer's output after the network was built must change what the network predicts; if it does not, the network kept a stale copy. *(1 mark for both)*

Fault (a): not caught. The count is one feedforward per set per epoch whatever order the sets are visited in, and neither method records the *arguments* a call was made with. A count cannot see an order. *(1 mark)*

Fault (b): not caught. `backpropagate()` is called the right number of times with the right arguments; only the sequence is wrong. Against the stub it is doubly invisible, because the stub's error stays at zero no matter when it is called. *(1 mark)*

Fault (c): not caught. With correctly sized training data every call returns `true`, so discarding the values changes nothing a passing test can observe. It only matters on data that is wrongly shaped, which a green test run does not supply. *(No mark of its own; credit it within the mark for faults (a) and (b) if a candidate answers all three.)*

What the stub would need to catch (b). A record of the *order* in which its methods were called, not just how often; a call log the test can read back, or a per-method sequence number, so a test can assert that the output layer's `backpropagate()` ran before the hidden layer's. Equivalently, letting the stub report the arguments it was handed would expose fault (a) as well. *(1 mark)*

Question 3 A network trained by hand

14 MARKS

(a)

3 MARKS

$x_1 = 1, x_2 = 1.$

$$s_1 = b_1 + x_1 w_1 + x_2 w_2 = 0.3 + 0.4 - 0.5 = 0.2 \Rightarrow y_1 = \sigma(0.2) = \mathbf{0.2}$$
$$s_2 = b_2 + x_1 w_3 + x_2 w_4 = -0.2 + 0.2 + 0.3 = 0.3 \Rightarrow y_2 = \sigma(0.3) = \mathbf{0.3}$$
$$s_3 = b_3 + y_1 w_5 + y_2 w_6 = 0.1 + 0.2(0.6) + 0.3(-0.7) = 0.1 + 0.12 - 0.21 = 0.01$$
$$y_3 = \sigma(0.01) = \mathbf{0.01}$$

(1 mark per node)

(b)

3 MARKS

$$\delta_3 = Y - y_3 = 1 - 0.01 = 0.99, \qquad e_3 = \delta_3 \cdot \sigma'(s_3) = 0.99 \cdot 1 = \mathbf{0.99}$$
$$\delta_1 = e_3 w_5 = 0.99(0.6) = 0.594, \qquad e_1 = 0.594 \cdot \sigma'(s_1) = 0.594 \cdot 1 = \mathbf{0.594}$$
$$\delta_2 = e_3 w_6 = 0.99(-0.7) = -0.693, \qquad e_2 = -0.693 \cdot \sigma'(s_2) = \mathbf{-0.693}$$

All three weighted sums are positive, so every ReLU derivative is 1. *(2 marks)*

Why the output layer first. A hidden node's deviation is a sum of the *next* layer's computed errors, weighted by the connections leaving it: $\delta_i = \sum_j e_j w_{ji}$. Those errors do not exist until the output layer has compared its own output against the reference value. Reversing the order does not fail; it silently uses whatever the output layer's error vector happened to hold from the previous example. *(1 mark)*

(c)

4 MARKS

Output layer: $\Delta c_3 = e_3 \cdot LR = 0.99(0.1) = 0.099.$

$$b_3 = 0.1 + 0.099 = \mathbf{0.199}$$
$$w_5 = 0.6 + 0.099(y_1) = 0.6 + 0.099(0.2) = \mathbf{0.6198}$$
$$w_6 = -0.7 + 0.099(y_2) = -0.7 + 0.099(0.3) = \mathbf{-0.6703}$$

(2 marks)

Hidden layer: $\Delta c_1 = 0.594(0.1) = 0.0594$, $\Delta c_2 = -0.693(0.1) = -0.0693.$

$$b_1 = 0.3 + 0.0594 = \mathbf{0.3594}, \qquad b_2 = -0.2 - 0.0693 = \mathbf{-0.2693}$$
$$w_1 = 0.4 + 0.0594(1) = \mathbf{0.4594}, \qquad w_3 = 0.2 - 0.0693(1) = \mathbf{0.1307}$$
$$w_2 = -0.5 + 0.0594(1) = \mathbf{-0.4406}, \qquad w_4 = 0.3 - 0.0693(1) = \mathbf{0.2307}$$

(2 marks)

Note that the weights are scaled by the *input to that layer*, y_1 and y_2 for the output layer, x_1 and x_2 for the hidden layer, while the biases are not. A candidate who used x for the output layer's weights has made the mistake Question 5(c) asks about.

(d) 2 MARKS

$$s_1 = 0.3594 + 0.4594 - 0.4406 = 0.3782 \Rightarrow y_1 = 0.3782$$
$$s_2 = -0.2693 + 0.1307 + 0.2307 = 0.0921 \Rightarrow y_2 = 0.0921$$
$$s_3 = 0.199 + 0.3782(0.6198) + 0.0921(-0.6703) = 0.199 + 0.234408 - 0.061735 = 0.371673$$
$$y_3 \approx \mathbf{0.3717}, \qquad \delta_3 = Y - y_3 = 1 - 0.3717 = \mathbf{0.6283}$$

(*1 mark*)

The deviation fell from 0.99 to 0.6283, a reduction of 0.3617, or about 37%. One step of one training set moved the output roughly a third of the way to its target, which is what a learning rate of 0.1 on a deviation this large looks like. (*1 mark*)

(e) 2 MARKS

With the **original** parameters and $x_1 = x_2 = 0$:

$$s_1 = 0.3 \Rightarrow y_1 = 0.3, \qquad s_2 = -0.2 \Rightarrow y_2 = \sigma(-0.2) = 0$$
$$s_3 = 0.1 + 0.3(0.6) + 0(-0.7) = 0.28 \Rightarrow y_3 = 0.28$$
$$\delta_3 = 0 - 0.28 = -0.28, \qquad e_3 = -0.28$$
$$\delta_2 = e_3 w_6 = -0.28(-0.7) = 0.196, \qquad e_2 = 0.196 \cdot \sigma'(s_2) = 0.196 \cdot 0 = \mathbf{0}$$

Therefore $\Delta c_2 = 0$, and b_2, w_3 and w_4 all keep their original values of -0.2, 0.2 and 0.3. (*1 mark*)

Note that the blame arriving at node 2 was not small, $\delta_2 = 0.196$; it was **annihilated** by the derivative. And the bias is the tell: w_3 and w_4 would have stayed put anyway, since both are multiplied by an input of zero, but b_2 has nothing to hide behind and it does not move either.

The phenomenon is the **dying ReLU** problem: once a node's weighted sum is negative, its derivative is exactly zero, its error is zero, its parameters stop updating, and nothing can drive the sum back up. The node is dead for the rest of training.

The countermeasure the book names is **Leaky ReLU**, which passes a weak signal below zero ($y = s \cdot \alpha$ for $s \leq 0$, with $\alpha \approx 0.01$) and so keeps the derivative non-zero everywhere. (*1 mark*)

Question 4 The activation function, and the argument it is given

10 MARKS

(a)

3 MARKS

`ActFunc`	`actFuncOutput(f, v)`	`actFuncDelta(f, v)`
`Relu`	`v` if `v > 0.0`, otherwise `0.0`	`1.0` if `v > 0.0`, else `0.0`
`Tanh`	`std::tanh(v)`	`1.0 - tanh(v) * tanh(v)`
`None`	`v` (identity)	`1.0`

(*2 marks*)

Why `None` is the `default` case. `ActFunc` has exactly three enumerators. `Relu` and `Tanh` are handled explicitly, so the only case that can reach `default` is `None`; there is no "cannot happen" branch to write, and inventing one would add an error path that is unreachable and therefore untestable. Putting `None` there also satisfies the compiler that the function returns on every path without a redundant trailing `return`. (*1 mark*)

(b)

4 MARKS

Tanh at $s = 1.5$.

Correct, the derivative evaluated at the activation's *input*:

$$\sigma'(s) = 1 - \tanh^2(1.5) = 1 - 0.905148^2 = 1 - 0.819293 = \mathbf{0.180707}$$

Wrong, the derivative evaluated at the activation's *output*, $y = \tanh(1.5) = 0.905148$:

$$1 - \tanh^2(0.905148) = 1 - 0.718795^2 = 1 - 0.516666 = \mathbf{0.483334}$$

$$\text{ratio} = \frac{0.483334}{0.180707} = \mathbf{2.675}$$

The wrong value is 2.7 times too large, so every parameter this node feeds moves 2.7 times too far on every step. The error grows with $|s|$: the true derivative collapses towards zero as the node saturates and the mistaken one does not, so the bug does its worst damage exactly where tanh is supposed to be applying the brakes. (*2 marks*)

ReLU. At $s = 1.5$: $y = 1.5$. From s: $1.5 > 0 \Rightarrow 1$. From y: $1.5 > 0 \Rightarrow 1$. At $s = -0.8$: $y = 0$. From s: $-0.8 > 0$ is false $\Rightarrow 0$. From y: $0 > 0$ is false $\Rightarrow 0$. The two agree in both cases.

They agree in *every* case, and not by luck: ReLU's derivative tests only the predicate "is this value positive", and $\max(0, s)$ is positive exactly when s is. (*1 mark*)

What that costs a test suite. A suite that only ever exercises `Relu` cannot detect the bug at all; every assertion passes, and the layer is wrong. Only a `Tanh` test separates the two, which is what `BackpropagateUsesPreActivationDerivative` in the test suite of Chapter 5 exists to do. The general lesson is that a test suite has to include the case that can *distinguish* two implementations, not merely a case each of them handles. (*1 mark*)

(c) 3 MARKS

`randomStartVal()` rescales `std::rand() / RAND_MAX` with $2x - 1$, so every bias and every weight starts somewhere in $[-1, 1]$, negative values included. A node can therefore start with a weighted sum that is negative for every input. For 2-bit XOR that takes $b < 0$ (the input `{0, 0}`), plus $b + w_1 < 0$, $b + w_2 < 0$ and $b + w_1 + w_2 < 0$, and nothing in the draw prevents it. Such a node is dead before the first epoch. A node that starts alive can also be driven there later, for instance by a learning rate large enough to overshoot. *(1 mark)*

Once some node's weighted sum is negative for **every** input in the training data, $\sigma'(s) = 0$ for that node on every set. Its Δe is zero, so its bias and weights never change again, and because they never change, the sum can never come back above zero. The node is permanently stuck, and nothing upstream or downstream can rescue it, because it also stops passing any blame backwards. *(1 mark)*

The countermeasures the book names:

- **Leaky ReLU**, which keeps a small non-zero derivative below zero, so a node driven negative can still recover.
- **`Tanh`**, whose derivative is non-zero everywhere. This is what the test suite of Chapter 5 trains with, precisely so the suite does not fail at random.

Also creditable: widening the hidden layer, so that losing a node is survivable. Chapter 5 records that with only two hidden nodes and ReLU, roughly six runs in seven fail to learn XOR at all, however long they train, and that about two networks in three contain a dead node before training has even begun. *(1 mark)*

Question 5 The dense layer, written out

12 MARKS

(a) 4 MARKS

```
bool Dense::feedforward(const Matrix1d& input) noexcept
{
    if (input.size() != weightCount())
    {
        std::fprintf(stderr, "Input dimension mismatch: expected %zu, actual: %zu!\n
    ↪ ",
            weightCount(), input.size());
        return false;
    }
    for (std::size_t i{}; i < nodeCount(); ++i)
    {
        auto sum{myBias[i]};

        for (std::size_t j{}; j < weightCount(); ++j)
```

```
        {
            sum += myWeights[i][j] * input[j];
        }
        myPreActOutput[i] = sum;
        myOutput[i]       = actFuncOutput(myActFunc, sum);
    }
    return true;
}
```

(*3 marks: 1 for the validation returning* `false` *without computing anything, 1 for the weighted sum starting at the bias, 1 for applying the activation function to it.*)

The member variable is `myPreActOutput`. It holds s, the weighted sum for each node *before* the activation function was applied. Both overloads of `backpropagate()` need it, because the activation's derivative has to be evaluated at the activation's input, not at its output (Question 4(b)). It is not on the interface because no other layer ever reads it: it is this layer's own bookkeeping, carried from the forward pass to the backward one. (*1 mark*)

(b) 4 MARKS

```
bool Dense::backpropagate(const Interface& nextLayer) noexcept
{
    if (nextLayer.weightCount() != nodeCount())
    {
        std::fprintf(stderr, "Layer dimension mismatch: expected %zu, actual: %zu!\n"
    ↪ ",
            nodeCount(), nextLayer.weightCount());
        return false;
    }
    for (std::size_t i{}; i < nodeCount(); ++i)
    {
        double err{};

        for (std::size_t j{}; j < nextLayer.nodeCount(); ++j)
        {
            err += nextLayer.error()[j] * nextLayer.weights()[j][i];
        }
        myError[i] = err * actFuncDelta(myActFunc, myPreActOutput[i]);
    }
    return true;
}
```

(*3 marks: 1 for the validation, 1 for the sum over the next layer's nodes, 1 for multiplying by the derivative at* `myPreActOutput[i]`.)

The indices. j ranges over the **next** layer's nodes, 0 … `nextLayer.nodeCount() - 1`. i ranges over **this** layer's nodes, 0 … `nodeCount() - 1`, and doubles as the weight index inside each next-layer node, which works precisely because the next layer has one weight per node in this layer, the condition the validation just checked. `weights()[j][i]` therefore reads "next-layer node j's weight on this layer's node i": the single connection along which node i's output travelled forward, and along which its share of the blame must travel back.

Swapped. `weights()[i][j]` reads a different entry entirely, the weight matrix transposed. If the two layers happen to be the same size it still compiles, still stays in bounds and still returns `true`, while handing every node the blame belonging to some other node; the network then trains on systematically misattributed gradients and simply never converges. If the sizes differ it indexes out of bounds. The silent version is the more dangerous of the two. *(1 mark)*

(c) 4 MARKS

```
bool Dense::optimize(const Matrix1d& input, const double learningRate) noexcept
{
    if ((0.0 >= learningRate) || (1.0 <= learningRate))
    {
        std::fprintf(stderr, "Invalid learning rate %g!\n", learningRate);
        return false;
    }
    if (input.size() != weightCount())
    {
        std::fprintf(stderr, "Input dimension mismatch: expected %zu, actual: %zu!\n
    ↪ ",
            weightCount(), input.size());
        return false;
    }
    for (std::size_t i{}; i < nodeCount(); ++i)
    {
        myBias[i] += myError[i] * learningRate;

        for (std::size_t j{}; j < weightCount(); ++j)
        {
            myWeights[i][j] += myError[i] * learningRate * input[j];
        }
    }
    return true;
}
```

(2 marks: 1 for both checks, 1 for the bias update carrying no `input[j]` factor while the weight update does.)

What `Shallow::train()` must pass to the output layer. `myHiddenLayer.output()`, the hidden layer's output vector, which is exactly what the output layer consumed during feedforward. (*1 mark*)

If it passed the network's training input instead. Usually nothing at all happens: the output layer has one weight per hidden node, not one per network input, so `input.size() != weightCount()` and the call returns `false` without touching a parameter. Training then stops with `false` if the return value is checked, and silently stops updating the output layer if it is not.

The dangerous case is a hidden layer that happens to have as many nodes as the network has inputs. The check then passes and every weight is scaled by the wrong number. w_{ij} is supposed to move in proportion to the value that flowed along it, and it would instead move in proportion to an unrelated one, so the update stops being the gradient with respect to that weight. The network fails to converge while every call reports success. (*1 mark*)

Question 6 Why an image is not just a long vector — 10 MARKS

(a) — 3 MARKS

1. Spatial structure is lost. An image is two- or three-dimensional, and has to be flattened into a one-dimensional vector before a dense layer can take it. Which pixels sat next to which is discarded, and no amount of training recovers it.

2. The parameter count explodes. Every pixel gets its own weight into every node, so the layer grows as (pixels × nodes). Part (b) puts a number on it.

3. Generalization gets worse. Each pixel is treated as a completely separate input, so the network has no way of knowing that the same pattern shifted a few pixels across is the same pattern. It needs far more training data and still transfers badly to new images.

(*1 mark each*)

(b) — 3 MARKS

$$1024 \times 1024 = 1{,}048{,}576 \text{ inputs}$$

$$\text{weights} = 1{,}048{,}576 \times 1000 = 1{,}048{,}576{,}000, \qquad \text{biases} = 1000$$

$$\text{total} = \mathbf{1{,}048{,}577{,}000} \text{ trainable values}$$

(*2 marks*)

As `double`, at 8 bytes each:

$$1{,}048{,}577{,}000 \times 8 = 8{,}388{,}616{,}000 \text{ bytes} \approx \mathbf{7.81\ GB}$$

(Using the paper's supplied 1 KB = 1024 B throughout, so 1 GB = 2^{30} B. A candidate who uses decimal GB and answers 8.39 has done the same work and should not lose the mark.)

And that is **one hidden layer**. A real network has several. (*1 mark*)

(c) 4 MARKS

Conv layer: one 3×3 kernel and one bias.

$$3 \times 3 + 1 = \mathbf{10}$$

Dense layer: $64 \times 64 = 4096$ inputs and 4096 outputs.

$$4096 \times 4096 + 4096 = 16{,}777{,}216 + 4096 = \mathbf{16{,}781{,}312}$$

$$\text{ratio} = \frac{16{,}781{,}312}{10} \approx \mathbf{1{,}678{,}131 : 1}$$

(2 marks)

The two constraints.

- **Locality.** Each output looks at a small window of the input rather than at all of it.
- **Weight sharing.** The same kernel is reused at every position, instead of each output/input pair owning its own weight.

(1 mark)

Splitting the ratio. Apply locality alone: each of the 4096 outputs keeps its own private 3×3 window of weights plus a bias.

$$4096 \times 9 + 4096 = 40{,}960 \qquad \frac{16{,}781{,}312}{40{,}960} \approx 410\times$$

Now add weight sharing: those 4096 private kernels collapse into one.

$$\frac{40{,}960}{10} = 4096 \times \qquad 410 \times 4096 \approx 1{,}678{,}000, \text{ the whole ratio.}$$

Weight sharing is the larger of the two factors, by an order of magnitude. Locality is what makes the layer look at neighbourhoods; sharing is what makes it cheap. *(1 mark)*

Question 7 A convolutional layer, traced by hand 14 MARKS

(a) 4 MARKS

pad $= 2/2 = 1$, so one zero on every side and the padded input is 6×6:

```
0 0 0 0 0 0
0 1 1 1 1 0
0 0 0 0 1 0
0 0 0 1 0 0
0 0 1 0 0 0
0 0 0 0 0 0
```

(1 mark)

Each output cell (i, j) is $s = b + \sum_{a,c} K[a][c] \cdot P[i+a][j+c]$, over the 2×2 window whose top-left corner is (i, j) in the padded matrix. Two worked cells:

$$s(0,0) = 0.5(0) + 1.0(0) + 1.0(0) - 0.5(1) - 0.2 = -0.70$$
$$s(1,1) = 0.5(1) + 1.0(1) + 1.0(0) - 0.5(0) - 0.2 = 1.30$$

Pre-activation:

```
-0.70   0.30   0.30   0.30
 0.80   1.30   1.30   0.80
-0.20  -0.20  -0.70   1.80
-0.20  -0.70   1.80   0.30
```

(*2 marks*)

Output (ReLU: every negative entry becomes 0):

```
0.00   0.30   0.30   0.30
0.80   1.30   1.30   0.80
0.00   0.00   0.00   1.80
0.00   0.00   1.80   0.30
```

(*1 mark*)

(b) 3 MARKS

Block	Values	Max	Position
top left	0.00, 0.30, 0.80, 1.30	1.30	(1, 1)
top right	0.30, 0.30, 1.30, 0.80	1.30	(1, 2)
bottom left	0.00, 0.00, 0.00, 0.00	0.00	(2, 0)
bottom right	0.00, 1.80, 1.80, 0.30	1.80	(2, 3)

Max pooling output:

```
1.3   1.3
0.0   1.8
```

Flatten output: [1.3, 1.3, 0.0, 1.8] (*2 marks*)

The tie blocks are the bottom-left and the bottom-right. The bottom-left block is four identical zeros, and the bottom-right block holds 1.80 twice, at $(2, 3)$ and $(3, 2)$.

The rule: on a tie, the first occurrence in row-major order wins. What matters is not which cell is chosen but that `feedforward()` and `backpropagate()` choose the *same* one. If they disagree the gradient is routed to a cell that did not produce the output, and the conv layer behind learns from the wrong pixel. (*1 mark*)

(c) 3 MARKS

Reshaped row-major into the pooling output's 2×2 shape:

$$\begin{bmatrix} 4 & -2 \\ 6 & 8 \end{bmatrix}$$

(1 mark)

Each gradient goes to the position its block's maximum came from, and every other position gets 0:

```
0   0   0   0
0   4  -2   0
6   0   0   8
0   0   0   0
```

(2 marks: 4 to (1,1), −2 to (1,2), 6 to (2,0), 8 to (2,3))

(d) 4 MARKS

The conv layer first multiplies each incoming gradient by $\sigma'(s)$ at that position, using the pre-activation matrix from (a):

Position	s	$\sigma'(s)$	**gradient in**	`outGrad`
$(1,1)$	1.30	1	4	4
$(1,2)$	1.30	1	−2	−2
$(2,0)$	−0.20	**0**	6	**0**
$(2,3)$	1.80	1	8	8

$$\text{biasGradient} = 4 - 2 + 0 + 8 = \mathbf{10}$$

(1 mark)

$$\text{kernelGradient}[a][b] = \sum_{i,j} P[i+a][j+b] \cdot \text{outGrad}(i,j)$$

Three positions contribute: $(1,1)$ with 4, $(1,2)$ with -2 and $(2,3)$ with 8. Each kernel entry therefore has three terms, one per contributing position, reading the padded input at the offset that entry sits at:

$$k_{00} = P[1][1](4) + P[1][2](-2) + P[2][3](8) = 1(4) + 1(-2) + 0(8) = \mathbf{2}$$
$$k_{01} = P[1][2](4) + P[1][3](-2) + P[2][4](8) = 1(4) + 1(-2) + 1(8) = \mathbf{10}$$
$$k_{10} = P[2][1](4) + P[2][2](-2) + P[3][3](8) = 0(4) + 0(-2) + 1(8) = \mathbf{8}$$
$$k_{11} = P[2][2](4) + P[2][3](-2) + P[3][4](8) = 0(4) + 0(-2) + 0(8) = \mathbf{0}$$

Kernel gradients:

```
2   10
8    0
```

(*2 marks*)

Updated parameters at $LR = 0.01$:

$$\text{bias} = -0.2 + 10(0.01) = \mathbf{-0.10}$$

```
0.52   1.10
1.08  -0.50
```

(*1 mark*)

The gradient that contributes nothing is the 6. Max pooling routed it to conv output position $(2, 0)$, whose pre-activation was -0.20. ReLU's derivative there is exactly zero, so `outGrad` at that position is $0 \times 6 = 0$, and the gradient is annihilated before it reaches the bias gradient, any kernel gradient, or the input gradients. It is worth seeing *why* that position was chosen at all: its whole pooling block had been flattened to zero by ReLU, and a block ReLU has flattened passes nothing back.

A candidate who instead answers "the -2, because k_{11} came out as 0" has found a different fact. k_{11} is zero because the corresponding image pixels are zero, not because a gradient died.

Question 8 Max pooling, and the value it forgets

8 MARKS

(a)

3 MARKS

Block	Values	Max	Position
top left	0.6, 0.2, 0.1, 0.4	0.6	$(0, 0)$
top right	0.3, 0.3, 0.1, -0.5	0.3	$(0, 2)$
bottom left	0.8, 0.2, 0.5, 0.9	0.9	$(3, 1)$
bottom right	-0.1, -0.6, -0.4, -0.3	-0.1	$(2, 2)$

Output:

```
0.6    0.3
0.9   -0.1
```

(*2 marks*)

The tie is in the top-right block, where 0.3 appears at both $(0, 2)$ and $(0, 3)$. The first occurrence in row-major order wins, so $(0, 2)$ is recorded. (*1 mark*)

(b) 2 MARKS

```
5    0   -3    0
0    0    0    0
0    0    7    0
0    2    0    0
```

(*2 marks*)

(c) 3 MARKS

With `maxValue` starting at 0.0 and `maxRow`/`maxCol` starting at 0:

- top left: 0.6 beats 0.0; $(0, 0)$ receives 5.
- top right: 0.3 beats 0.0; $(0, 2)$ receives -3.
- bottom left: 0.9 beats 0.0; $(3, 1)$ receives 2.
- bottom right: every value is negative, so **nothing ever beats 0.0**. `maxRow` and `maxCol` keep their initial value of 0, and 7 is written to $(0, 0)$.

```
7    0   -3    0
0    0    0    0
0    0    0    0
0    2    0    0
```

(*1 mark*)

The two distinct things that go wrong:

1. The bottom-right block's gradient never reaches the bottom-right block. Position $(2, 2)$ produced the pooled output and receives nothing, so the conv layer behind gets no gradient at all for that quarter of the image and never learns from it.

2. The gradient is written into a different block, destroying what was there. $(0, 0)$ belongs to the top-left block and had already, correctly, been given 5. The assignment (`=`, not `+=`) overwrites it with 7. So one gradient is lost and another is fabricated at a position that did not earn it, and the top-left block's legitimate gradient disappears as collateral damage. (*1 mark; both halves needed*)

Why an all-non-negative test suite passes. After a ReLU conv layer, every value in the feature map is ≥ 0, so every block contains at least one value that beats the initial 0.0, the search always finds a real maximum, and `maxRow`/`maxCol` are always assigned. The defect is invisible.

It only surfaces where the layer in front can produce negative values, which is exactly what happens once the conv layer uses `Tanh`. The fix is to initialize the search from the block's own first value rather than from 0.0. The same defect appears in `feedforward()`, where the test suite of Chapter 9 catches it with `FeedforwardHandlesNegativeValues`; the backward half needs the same all-negative input to be exposed. (*1 mark*)

Question 9 The finished network, and the target it has to run on

8 MARKS

(a) 2 MARKS

After	Shape
conv layer	16×16
max pooling	4×4
flatten	1×16
dense (4 nodes)	1×4

The conv layer preserves the size because it zero-pads; the pooling layer divides by the pool size, $16/4 = 4$; the flatten layer produces $4 \times 4 = 16$ values. (*2 marks*)

(b) 3 MARKS

Layer	Trainable parameters
conv	$5 \times 5 + 1 =$ **26**
max pooling	**0**
flatten	**0**
dense	$4 \times 16 + 4 =$ **68**
total	**94**

(*2 marks*)

The two that contribute nothing are max pooling and flatten, and both are purely structural. Max pooling selects a value that was already present and stores no weights of its own; flatten changes the shape of the data and computes nothing at all. Neither has anything an `optimize()` could adjust, which is why the book's `MaxPool::optimize()` is a genuine no-op rather than a placeholder, and why `flatten_layer::Interface` declares no `optimize()` in the first place. (*1 mark*)

(c) 3 MARKS

$$128 \times 256 + 128 = 32{,}768 + 128 = \textbf{32,896} \text{ values}$$

Type	Bytes each	Total footprint
`double`	8	263,168 B (exactly 257 KB)
`float`	4	131,584 B (128.5 KB)
`int8`	1	32,896 B (32.125 KB)

(*2 marks*)

The budget is $256 \times 1024 = 262{,}144$ bytes.

$$263{,}168 > 262{,}144 \Rightarrow \textbf{double does not fit}, \text{ by } 1024 \text{ bytes}$$
$$131{,}584 < 262{,}144 \Rightarrow \textbf{float fits}$$
$$32{,}896 < 262{,}144 \Rightarrow \textbf{int8 fits}$$

The comparison has to be made in bytes. "About 257 KB against a 256 KB budget" reads like a rounding question and is in fact a miss; it is a miss of exactly one kilobyte, which is the kind of margin that turns into a link failure rather than a warning. (*1 mark*)

APPENDIX F

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct rule with an arithmetic slip in it is worth more than a correct number with no working, and later parts consume earlier answers, so an error should be followed through rather than penalized twice.

Code answers are marked on the algorithm, not on the syntax. A candidate who writes correct pseudocode loses nothing. The questions are in Appendix D.

Question 1 The model, and the rate that destroys it — 10 MARKS

(a) — 2 MARKS

Traditional programming is given the **input** and the **rules** and produces the **output**. The rules are written by hand.

Machine learning is given the **input** and the **output** and produces the **rules**. It is handed pairs of examples and finds the relationship itself. (*1 mark*)

Why the traditional arrangement fails on images. The input varies without bound. Two photographs of the same object share almost no pixel values, so there is no finite set of conditions on pixel values that recognizes the object; a rule that covers one image fails on the next. Symbolic AI could handle chess, where every position can be described exactly, and could not handle vision, speech or translation, where the input cannot be enumerated. (*1 mark*)

(b) — 3 MARKS

$y_p = kx + m$, $\delta = y_{\text{ref}} - y_p$, $\Delta e = \delta \cdot LR$, then $m \mathrel{+}= \Delta e$ and $k \mathrel{+}= \Delta e \cdot x$, with $LR = 0.5$.

Step 1 ($x = 3$, $y_{\text{ref}} = 7$):

$$y_p = 0, \quad \delta = 7, \quad \Delta e = 3.5, \quad m = 3.5, \quad k = 0 + 3.5(3) = 10.5$$

Step 2 ($x = 4$, $y_{\text{ref}} = 9$):

$$y_p = 10.5(4) + 3.5 = 45.5, \quad \delta = 9 - 45.5 = -36.5, \quad \Delta e = -18.25$$
$$m = 3.5 - 18.25 = -14.75, \quad k = 10.5 - 18.25(4) = -62.5$$

Step 3 ($x = 3, y_{\text{ref}} = 7$):

$$y_p = -62.5(3) - 14.75 = -202.25, \quad \delta = 7 + 202.25 = 209.25, \quad \Delta e = 104.625$$
$$m = -14.75 + 104.625 = \mathbf{89.875}, \quad k = -62.5 + 104.625(3) = \mathbf{251.375}$$

(*1 mark per step*)

$|\delta|$ runs $7 \to 36.5 \to 209.25$: it grows by roughly a factor of 5 to 6 every step, and alternates in sign. This is not slow convergence, it is divergence. The parameters are further from $(2, 1)$ after three steps than they were at $(0, 0)$.

(c)

3 MARKS

Let $p = kx + m$ and $\delta = y_{\text{ref}} - p$. One optimization step gives

$$m' = m + \delta LR, \qquad k' = k + \delta LR\, x$$

so the new prediction for the **same** input is

$$p' = k'x + m' = (k + \delta LR\, x)x + m + \delta LR = p + \delta LR\,(1 + x^2)$$

and therefore

$$\delta' = y_{\text{ref}} - p' = \delta - \delta LR\,(1 + x^2) = \boxed{\delta\left[1 - LR(1 + x^2)\right]}$$

(*2 marks*)

The deviation shrinks when $|1 - LR(1 + x^2)| < 1$, that is when $0 < LR(1 + x^2) < 2$:

$$LR < \frac{2}{1 + x^2}$$

At $x = 4$: $LR < 2/17 = \mathbf{0.1176}$. (*1 mark*)

Note what the formula says: the bias contributes the 1 and the weight contributes the x^2, because the weight is adjusted by $\Delta e \cdot x$ and that adjustment is then multiplied by x again on the next prediction. The loop gain grows with the *square* of the input.

(d)

2 MARKS

The reasons the book states. A learning rate of 0.0 or less can never improve the model: zero makes every adjustment zero, and a negative rate moves every parameter away from the reference value. A learning rate of 1.0 or more corrects by at least the full error on every step, which makes training oscillate or diverge rather than converge. (*1 mark*)

Why passing the check guarantees nothing. The $LR < 1.0$ bound coincides with the bound from (c) only at $|x| = 1$, where $2/(1 + x^2) = 1$. At $x = 0$, where the weight does not move at all, the real bound is looser, $LR < 2$; as soon as $|x| > 1$ it is tighter, and it tightens quadratically. At $x = 4$ it is 0.1176; the $LR = 0.5$ used in (b) sails through the check in `train()` and diverges anyway, which is exactly what (b) shows happening.

The check is necessary, not sufficient. What actually bounds the safe learning rate is the scale of the input data, which `train()` never looks at. *(1 mark)*

Question 2 Order, precision, and knowing when to stop

10 MARKS

(a)

2 MARKS

Trained in a fixed order every epoch, the model risks fitting the **ordering** rather than the data: it sees the same sequence of corrections over and over and can settle into a pattern that reflects how the training sets happen to be stored, rather than the relationship between x and y. Local minima are not the issue here: the book notes that a linear model has none, since its squared-error surface has exactly one minimum. That benefit of shuffling arrives with the non-linear networks from Chapter 3 on, where it helps the parameters escape poor regions. *(1 mark)*

What the shuffle protects is the model's generalization: reshuffling every epoch exposes the model to the same data in varying order, so nothing in the storage order can be learned. Shuffling once at construction would not do it; one fixed order is still one fixed order. *(1 mark)*

(b)

3 MARKS

The `static` local (`initialized`) keeps its value between calls, so the body that seeds the generator runs on the first call and never again. *(1 mark)*

Without it, every call would re-seed. `std::srand(std::time(nullptr))` has one-second resolution, so several calls in the same second would reset the generator to the identical state and `std::rand()` would return the identical sequence each time; the shuffle would stop shuffling. *(1 mark)*

What would have broken in the first arrangement. An anonymous namespace gives every translation unit its own private copy of the function *and its own private* `initialized` *flag*. With two `.cpp` files that each need the generator, the seeding happens once per file rather than once per program: the second file's call re-seeds a generator the first file had already set up, and the "exactly once" guarantee is gone. Seeding has to happen once per **program**, which is what a single definition in `utils.cpp`, shared through `utils.hpp`, gives. *(1 mark)*

(c)

3 MARKS

$$|2 - 1.85| = 0.15, \quad |4 - 3.9| = 0.1, \quad |6 - 6.3| = 0.3, \quad |8 - 8.1| = 0.1$$

$$MAE = \frac{0.15 + 0.1 + 0.3 + 0.1}{4} = \frac{0.65}{4} = 0.1625$$

$$\text{precision} = 1.0 - 0.1625 = \mathbf{0.8375}$$

(1 mark)

Why the threshold must be strictly inside $(0.0, 1.0)$**.**

- **At** 1.0 **or above:** unreachable. Precision is $1 - MAE$ and the mean *absolute* error cannot be negative, so precision can never exceed 1.0 and reaches it only on an exactly perfect model. Training would always run the full epoch count and the early stop would never fire.
- **At** 0.0 **or below:** it accepts a model whose mean absolute error is 1.0 or worse, which defeats the point of checking.

(*1 mark*)

A negative precision. Predictions $3, 7, 1, 15$ against $2, 4, 6, 8$ give absolute errors $1, 3, 5, 7$, so $MAE = 4$ and precision $= 1 - 4 = \mathbf{-3}$.

What this reveals: **precision here is not a percentage and not a bounded score.** It is $1 - MAE$ measured in the *units of the output data*, so it depends entirely on the scale of y. On data whose outputs run into the thousands, even a very good model reports a large negative "precision". It is a stopping heuristic tuned to the book's small integer training data, not a general accuracy metric, and the name flatters it. (*1 mark*)

(d) 2 MARKS

What it saves. Computing the precision scans every training set and performs a prediction for each. Over thousands of epochs that is a full extra forward pass over the whole data set per epoch, which can be a large fraction of the total work when the training loop itself is cheap. (*1 mark*)

The two prices.

1. Up to nine epochs of unnecessary training after the threshold has actually been reached, since the check only fires on every tenth.
2. The reported epoch count is quantized to the epochs the check runs after, 1, 11, 21 and so on, and so is not the true one. This is why the example in Chapter 2 prints "reached after 11 epochs" for data that in fact reached the threshold somewhere in epochs 2 to 11.

(*1 mark*)

Question 3 Two output nodes 14 MARKS

(a) 3 MARKS

$x_1 = 1, x_2 = 0$.

$$
\begin{aligned}
s_1 &= 0.2 + 1(0.3) + 0(0.6) = 0.5 && \Rightarrow y_1 = \mathbf{0.5} \\
s_2 &= 0.5 + 1(-0.4) + 0(0.2) = 0.1 && \Rightarrow y_2 = \mathbf{0.1} \\
s_3 &= -0.1 + 0.5(0.7) + 0.1(0.5) = -0.1 + 0.35 + 0.05 = 0.3 && \Rightarrow y_3 = \mathbf{0.3} \\
s_4 &= 0.4 + 0.5(-0.2) + 0.1(0.9) = 0.4 - 0.1 + 0.09 = 0.39 && \Rightarrow y_4 = \mathbf{0.39}
\end{aligned}
$$

Every sum is positive, so ReLU passes all four through unchanged. *(3 marks)*

(b) 4 MARKS

Output layer:

$$\delta_3 = Y_1 - y_3 = 0 - 0.3 = -0.3, \qquad e_3 = -0.3 \cdot 1 = \mathbf{-0.3}$$
$$\delta_4 = Y_2 - y_4 = 1 - 0.39 = 0.61, \qquad e_4 = 0.61 \cdot 1 = \mathbf{0.61}$$

(2 marks)

Hidden layer: each hidden node feeds **both** output nodes, so its deviation is the sum of two terms, one per output node, each weighted by the connection from that hidden node to it:

$$\delta_1 = e_3 w_5 + e_4 w_7 = -0.3(0.7) + 0.61(-0.2) = -0.21 - 0.122 = -0.332 \Rightarrow e_1 = \mathbf{-0.332}$$
$$\delta_2 = e_3 w_6 + e_4 w_8 = -0.3(0.5) + 0.61(0.9) = -0.15 + 0.549 = 0.399 \Rightarrow e_2 = \mathbf{0.399}$$

(2 marks)

How many terms, and why. Two, one per node in the next layer. A hidden node's output was sent to every output node, so it is partly responsible for every output node's error, and its share along each path is that path's weight. In general $\delta_i = \sum_j e_j w_{ji}$ over all j nodes in the next layer. A candidate who used only one term has effectively disconnected one of the two output nodes.

Note δ_2: the two contributions have **opposite signs**, and the larger one wins. Node 2 is being pulled down by output node 3 and up harder by output node 4.

(c) 4 MARKS

Output layer. $\Delta c_3 = -0.3(0.1) = -0.03$, $\Delta c_4 = 0.61(0.1) = 0.061$. The weights are scaled by the output layer's input, which is $y_1 = 0.5$ and $y_2 = 0.1$:

$$b_3 = -0.1 - 0.03 = \mathbf{-0.13}, \qquad b_4 = 0.4 + 0.061 = \mathbf{0.461}$$
$$w_5 = 0.7 - 0.03(0.5) = \mathbf{0.685}, \qquad w_7 = -0.2 + 0.061(0.5) = \mathbf{-0.1695}$$
$$w_6 = 0.5 - 0.03(0.1) = \mathbf{0.497}, \qquad w_8 = 0.9 + 0.061(0.1) = \mathbf{0.9061}$$

(2 marks)

Hidden layer. $\Delta c_1 = -0.332(0.1) = -0.0332$, $\Delta c_2 = 0.399(0.1) = 0.0399$. The weights are scaled by the network's input, $x_1 = 1$ and $x_2 = 0$:

$$b_1 = 0.2 - 0.0332 = \mathbf{0.1668}, \qquad b_2 = 0.5 + 0.0399 = \mathbf{0.5399}$$
$$w_1 = 0.3 - 0.0332(1) = \mathbf{0.2668}, \qquad w_3 = -0.4 + 0.0399(1) = \mathbf{-0.3601}$$
$$w_2 = 0.6 - 0.0332(0) = \mathbf{0.6}, \qquad w_4 = 0.2 + 0.0399(0) = \mathbf{0.2}$$

(2 marks)

(d) 2 MARKS

$s_1 = 0.1668 + 0.2668 = 0.4336 \Rightarrow y_1 = 0.4336$
$s_2 = 0.5399 - 0.3601 = 0.1798 \Rightarrow y_2 = 0.1798$
$s_3 = -0.13 + 0.4336(0.685) + 0.1798(0.497) = -0.13 + 0.297016 + 0.089361 = \mathbf{0.256377}$
$s_4 = 0.461 + 0.4336(-0.1695) + 0.1798(0.9061) = 0.461 - 0.073495 + 0.162917 = \mathbf{0.550422}$
$\delta_3 = 0 - 0.256377 = \mathbf{-0.2564}$ (was -0.3)
$\delta_4 = 1 - 0.550422 = \mathbf{0.4496}$ (was 0.61)

Both deviations shrank in magnitude, so the step helped, and it helped node 4 substantially more than node 3, which is what you would expect from the much larger error node 4 started with. (*2 marks*)

(e) 1 MARK

w_2 and w_4 are unchanged, at 0.6 and 0.2. Both are the hidden layer's weights on input x_2, and $x_2 = 0$ for this training set. The weight update is $w \mathrel{+}= \Delta c \cdot \text{input}$, so a weight whose input is zero receives an adjustment of exactly zero however large its node's error was.

This is not the dying ReLU problem. Nothing is broken: a weight that had no influence on this prediction gets no share of the blame for it, which is correct. It will move on the next training set that has $x_2 = 1$. (*1 mark*)

Question 4 The network class, and what a stub can prove

12 MARKS

(a) 3 MARKS

Two consequences of storing the layers by reference:

1. **`Shallow` does not own its layers.** The caller constructs them and must keep them alive for at least as long as the network; a network outliving its layers holds dangling references. It also means the class has nothing to destroy, which is why its destructor is `default`.
2. **A layer can be replaced without touching `Shallow`.** This is precisely what Chapter 5 does: `Stub` is swapped for `Dense` and not a line of the network class changes. It also lets a *test* reach the layer after the network was built and manipulate it, which is what `setOutput()` exists for.

Also creditable: reference members make the class non-assignable, which is consistent with the book deleting the copy and move operators anyway. (*2 marks for any two*)

`predict()` returns `myOutputLayer.output()`, a reference to the output layer's own output vector. No member variable is needed because the output layer already stores it and the network holds that layer by reference: the value is always live. Keeping a copy would

introduce the possibility of it going stale, which is the exact failure `setOutput()` is designed to expose. (*1 mark*)

(b) 3 MARKS

The three steps, per training set: **feedforward**, then **backpropagation**, then **optimization**. (*1 mark*)

Constraint 1: between the steps. Backpropagation computes each node's error from the outputs feedforward just produced, so it cannot run first. Optimization adjusts parameters using the errors backpropagation just computed, so it cannot run before that. The three are a strict chain. (*1 mark*)

Constraint 2: inside backpropagation. The output layer must go first. The hidden layer's deviation is $\sum_j e_j w_{ji}$ over the output layer's *computed errors*, which do not exist until the output layer has compared its output against the reference value. (*1 mark*)

Worth noting what is *not* constrained: the two `optimize()` calls can be made in either order. Optimizing the hidden layer does not change `myHiddenLayer.output()`; only `feedforward()` writes that, so the output layer still receives the values it actually consumed.

(c) 4 MARKS

`setOutput()` sets every element of the stub's output to a chosen value. It makes it testable that `predict()` reads its output layer **live**: build a network, change the output layer's output, and the network's prediction must change with it. Nothing on `Interface` can establish this, because a network that cached a copy at construction would satisfy every interface method identically; it would return a plausible vector of the right size, and only an *external* change to the layer can tell the two apart. (*1 mark*)

`feedforwardCount()` tallies calls to `feedforward()`. It makes the **shape of the training loop** testable: `train()` must perform exactly one feedforward per training set per epoch, so the count after training must be *sets* × *epochs*. Nothing else can establish it. A loop that runs a single pass instead of every epoch lines up dimensionally, touches no invalid memory, and returns `true` exactly as a correct loop does; without a tally the two are indistinguishable. (*1 mark*)

Note that these two work because the stub *computes nothing*. A layer whose output is a fixed constant is what makes the network's wiring visible: any change in what the network predicts must have come from the wiring, because it cannot have come from the arithmetic. (*1 mark*)

Why the counter is incremented before the size check. The tally then records how often the layer was **asked** to feed forward, not how often it agreed to. That is the quantity a test of the training loop actually wants: the loop's job is to make the right number of calls, and whether each call was accepted is a separate question, tested separately by the return value. Incrementing after the check would conflate the two, and a loop that made the right calls with wrongly sized data would under-report rather than fail. (*1 mark*)

(d) 2 MARKS

The rule: `std::terminate()` is reserved for the constructor. Everything else reports failure through its return value. *(1 mark)*

The reason: a constructor has no return value, so it has no way to tell the caller that construction failed. A `Shallow` built with no training data, or with an output layer whose weight count does not match the hidden layer's node count, can never do anything useful, and letting it exist only defers the failure to somewhere less informative. `train()` does have a return value, so it can hand the decision back to the caller, who may reasonably want to retry, with a different epoch count or threshold or simply again, rather than have the program killed; `train()` resets both layers first, so every retry starts from freshly drawn parameters. *(1 mark)*

Question 5 The dense layer, and the mistake only Tanh can catch

12 MARKS

(a) 3 MARKS

Member	Size	Quantity from Chapter 3
`myOutput`	`nodeCount()`	y, each node's output, after the activation
`myPreActOutput`	`nodeCount()`	s, each node's weighted sum, before it
`myError`	`nodeCount()`	Δe, each node's computed error
`myBias`	`nodeCount()`	b, each node's bias
`myWeights`	`nodeCount()` × `weightCount()`	w, each node's weights
`myActFunc`	none	σ, the layer's activation function

(3 marks; deduct for a missing `myPreActOutput`, which is the one candidates forget and the one part (c) is about.)

(b) 5 MARKS

Feedforward.

$$s = b + w_0 x_0 + w_1 x_1 = 0.3 + 0.6(1.0) + 0.6(1.0) = \mathbf{1.5}$$
$$y = \tanh(1.5) = \mathbf{0.905148}$$

(1 mark)

Backpropagation.

$$\delta = y_{\text{ref}} - y = 0.5 - 0.905148 = \mathbf{-0.405148}$$
$$\sigma'(s) = 1 - \tanh^2(1.5) = 1 - 0.905148^2 = 1 - 0.819293 = \mathbf{0.180707}$$
$$\Delta e = \delta \cdot \sigma'(s) = -0.405148 \times 0.180707 = \mathbf{-0.073213}$$

(*2 marks*)

Optimization. $\Delta e \cdot LR = -0.073213 \times 0.2 = -0.014643$.

$$b = 0.3 - 0.014643 = \mathbf{0.285357}$$
$$w_0 = 0.6 - 0.014643(1.0) = \mathbf{0.585357}, \qquad w_1 = 0.6 - 0.014643(1.0) = \mathbf{0.585357}$$

Both weights move by the same amount because both inputs are 1.0. (*1 mark*)

Verification.

$$s = 0.285357 + 0.585357 + 0.585357 = 1.456072 \Rightarrow y = \tanh(1.456072) = 0.896887$$
$$\delta = 0.5 - 0.896887 = \mathbf{-0.396887} \quad (\text{was } -0.405148)$$

(*1 mark*)

The deviation moved by only 0.008 on an error of 0.41, about 2%. This is tanh saturation doing exactly what it is supposed to: at $s = 1.5$ the derivative is already down to 0.18, so a large error produces a small step. A candidate who observes that this node will take many epochs to move, and that this is a feature rather than a fault, has understood the question.

(c) 4 MARKS

Passing the **output** instead of the weighted sum:

$$\sigma'(y) = 1 - \tanh^2(0.905148) = 1 - 0.718795^2 = 1 - 0.516666 = \mathbf{0.483334}$$
$$\Delta e_{\text{wrong}} = -0.405148 \times 0.483334 = \mathbf{-0.195822}$$
$$\text{ratio} = \frac{-0.195822}{-0.073213} = \mathbf{2.675}$$

(*2 marks*)

The step is 2.7 times too large. And the error is worst where it matters most: as $|s|$ grows the true derivative collapses toward zero while the mistaken one flattens out at a much larger value, so the bug removes precisely the braking that tanh saturation is there to provide.

Why ReLU hides it. ReLU's derivative tests only the predicate "is this value positive", and $\max(0, s)$ is positive exactly when s is. So `actFuncDelta(Relu, s)` and `actFuncDelta(Relu, y)` return the same number for every s, not merely for the values that happen to be tested. There is no input that distinguishes them. (*1 mark*)

The consequence for testing. A suite that exercises only `Relu` cannot detect the bug at all; every assertion passes and the layer is wrong. The test has to be written against `Tanh` specifically, which is what `BackpropagateUsesPreActivationDerivative` in the test suite of Chapter 5 does. The general point: a test has to include a case that *distinguishes* the correct implementation from the plausible wrong one, not merely a case both of them survive.

(*1 mark*)

Question 6 Padding, sharing, and the kernel's gradient

14 MARKS

(a)

3 MARKS

pad $= 3/2 = 1$, so the padded input is 6×6:

```
0 0 0 0 0 0
0 0 1 1 0 0
0 1 1 1 1 0
0 1 1 1 1 0
0 0 1 1 0 0
0 0 0 0 0 0
```

(1 mark)

With `ActFunc::None` and a bias of 0, the output is the raw weighted sum. The kernel's middle column is all zeros, so each cell is (sum of the window's left column) minus (sum of its right column):

$$s(0,0) = (0+0+0) - (0+1+1) = -2, \qquad s(0,3) = (0+1+1) - (0+0+0) = 2$$

Output:

```
-2  -1   1   2
-3  -1   1   3
-3  -1   1   3
-2  -1   1   2
```

(1 mark)

What the kernel responds to. It is a **vertical edge detector**. It compares the three pixels down the left of the window against the three down the right and ignores the middle column entirely, so it reacts to a left-right change in brightness and not at all to a top-bottom one.

What the sign means. Positive means brighter on the left of the window than on the right; negative means the reverse. Zero means the neighbourhood is left-right symmetric, which is why the output above is antisymmetric across the image: this input is a symmetric blob, so the left half gives negative responses and the right half gives the mirrored positive ones. The magnitude is the strength of the edge. *(1 mark)*

(b)

3 MARKS

Both cases pad to 6×6: pad $= 3/2 = 1$ and pad $= 2/2 = 1$ are the same number, so the padded input is $4 + 2(1) = 6$ either way. *(1 mark)*

With the 3×3 **kernel**, output cell (i, j) reads padded rows $i \ldots i+2$. The largest output index is 3, so the largest padded index read is $3 + 2 = 5$, the last row and column. Everything is used.

With the 2×2 kernel, output cell (i, j) reads padded rows $i \dots i+1$. The largest padded index read is $3 + 1 = 4$. **Padded row 5 and padded column 5 are never touched by `feedforward()` at all**, and correspondingly never receive anything in `backpropagate()`.

(1 mark)

Why, and what it implies. A same-size output needs $kernelSize - 1$ extra rows in total. For an odd kernel that is an even number and splits evenly, one on each side. For a 2×2 kernel it is 1, but `pad = kernelSize / 2` adds 1 on *each* side, which is one too many; the loop starts at $(0, 0)$ so the surplus lands at the bottom and the right and is dead storage.

The alignment consequence is the real point. The padding is effectively one row and column *before* the image and none after, so output cell (i, j) is computed from the window covering input cells $(i-1, j-1)$ to (i, j). There is no centre cell for an even kernel to sit on, so the output is not centred on the input: a feature at input position (r, c) reaches outputs (r, c) to $(r+1, c+1)$, so it appears in the output shifted down and right by half a pixel. An odd kernel has a true centre and does not suffer this. *(1 mark)*

(c) 4 MARKS

With `ActFunc::None` the derivative is 1 everywhere, so `outGrad` equals the incoming gradient at every position and no gradient is lost.

$$\text{biasGradient} = 2 + 4 + 6 + 8 = \mathbf{20}$$

(1 mark)

$$\text{kernelGradient}[a][b] = \sum_{i,j} P[i+a][j+b] \cdot g[i][j]$$

Only four gradients are non-zero: $g(0,1) = 2$, $g(1,3) = 4$, $g(2,0) = 6$, $g(3,2) = 8$. Each kernel entry therefore has four terms. The centre entry, as a worked example:

$$k_{11} = P[1][2](2) + P[2][4](4) + P[3][1](6) + P[4][3](8) = 1(2) + 1(4) + 1(6) + 1(8) = 20$$

Kernel gradients:

```
12   14   14
12   20    8
 6    6    8
```

(2 marks)

Updated parameters at $LR = 0.01$:

$$\text{bias} = 0.0 + 20(0.01) = \mathbf{0.2}$$

```
1.12   0.14  -0.86
1.12   0.20  -0.92
```

```
1.06   0.06  -0.92
```

(*1 mark*)

(d) 4 MARKS

Conv layer: $3 \times 3 + 1 = \mathbf{10}$ trainable parameters.

Dense layer producing 16 outputs from 16 inputs: $16 \times 16 + 16 = \mathbf{272}$. (*1 mark*)

How many positions contributed to the centre kernel gradient. Four, the four output positions with a non-zero gradient, $(0,1)$, $(1,3)$, $(2,0)$ and $(3,2)$. In principle all **sixteen** output positions contribute; twelve of them contributed a term of zero here only because their gradients were zero. (*1 mark*)

Why one update nevertheless. The kernel is a *single* set of nine numbers used at every one of the sixteen output positions. Each position therefore gives a partial derivative with respect to the same nine parameters, and the derivative of the total error with respect to one parameter is the **sum** of those partials. Accumulating first and applying once is not an optimization; it *is* the gradient. (*1 mark*)

Applying an update per position instead would be a different algorithm and a wrong one: each of the sixteen updates after the first would be computed against a kernel that the previous updates had already moved, so the sum would no longer be the gradient at the point where it was evaluated. This is the one place where a conv layer genuinely differs from a dense layer in its backward pass, and it is a direct consequence of weight sharing: a parameter used in many places accumulates blame from all of them. (*1 mark*)

Question 7 A pooling layer that runs and returns true 12 MARKS

(a) 4 MARKS

Fault 1: `>=` where `feedforward()` uses `>`. `feedforward()` keeps a value only when it is strictly greater than the best so far, so on a tie it records the **first** occurrence. `backpropagate()` accepts equality, so it walks past the first occurrence and records the **last**. On any block with a tied maximum the two halves disagree.

What the conv layer behind receives: a gradient at a position that did **not** produce the pooled output, and nothing at the position that did. The gradient is real and lands on the wrong pixel, so the conv layer updates its kernel from a neighbourhood the prediction never used. (*2 marks*)

Fault 2: `myInputGradients` is never reset to zero. Each call writes exactly one cell per block and leaves every other cell untouched, so those cells retain whatever the *previous* call left in them.

What the conv layer behind receives: this example's gradients mixed with stale gradients at up to $poolSize^2 - 1$ positions per block, each left there by whichever earlier training example

last wrote that cell, possibly many examples ago. The layer never sees a clean gradient matrix after the first training set. (*2 marks*)

(b)

3 MARKS

Block	Values (position)	First max	Last max
top left	0.5 (0,0), 0.5 (0,1), 0.2 (1,0), 0.3 (1,1)	(0, 0)	(0, 1)
top right	−0.1 (0,2), −0.4 (0,3), −0.6 (1,2), −0.9 (1,3)	(0, 2)	(0, 2)
bottom left	1.0 (2,0), 0.4 (2,1), 0.1 (3,0), 0.9 (3,1)	(2, 0)	(2, 0)
bottom right	0.7 (2,2), 0.2 (2,3), 0.3 (3,2), 0.8 (3,3)	(3, 3)	(3, 3)

What the code produces (`myInputGradients` holding all zeros beforehand, as it does on the first call after construction):

```
 0   10   20    0
 0    0    0    0
30    0    0    0
 0    0    0   40
```

What it should have produced:

```
10    0   20    0
 0    0    0    0
30    0    0    0
 0    0    0   40
```

(*2 marks*)

They differ in the top-left block, the only one with a tied maximum: the gradient 10 lands on $(0, 1)$ instead of $(0, 0)$. Note how small the visible difference is, one value one column across, and that the layer returns `true` either way. (*1 mark*)

(c)

3 MARKS

What is wrong. `MaxPool` has **no trainable parameters**, and `myInput` is not one. It is the last feedforward input, stored only so `backpropagate()` can re-locate which cell held each block's maximum, and it is overwritten wholesale by the next `feedforward()`. This method therefore adjusts something that is never learned and never read again: it burns $O(n^2)$ work per training example to corrupt the layer's own record of what it was last shown. (*1 mark*)

The correct body:

```
bool MaxPool::optimize(const double learningRate) noexcept
{
    (void)(learningRate);
    return true;
}
```

The general rule: a layer with no trainable parameters has an `optimize()` that is a **genuine no-op**, not a placeholder awaiting an implementation. There is nothing to update, and inventing something to update is worse than doing nothing. (*1 mark*)

How `flatten_layer::Interface` differs, and which is stronger. It declares **no `optimize()` at all**, not an empty one, not a defaulted one. The interface simply does not ask for it.

That is the stronger design, because it moves "this layer has nothing to optimize" from a convention into the type system: a `Flatten` that tried to optimize something would not compile, whereas `MaxPool` relies on whoever writes it remembering that the method must do nothing, exactly the memory lapse this listing shows. The book keeps `optimize()` on `conv_layer::Interface` only because `Conv` and `MaxPool` share that interface and `Conv` genuinely needs it; the no-op is the price of the shared interface, not a preference. (*1 mark*)

(d) 2 MARKS

The symptom. Almost nothing, on the surface. The network compiles, runs, predicts, and every call returns `true`; there is no error message anywhere. Most training runs still end up classifying all four digits, because the dense layer does most of the classifying and the conv layer behind the pooling layer is only partly misled. What the user sees is a network that now and then fails to learn: measured on the digits 0 to 3, a `MaxPool` with all three faults leaves roughly one training run in seven unable to tell all four digits apart, where the correct layer never fails. Retraining usually fixes it, which is exactly what makes the fault easy to ship. (*1 mark*)

The tests. The test suite of Chapter 10 carries the `MaxPool` unit tests of Chapter 9 over unchanged, and they are what catches all three:

- **Fault (a) 1, `>=`:** `BackpropagateRoutesToFirstMaxOnTies` fails, and so does `BackpropagateMatchesHandTrainedExample`, since the conv output from the hand-training exercise of Chapter 6 holds 1.9 three times in its top-right block.
- **Fault (c), the invented `optimize()`:** `OptimizeAcceptsAnyLearningRate` fails, because this `optimize()` validates a learning rate it has no use for and returns `false` for the zero, negative and out-of-range rates the test passes.
- **Fault (a) 2, the missing reset:** `BackpropagateResetsGradientsBetweenCalls` fails. It feeds two inputs whose maxima sit in different positions and backpropagates after each, so the gradients the first call wrote are still there after the second, where the correct layer has zeros. It is the only test that fails every time: `LearnsToRecognizeAllFourDigits`, the component test that trains on the digits 0 to 3 and checks the right output node wins for each, catches this fault only on some runs, roughly one in five. A test that backpropagated the same input twice would miss it entirely, since an assignment that writes the same cells twice looks clean.

(*1 mark for a unit test that fails for each of the three faults.*)

Question 8 Every layer, side by side

8 MARKS

(a)

4 MARKS

Layer	Trainable parameters	`feedforward()`	`backpropagate()`	`optimize()`
`Fixed`	weight k, bias m	$y = kx + m$	$\delta = y_{\text{ref}} - y_p$	$k \mathrel{+}= \delta \cdot LR \cdot x$, $m \mathrel{+}= \delta \cdot LR$
`Dense`	one bias and `weightCount()` weights per node	$y = \sigma(b + \sum w_i x_i)$ per node	$\Delta e = \delta \cdot \sigma'(s)$ per node, δ from the reference values or from the next layer	$b \mathrel{+}= \Delta e \cdot LR$, $w \mathrel{+}= \Delta e \cdot LR \cdot input$
`Conv`	one shared kernel and one bias	as `Dense`, but local (one small window per output) and shared (the same kernel everywhere)	gradients accumulated across every position the kernel visited, plus input gradients for the layer behind	$kernel \mathrel{+}= grad \cdot LR$, $bias \mathrel{+}= grad \cdot LR$
`MaxPool`	none	passes the largest value of each block forward	routes the incoming gradient to the position that held that maximum; every other position gets 0	no-op: nothing to update
`Flatten`	none	reshapes 2D to 1D, row-major	reshapes 1D back to 2D, the exact inverse	does not exist in the interface

(*4 marks; roughly one per row beyond the first, with the* `Fixed` *row treated as free.*)

(b)

2 MARKS

Locality. Each output looks only at a small window of the input rather than at all of it.

Buys: the parameter count stops scaling with the input size, and the spatial structure of the image is preserved rather than flattened away. *Costs:* a single layer cannot relate two distant parts of the image at all; reach has to be built up by stacking layers, or by pooling between them to shrink the image under a fixed window. (*1 mark*)

Weight sharing. The same kernel is reused at every position instead of each output owning its own weights.

Buys: the parameter count becomes independent of the image size entirely, and a pattern is recognized wherever in the image it appears, which is what makes the layer generalize from far less training data. *Costs:* the layer *cannot* learn anything position-specific; it is forced

to treat every location identically. That is right for images and wrong for data where position itself carries meaning. (*1 mark*)

(c) 2 MARKS

Feedforward, left to right, each layer given the previous layer's `output()`:

```
Conv::feedforward(input)
MaxPool::feedforward(conv.output())
Flatten::feedforward(maxPool.output())
Dense::feedforward(flatten.output())
```

Backpropagation, right to left. The target vector goes straight into the dense layer; every earlier layer is given the next layer's `inputGradients()`:

```
Dense::backpropagate(target)
Flatten::backpropagate(dense.inputGradients())
MaxPool::backpropagate(flatten.inputGradients())
Conv::backpropagate(maxPool.inputGradients())
```

(*1 mark for both, including the target going into* `Dense` *and* `inputGradients()` *carrying the rest.*)

Optimization, using gradients `backpropagate()` has already computed:

```
Conv::optimize(learningRate)
MaxPool::optimize(learningRate)      // no-op
Dense::optimize(flatten.output(), learningRate)
Flatten                              // no optimize() exists
```

The dense layer needs the previous layer's `output()` as well as the learning rate, because its weight update is scaled by the value that actually flowed along each weight. The order of the `optimize()` calls does not matter: all the gradients already exist, and no layer's optimization changes any other layer's stored output. (*1 mark*)

Question 9 Off the laptop

8 MARKS

(a)

3 MARKS

Shapes: conv preserves 32×32 (it pads); pooling halves it to 16×16; flatten gives $16 \times 16 = 256$ values; the dense layer has 64 nodes over those 256 inputs.

Layer	Trainable parameters
conv	$3 \times 3 + 1 = 10$
max pooling	0
flatten	0
dense	$64 \times 256 + 64 = 16{,}448$
total	**16,458**

(*1 mark*)

Type	Bytes each	Total footprint
`double`	8	131,664 B (128.58 KB)
`float`	4	65,832 B (64.29 KB)
`int8`	1	16,458 B (16.07 KB)

(*1 mark*)

The budget is $64 \times 1024 = 65{,}536$ bytes.

$$131{,}664 > 65{,}536 \Rightarrow \textbf{double does not fit}$$
$$65{,}832 > 65{,}536 \Rightarrow \textbf{float does not fit either, by 296 bytes}$$
$$16{,}458 < 65{,}536 \Rightarrow \textbf{only int8 fits}$$

The `float` case is the one that has to be done in bytes. "About 64 KB against a 64 KB budget" reads like a fit and is a miss, by 296 bytes; that is before a single byte of program code, stack, or input buffer. This is the argument for post-training quantization in one line: train at full precision on a workstation, deploy at `int8`. (*1 mark*)

(b)

3 MARKS

Q16.16 scales by $2^{16} = 65536$.

$$\texttt{toFixed(0.75)} = 0.75 \times 65536 = \textbf{49,152}$$
$$\texttt{toFixed(-2.5)} = -2.5 \times 65536 = \textbf{-163,840}$$

(*1 mark*)

$$49{,}152 \times (-163{,}840) = \textbf{-8,053,063,680}$$

A 32-bit signed integer holds −2,147,483,648 to 2,147,483,647. The product exceeds that in magnitude by nearly a factor of four, so it must be formed in `std::int64_t` before being scaled back down, which is precisely what the cast in `fixedMultiply()` is for. Two values each scaled by 2^{16} multiply to a result scaled by 2^{32}, so the operands can each be small and the intermediate still overflow.

$$-8{,}053{,}063{,}680 \gg 16 = -122{,}880 \qquad \texttt{toDouble}(-122{,}880) = \frac{-122{,}880}{65{,}536} = \mathbf{-1.875}$$

and $0.75 \times (-2.5) = -1.875$, exactly. *(1 mark)*

$$\text{resolution} = \frac{1}{65536} \approx \mathbf{0.0000153}$$

with a representable range of roughly $\pm 32{,}768$. That is the trade: fixed dynamic range and fixed absolute precision, in exchange for arithmetic that is a single integer instruction on a core with no FPU rather than a software-emulated floating-point call. *(1 mark)*

(c) 2 MARKS

Two other constraints, any two:

- **Dynamic memory allocation.** Every layer in the book is built on `std::vector`, which allocates on the heap. Long-running embedded systems risk heap fragmentation, allocation is not constant-time, and some bare-metal targets have no heap at all. The usual answer is `std::array` or plain arrays sized at compile time, trading runtime-sized layers for fully static memory use.
- **Real-time deadlines.** Inference inside a control loop has a hard budget; the book's example is a 1 kHz motor controller, giving sensor read, inference and actuation 1 ms in total.
- Also creditable: the need to quantize before deployment, and the absence of a filesystem or of `printf()` for diagnostics.

(1 mark)

Multiply-accumulate count. A conv layer performs roughly $inputSize^2 \times kernelSize^2$ multiply-adds per feedforward pass:

$$32 \times 32 \times 3 \times 3 = \mathbf{9{,}216}\ \text{MACs}$$

Why worst case, not average. A hard deadline is missed by a single late iteration; it is not an average that has to be met but every instance. A network that is usually fast and occasionally spikes still fails, and it fails in the way casual testing does not reveal, because the average looks fine and the spike is rare. Scheduling has to be budgeted from the worst case, so the worst case is the number that has to be measured. *(1 mark)*